South Valley University
Faculty of Engineering
Electrical Engineering Department

Hospital Management System
(Web & Mobile Applications)

Supervised By

Dr. Ahmed Abdel-Basset Donkol

Presented to

Electronics and Communications Engineering Division

Presented By

Abdel-Rahman Adel Kamal
Abdel-Sabour Gamal Abdel-Sabour
Ahmed Hasan Muhammed
Donia Mahmoud Abdel-Fattah
Mahmoud Ahmed Muhammed Sayed
Mahmoud Wael Mahmoud
Osama Fathy Muhammed
Sherif Muhammed Saady
Wafaa Abdel-Nasser Abd-Allah

Submitted for the fulfillment of bachelor's degree in
Electronics and Communications Engineering

JUNE, 2025

ABSTRACT

In the rapidly evolving healthcare sector, the demand for efficient, accurate, and scalable digital solutions has become critical. Traditional hospital systems often operate in silos, leading to fragmented patient care, redundant workflows, and delayed medical decisions. This project addresses these challenges through the development of **MediCare**—a unified Hospital Management System (HMS) deployed on web and Mobile platforms. MediCare integrates core clinical and administrative functions, including centralized patient health records, appointment scheduling, pharmacy inventory management, and automated billing.

The system was developed using agile methodology and modular architecture to ensure scalability and adaptability. The frontend leverages HTML, CSS, JavaScript, and the Bootstrap framework (web) and Flutter (Mobile) for responsive cross-device compatibility, while the backend employs Django (Python) with RESTful APIs for seamless module integration. Data security and role-based access were prioritized through JWT & session authentications and SQLite database. Key features include multi-role dashboards (doctors, admins, patients, pharmacist, lab worker), and automated inventory alerts.

Testing and validation was performed both manually through the tutorials and by automated testing of the API through the Postman module. The mobile interface has been tested with Flutter testing tools on various Android versions to ensure a stable implementation. The system was implemented with PythonAnywhere.com for the web application and was tested for performance in real time in different usage scenarios.

The project achieved its objectives by setting up a live interactive hospital management platform which improves patient care, reduces staff errors and improves the efficiency of hospital workflow. Future improvements may include integration with artificial intelligence-powered diagnostics, telemedicine and cloud-based health-care systems, in order to further advance the digital transformation of healthcare.

ACKNOWLEDGMENT

In the Name of Allah, the Most Gracious, the Most Merciful

"And say, 'My Lord, increase me in knowledge.'"

— Surah Taha, 20:114

All praise is due to Allah, the Lord of all worlds. With His infinite mercy and boundless grace, we were granted the strength, patience, and insight to bring this project to completion. It is through His guidance that every step of this journey became possible, and to Him belongs all thanks and gratitude.

We extend our heartfelt appreciation to our beloved families, whose unwavering support, prayers, and constant encouragement have been a source of inspiration throughout our academic path. Their love has been our anchor during times of difficulty and our motivation to strive for excellence.

We are deeply honored to express our sincere gratitude to our esteemed supervisors, **Dr. Ahmed Abdel-Basset Donkol, Dr. Ayman Mohammed Ismail, Eng. Mohammed Basha** and **Eng. Omar Ahmed**, for their exceptional guidance, insightful feedback, and continuous support. Their expertise and dedication have been instrumental in shaping the direction and success of this project. Working under their supervision has been both a privilege and a learning experience we will always cherish.

We are also thankful to all our respected professors and mentors who have imparted their knowledge and wisdom to us throughout our studies. Each lecture, discussion, and lesson has played a vital role in our personal and academic growth.

May Allah reward everyone who supported us in this journey, bless them abundantly, and guide us all toward further success.

All praise and thanks be to Allah, by whose grace all good things are completed.

TABLE OF CONTENTS

CHAPTER 4: SYSTEM IMPLEMENTATION

CHAPTER 5: TESTING AND EVALUATION

CHAPTER 6: CONCLUSION AND FUTURE WORK

LIST OF FIGURES

LIST OF TABLES

CHAPTER ONE
Introduction

Introduction

1.1 Overview of the Project

This project introduces a robust and scalable web and mobile Hospital Management System (HMS) designed to transform and streamline healthcare services through digital innovation. By leveraging modern technologies and open-source frameworks, the system integrates various healthcare functionalities into a centralized platform that serves doctors, patients, administrators, lab workers, and pharmacists efficiently.

The web application backend is built using the Django web framework, supported by REST APIs for exchanging data with mobile applications which are built by Flutter, and secure third-party integrations such as payment gateways and email services. It allows patients to seamlessly book appointments, purchase medicines online, receive lab reports, and access prescriptions — all from the comfort of their homes.

On the backend, the system is designed to handle complex hospital operations including doctor registration and verification, department and staff management, and role-based access control. The goal of this project is not only to digitalize healthcare interactions but also to ensure data consistency, improve communication across departments, and enhance the overall patient experience.

Furthermore, MediCare-System ensures data integrity, secure patient record handling, and the ability to scale across multiple hospitals. It embodies the future of healthcare management by promoting accessibility, transparency, and operational excellence across the entire healthcare ecosystem.

1.1.1 What is a Hospital Management System (HMS)

Hospital Management System (HMS) is a comprehensive, integrated information system designed to manage all aspects of a hospital's operations. These include administrative, medical, legal, and financial components. HMS helps hospitals streamline patient care, enhance workflow efficiency, reduce manual paperwork, and ensure smooth communication between departments.

1.1.2 Importance of Digital Healthcare Solutions

Digital healthcare solutions are crucial in today's fast-paced world where efficiency, accuracy, and accessibility are paramount. By digitizing healthcare operations, hospitals can improve patient outcomes, reduce errors, ensure better data management, and enable remote access to health records. They also help healthcare providers offer more personalized and timely care.

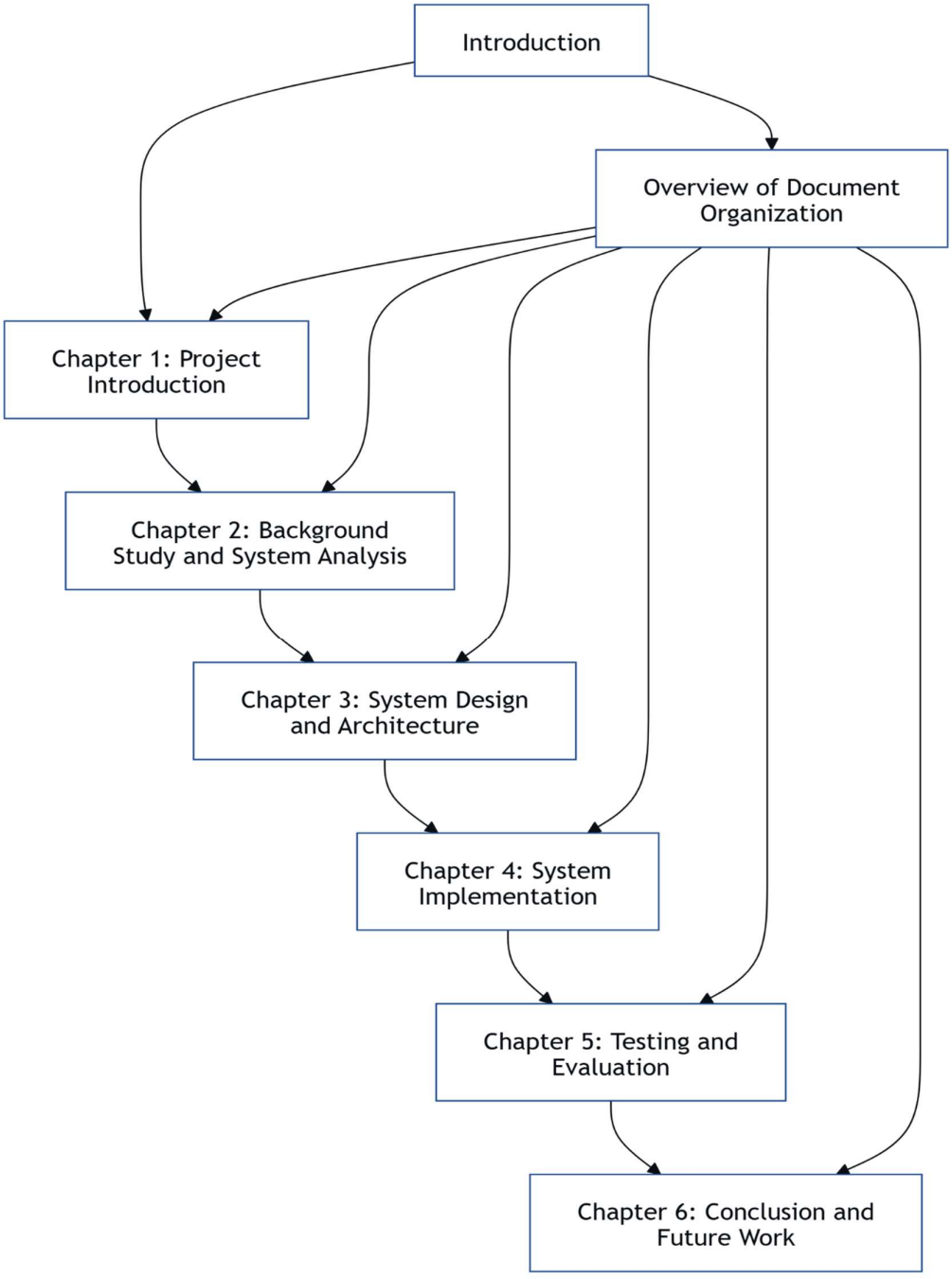

Figure 1.1: Overview of the Chapters

1.1.3 Core Functionalities of a Modern HMS

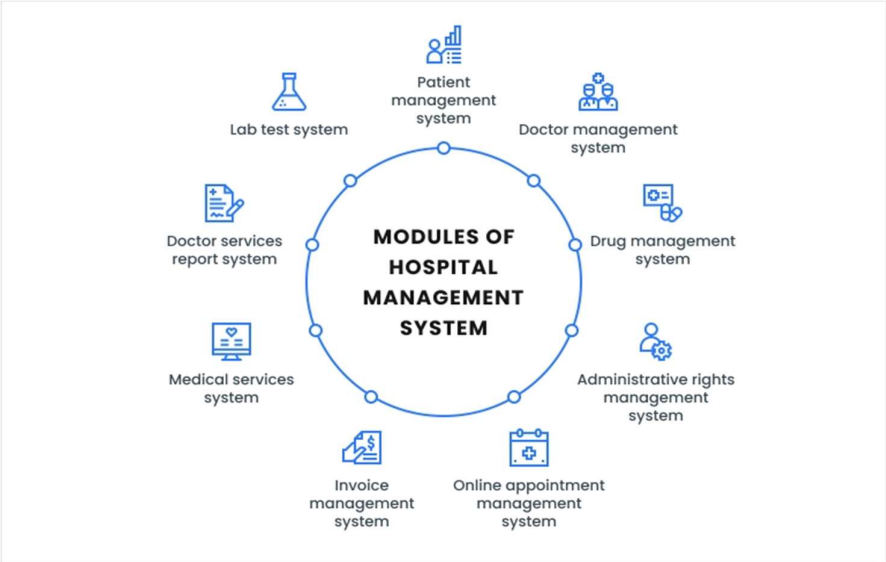

Figure 1.2: Modules of Hospital Management System

A modern Hospital Management System (HMS) typically encompasses a range of core functionalities designed to streamline operations and enhance patient care. These include:

1. Patient Lifecycle Management & Information Access:
   - Patient registration and onboarding
   - Hospital information and directory services, including profiles detailing departments and doctor listings
   - Appointment booking and scheduling

2. Clinical Data Management:
   - Electronic Health Records (EHR) management
   - Prescription generation and management
   - Laboratory test management, including report uploads and access

3. Financial and Administrative Operations:
   - Online payment processing for services
   - Billing and invoicing

4. Resource and Staff Management:
   - Pharmacy and inventory management
   - Dedicated portals or panels for doctors and hospital administrators

5. System-Wide Features:
    - Role-Based Access Control (RBAC) to ensure secure access for different user types (e.g., doctors, pharmacist, lab worker, administrators, patients).

1.1.4 Trends in HealthCare Technology

Modern healthcare is undergoing a revolutionary transformation driven by emerging technologies. These advancements are enhancing the quality of patient care, optimizing hospital operations, and creating new models for patient engagement. Below are some of the most impactful trends shaping the future of healthcare:

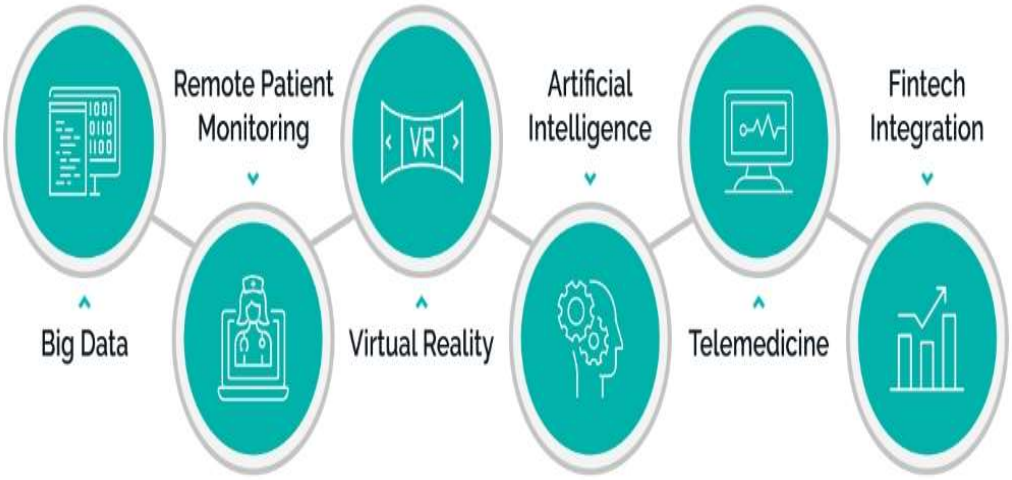

Figure 1.3: Trends in HealthCare Technologies

1. Big Data: Enables the analysis of massive health datasets to identify patterns, improve diagnoses, and enhance patient outcomes through data-driven insights.

2. Remote Patient Monitoring (RPM): Allows tracking of patient health metrics from home, reducing hospital visits and improving chronic disease management.

3. Virtual Reality (VR): Used for pain management, therapy, surgical training, and patient education, creating immersive and effective experiences.

4. Artificial Intelligence (AI): Supports clinical decision-making, diagnostic accuracy, and personalized treatment through advanced machine learning models.

5. Telemedicine: Expands access to healthcare by allowing remote consultations, follow-ups, and real-time communication between doctors and patients.

6. Fintech Integration: Simplifies medical billing, insurance processing, and digital payments, making healthcare transactions faster and more transparent.

1.1.5 Different HMS Types Calls for Different Cost-Ranges

The actual hospital management system development cost largely depends on the system's complexity or the platform that is used for its development. Here's a general overview of the development costs for different types of hospital management software

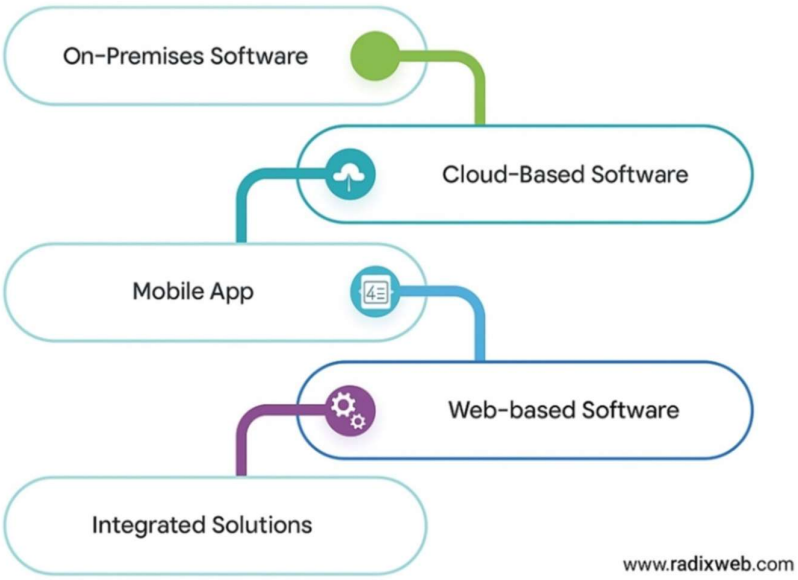

Figure 1.4: Types of Hospital Management Software

1. On-Premises Software: creating an on-premises hospital software systems, such as Electronic Health Records Software, may require a significant capital investment because the need for hardware, infrastructure, and additional maintenance costs must be factored in. E-Prescribing software or medical image analysis are a few healthcare software types that can fall into this category.

2. Cloud-Based Software: cloud hospital management systems are generally considered a better financial decision as applications requiring high infrastructure and hardware investment.

3. Mobile Apps: developing a mobile app for hospital management, like the ones for smartphones that run on iOS and Android. Mobile apps can be deployed, installing extra benefits for healthcare providers and patients.

4. Web-based Software: web-based HMS are broadly cheap because the costs are very low for ones with a few features, and high only for complicated ones with high features. Internet-enabled software has users relying heavily on it regardless of location and device. This is one of the reasons healthcare providers consider this solution as their ideal healthcare web development solutions.

5. Integrated Solutions: implementing an integrated system that aggregates the whole commission of hospital management software, including EHR, billing, and scheduling, can be more complicated and costly.

1.2 Problem Statement and Motivation

The healthcare sector, while being critical to societal well-being, is prone to operational inefficiencies caused by outdated or fragmented management systems. Traditional hospital management routines, founded on manual procedures, paper-based records, and heterogeneous software solutions, present significant challenges. These challenges directly impact the quality of care, operational cost, and overall efficiency of healthcare provision.

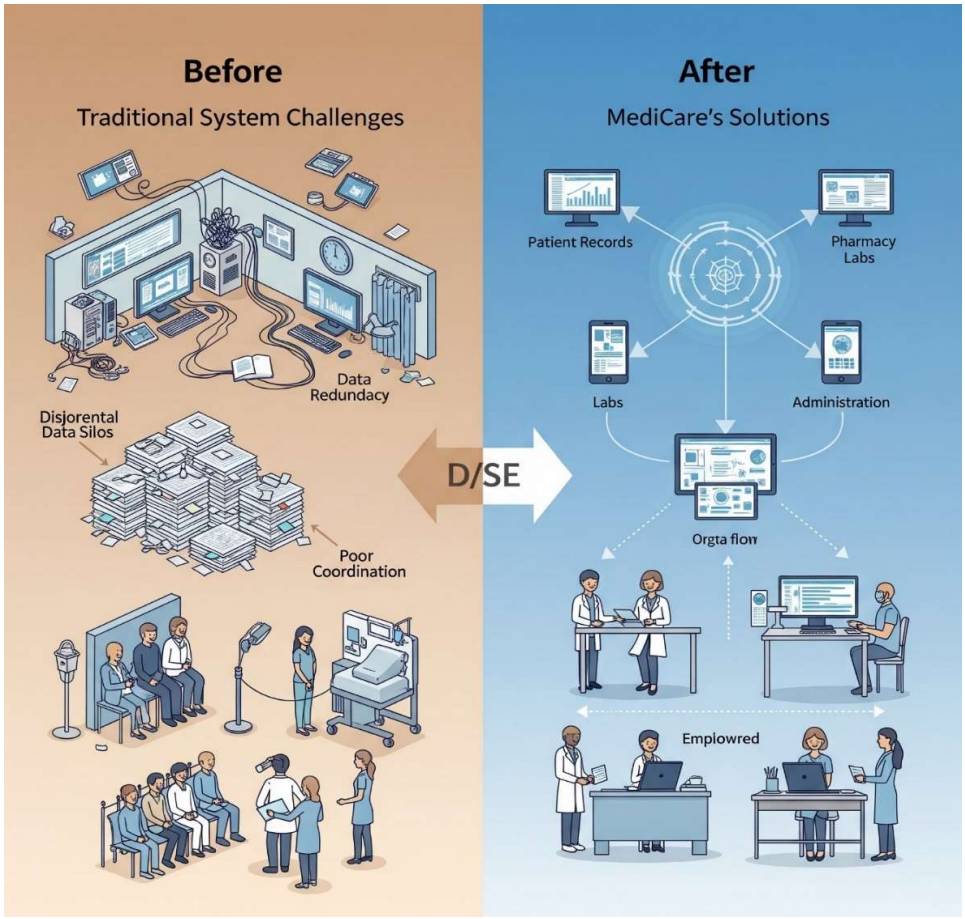

Figure 1.5: An Infographic depicting the Before and After Scenarios

Current practices in hospital management, especially in environments that lack integrated digital solutions, are plagued by several core issues:

1. Fragmented and Inaccessible Patient Data: Patient data tends to be siloed, hard to access, susceptible to loss or destruction, and hard to exchange securely between healthcare providers. This can result in inaccurate medical histories, duplicated tests, and possible medical mistakes.

2. Poor Appointment Scheduling: Manual or semi-automated appointment systems are time-consuming for both patients and staff. Patients face long waiting times or difficulty scheduling appropriate slots, whereas staff have to spend a lot of time managing schedules, leading to potential under-utilization of resources.

3. Complex and Error-Prone Billing Processes: Manual billing and invoicing are labor-intensive, subject to human mistake, and can lead to delayed revenue collection and errors that frustrate patients and strain hospital finances.

4. Inefficient Staff and Resource Management: Manual management of staff schedules, payroll, and performance metrics is a waste of time. Similarly, not keeping track of medical supplies and equipment efficiently can result in stockouts of essential items or wastage of resources, impacting patient care and costs.

5. Restricted Access to Real-time Information for Decision Making: In the absence of a central system, it is not possible to produce comprehensive reports on auditing, financial analysis, and strategic decision-making, thereby impacting the ability of the administrators to streamline operations and improve delivery of services.

6. Restricted Patient Engagement and Access to Information: Patients lack direct access to their health information or easy-to-use tools to manage their healthcare experience, such as online appointment scheduling or accessing their medical history, resulting in a less patient-centric experience.

The motivation to develop this powerful Hospital Management System (Web Application) with a companion Mobile application for patients stems from the desire to correct these aforementioned issues and utilize the strengths of modern technology in an effort to revolutionize healthcare delivery.

1. Enhancing Patient Care Quality: By centralizing patient records, streamlining appointment scheduling, and providing easier access the information, the system aims to improve diagnostic accuracy, and reduce waiting times.

2. Improving Operational Efficiency: Automating key administrative and clinical processes such as billing, staff scheduling, and inventory management will significantly reduce manual effort, minimize errors, and optimize the utilization of hospital resources, leading to cost savings and smoother workflows.

3. Empowering Stakeholders: The system will empower all of the stakeholders:
   - Patients: Through the Mobile application, patients gain control over booking appointments, viewing their records, and accessing hospitals and doctors' information, enhancing their engagement and satisfaction.
   - Doctors and lab workers: With easy access to patient data, efficient scheduling tools, and streamlined workflows, clinical staff can focus more on patient care.
   - Administrators: Real-time data and detailed reporting will facilitate informed decision-making, improved utilization of resources, and enhanced financial management.

4. Utilizing Existing Technologies: The project offers a chance to utilize and combine existing web and mobile technologies (like (HTML, CSS, JavaScript with Bootstrap) for the frontend, the Django backend framework, RESTful APIs, JWT & session authentications, and Flutter for mobile app) to create a stable, manageable, and secure solution that will adhere to existing industry's best practices.

5. Addressing a Real-World Need: Developing an affordable HMS is an appropriate and relevant issue. The background to the project lies in a desire to contribute towards providing a practical solution as a positive contribution to healthcare facilities' operational effectiveness and the quality of patient care experience.

6. Academic and Skills Development: Being academic in nature, this project forms a significant exercise as the project team can relate theoretical lessons gathered from studying Engineering to a complex, real-world software development lifecycle that includes design, implementation, testing, and deployment.

1.3 Overview of Web and Mobile Development

Web development involves the creation and maintenance of websites and web applications. It consists of two main parts: the front end and the back end. Each part has specific languages and technologies associated with it, as well as a database component to store and manage data. It defines the interactions between applications, middleware systems and databases to ensure multiple applications can work together. To manage these components, software engineers devised web application architecture to logically define relationships and manner of interactions between all of these components for a web app.

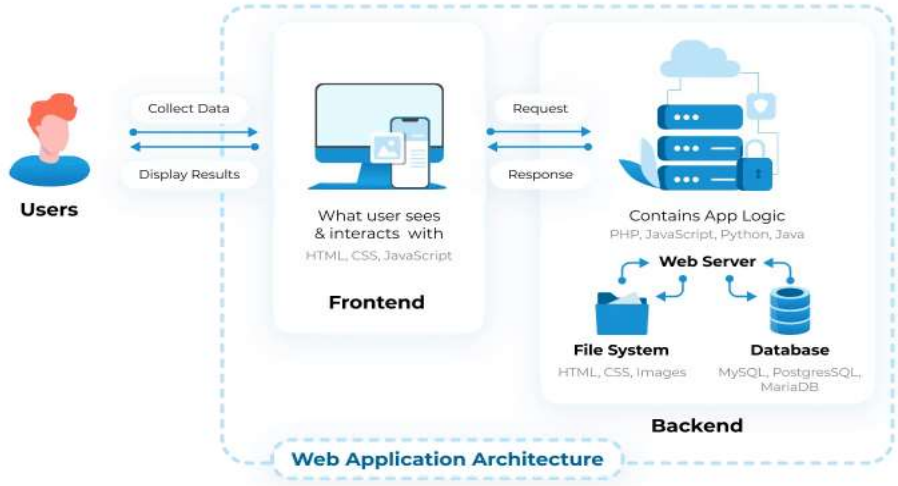

Figure 1.6: Web Architecture

Mobile application development with cross-platform frameworks like Flutter, involves the creation and maintenance of applications designed to run on multiple mobile operating systems, primarily Android and iOS, from a single codebase. This process also encompasses two primary aspects: the User Interface (UI), which defines the application's visual presentation and user interaction, and the application logic, which manages data handling, business rules, and communication with device features or external services.

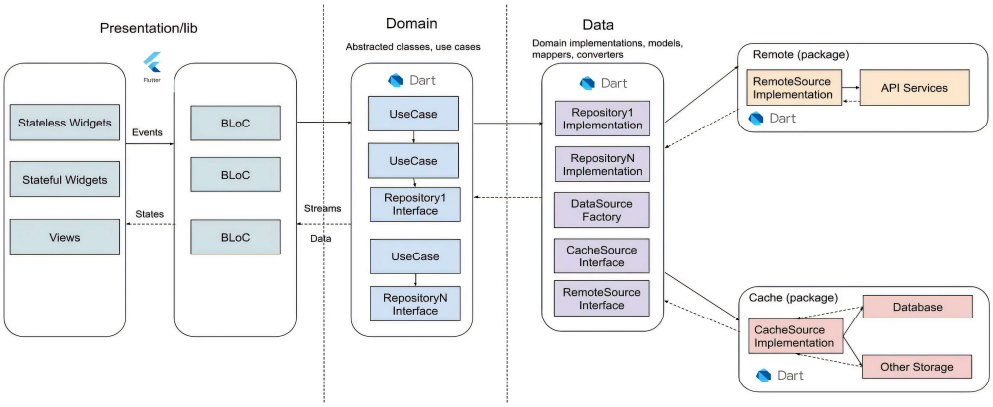

Figure 1.7: Mobile Architecture with Flutter

1.3.1 Frontend Development

Frontend or client-side development focuses on the aspects of a web application that users directly see and interact with in their browsers. This includes the visual presentation layout, design, text, and images as well as all interactive functionalities. Core of frontend development is HTML for structuring content, CSS for styling and visual appearance, and JavaScript for creating dynamic and interactive user experiences. These foundational technologies were enhanced by the Bootstrap UI framework to efficiently build responsive and complex user interfaces, rather than employing a separate advanced JS framework like Reak, Angular, or Vue.js for the primary web frontend.

1.3.2 Backend Development

Backend or server-side development encompasses the non-visible components of a web application that manage server operations, application logic, and database management. Its core functions include processing business logic, interacting with databases, ensuring user authentication, and overseeing server configuration. Common backend technologies include languages such as PHP, Python (with popular frameworks like Django and Flask), Node.js, a JavaScript runtime, is also used for building network applications, typically with frameworks like Express.js to provide structure and features.

1.3.3 Databases

Databases are fundamental to web development, providing the means to store, retrieve, and manage data for web applications. Different database types cater to specific requirements and use cases. Relational databases, such as MySQL, PostgreSQL, and SQLite, store data in structured tables (rows and columns) and are queried using SQL (Structured Query Language). In contrast, NoSQL databases are designed for data models beyond tabular relations, often used for large-scale, unstructured, or semi-structured data. Examples of NoSQL databases include document-oriented systems like MongoDB, high-performance in-memory databases such as Redis, and graph databases like Neo4j for interconnected data.

1.3.4 Mobile Application Development

Mobile application development is now a critical aspect of modern software, focusing on creating applications for smartphones and tablets. These apps typically provide a faster, more responsive, and native user experience compared to web-based alternatives. Developers can opt for native development, using languages like Java and Kotlin for

Android, or Swift and Objective-C for iOS. Alternatively, cross-platform frameworks allow for a single codebase to target multiple operating systems; prominent examples include Flutter (a UI toolkit from Google using Dart), React Native (which employs JavaScript and React), and Xamarin (a Microsoft framework using C#). A key architectural consideration is the communication between mobile apps and backend services, typically facilitated through APIs, allowing them to share server-side logic and databases with web applications for a consistent, integrated user experience.

1.4 Project Objectives

The primary objectives of this Hospital Management System project are:

1. To design a comprehensive, integrated Hospital Management System have both:
   - A centralized, feature-rich web-based platform for overall hospital operations.
   - A dedicated Android mobile application to enhance patient engagement.

2. To empower patients by providing secure access to healthcare services, including:
   - Web Platform: Accessing detailed medical history, booking appointments, viewing prescriptions and lab reports, and utilizing online pharmacy services.
   - Android Application: Booking and managing appointments, viewing hospital profiles, browsing departments and doctor listings, and updating personal account information.

3. To equip doctors and clinical staff with efficient tools for patient care and schedule management, enabling them to:
   - Manage their appointment schedules effectively.
   - Securely access and update patient electronic health records (EHR).
   - Create, manage, and view patient prescriptions.

4. To provide hospital administrators with robust tools for complete operational control and strategic decision-making, including:
   - Managing staff schedules, and performance metrics.
   - Organizing and overseeing hospital departments.
   - Administering user roles and access permissions across the system.
   - Implementing an automated billing and invoicing system integrated with patient visits and services.

- Managing inventory of medicines, medical equipment, and other resources.
- Generating comprehensive medical, financial, and administrative reports.

5. To ensure a secure, reliable, and user-friendly interface for all stakeholders (patients, doctors, pharmacist, lab worker, administrators) across both the web and mobile platforms, prioritizing data privacy and ease of use.

6. To build a scalable and maintainable system architecture that can accommodate future growth in data, user load, and functionality.

7. To integrate essential third-party services where appropriate, such as secure payment gateways for online transactions and automated notification systems (e.g., email, SMS) for appointments, alerts, and other communications.

Figure 1.8: Some Objectives

1.5 Scope of the Project

The "MediCare" project encompasses the design, development, and deployment of an integrated, multi-hospital Hospital Management System. This system is comprised of a comprehensive web-based platform catering to various user roles (patients, doctors, hospital administrators, lab workers, pharmacists) and overall management, alongside a dedicated Android mobile application focused on patient engagement and core functionalities. The system architecture relies on RESTful APIs for communication between the frontend components (web and mobile) and the backend services.

The overall system scope includes:

1. Development of a robust backend supporting a wide range of HMS functionalities across multiple hospitals.

2. Implementation of REST APIs to facilitate data exchange and interactions for both the web and mobile applications.

3. Utilization of a database (SQLite) for data storage, with considerations for future scalability to cloud-based database solutions.

4. Integration with essential third-party services, including an automated email notification system (using Mailtrap for development) for key events like appointment management and doctor registrations, and integration with payment gateways like PayPal for online transactions.

5. Support for generating PDF documents like prescriptions and lab reports.

6. Design for future scalability in terms of user load, data volume, and feature expansion.

Figure 1.9: Features Overview

1.5.1 Web Application Scope

The web application component will serve as the primary interface for hospital staff, administrators, and will also offer extensive functionalities for registered patients. Public-facing pages will provide general information about hospitals and services. The scope includes:

1. Public-Facing Information Portal:
    - Homepage showcasing the platform's services.
    - Browsable listings of hospitals, their departments, and associated doctors.
2. Multi-User Role Management & Functionalities:
    - Patients:
        - Secure account registration, login, and profile management.
        - Viewing detailed information of multiple hospitals, including emergency contact and doctor lists.
        - Searching for departments and doctors within hospitals.
        - Booking and managing appointments, with email confirmations.
        - Viewing personal prescriptions and lab reports.
        - Booking lab tests associated with prescriptions and facilitating online payment for these tests.
        - Accessing an online "Medical Shop" for medicines and purchase online.
    - Doctors:
        - Secure account registration to a specific hospital.
        - Personalized dashboard.
        - Managing appointments: accepting or rejecting patient appointment requests, with email notifications sent to the patient upon status change.
        - Viewing profiles of patients whose appointments are accepted.
        - Creating detailed digital prescriptions for patients.
        - Managing personal profile settings.
    - Hospital Administrators:
        - Dashboard for an overview of hospital operations.
        - Managing doctor registrations: accepting or rejecting doctor applications, with email notifications sent to the doctor.
        - Managing Lab Workers: viewing and adding new lab worker accounts.
        - Managing Pharmacists: viewing and adding new pharmacist accounts.
        - Comprehensive Hospital Management: Adding, editing, and viewing detailed hospital information, including name, address, contact details, etc.
    - Lab Workers:
        - Personalized dashboard.
        - Viewing pending lab reports based on prescriptions.

- Creating and submitting detailed lab reports, including specimen details, test results, units, and reference values.
- Managing a list of available tests and their information.

- Pharmacists:
  - Managing the online "Medical Shop" inventory: adding new medicines, editing existing medicine information, and deleting medicines.
  - Searching and viewing medicine information.

3. User Interface and Experience:
   - A responsive web interface with intuitive navigation designed for ease of use across various devices and for all user roles.

4. Security and Access Control:
   - Robust role-based access control (RBAC) ensuring users only access features and data relevant to their roles.

1.5.2 Android Application Scope

The Android mobile application is designed exclusively for patients, providing a streamlined and convenient interface for accessing essential healthcare services and personal health information. The scope includes:

1. Patient Account Management:
   - User registration for new patients.
   - Secure login functionality.
   - Password recovery mechanism ("Forgot Password").
   - Ability for patients to view and edit their personal profile data.

2. Information Browsing and Discovery:
   - A home screen interface providing an overview, including quick access to departments and their doctors.
   - Browsing and viewing detailed profiles of all registered hospitals, including their departments and lists of associated doctors.
   - Browsing and viewing detailed profiles of all registered doctors, including their specialization and affiliated hospital/department.

3. Appointment Management:
   - Functionality to search for doctors and book appointment based on availability.

- Viewing a list of booked (upcoming, confirmed and cancelled) appointments.

4. Access to Medical Information:
   - Secure viewing of personal prescriptions.
   - Secure viewing of personal lab reports generated via the web platform.

5. User Interface and Experience:
   - A cross-platform interface designed for optimal performance, ease of navigation, and user experience on mobile devices.

6. Backend Communication:
   - Secure and efficient communication with the "MediCare" backend via REST APIs to fetch and submit data.

1.6 Target Audience

The "MediCare" Hospital Management System is developed for the following key audiences, offering distinct advantages to each:

1. Patients: Seeking easy and remote access to medical services, appointment booking, personal health records, and hospital/doctor information via web and mobile platforms.

2. Doctors & Clinical Staff: Needing an efficient platform for managing patient appointments, accessing EHRs, and issuing prescriptions.

3. Hospital Administrators: Requiring comprehensive tools for operational control, staff and department management, and data-driven decision-making.

4. Pharmacists & Lab Workers: Utilizing the system for streamlined medicine inventory management, prescription processing, and lab test/report management.

5. Healthcare Providers (Hospitals, Clinics, Startups): Aiming to modernize their operations, enhance patient experience, and improve service delivery through a scalable and affordable digital solution.

6. Academic & Development Community: Individuals studying or building health-tech solutions, particularly those interested in Django and Flutter-based platforms for hospital management.

1.7 Structure of the Report

This report is systematically organized into six main chapters, followed by References and Appendices, to comprehensively detail the design, development, and evaluation of the "MediCare" Hospital Management System.

1. Chapter 1, which you are currently reading, introduces the project. It has covered an overview of the project, defined what a Hospital Management System is, highlighted the importance of digital healthcare solutions, outlined core HMS functionalities and current trends, presented the problem and motivation, given an overview of web and mobile development technologies, and clearly stated the project objectives, scope (web and mobile applications), and target audience.

2. Chapter 2 will delve into the Background Study and System Analysis. This chapter will begin by examining the challenges inherent in traditional hospital management systems. It will present a literature review and comparative study of existing HMS solutions. Following this, a detailed requirements analysis will be conducted, covering functional, non-functional, and user requirements. It will also thoroughly discuss the selected technology stack for frontend, backend, database, authentication, API design, and mobile application development. Finally, it will address security and privacy considerations pertinent to healthcare applications.

3. Chapter 3 focuses on System Design and Architecture. This section will outline the project planning and methodology adopted. It will then detail the overall system architecture, including specific architectures for the web and mobile applications. A significant portion will be dedicated to database design, featuring Entity-Relationship Diagrams (ERD) and the database schema. Data Flow Diagrams (DFDs) and Use Case Diagrams will also be presented, along with the User Interface (UI) and User Experience (UX) design principles.

4. Chapter 4 describes the System Implementation phase. It will start with the setup of the development environment. Subsequent sections will detail the backend implementation, including API development, business logic, and database integration. The frontend implementation for the web application will be covered extensively, module by module (Patient Registration & Management, Appointment Scheduling, Billing & Invoicing, Staff Management, Inventory Management, and Report Generation). The implementation of the patient-focused Android mobile

5. application will also be detailed, covering user authentication, information browsing, appointment management, and API integration. This chapter will also discuss the implementation of Role-Based Access Control (RBAC), overall security measures, and the deployment of the application on PythonAnywhere.com.

6. Chapter 5 is dedicated to Testing and Evaluation. This chapter will outline the testing strategy and approach. It will then describe the various testing phases, including unit testing (frontend and backend), integration testing, system testing, User Acceptance Testing (UAT), and performance testing. Specific test cases and their results for both the web and mobile will be presented, along with the processes for bug reporting and resolution. The chapter will conclude with an evaluation of the project's outcomes against the initially defined objectives.

7. Finally, Chapter 6 provides the Conclusion and Future Work. This concluding chapter will offer a summary of the project's achievements, discuss challenges encountered and lessons learned during the development process, and candidly address the limitations of the current system. It will then propose potential future enhancements and expansions for both the web and mobile applications, as well as possibilities for integration with other systems, followed by overall final reflections.

The report will conclude with a list of References cited throughout the document and Appendices containing supplementary materials such as sample code snippets, screenshots of the application interfaces, user manuals, detailed ER and DFD diagrams, and an optional project timeline.

CHAPTER TWO

System Analysis and Background

System Analysis and Background

2.1 Challenges in Traditional Hospital Management

Traditional hospital management systems often suffer from inefficiencies and fragmentation. Many hospitals rely on manual record-keeping or partially digitized systems, leading to several issues:

1. Data Redundancy and Inconsistency: Paper-based or outdated systems frequently result in duplication of patient records and inconsistent data across departments.

2. Delayed Access to Information: Staff may spend valuable time retrieving physical files or switching between non-integrated digital systems.

3. Poor Coordination Between Departments: Lack of real-time communication and centralized data can hinder collaboration between doctors, labs, and admins.

4. Security and Compliance Risks: Without secure digital infrastructure, sensitive patient data may be vulnerable to breaches or fail to meet data protection standards.

5. Limited Patient Engagement: Traditional systems offer minimal support for patients to interact with their own health data, book appointments online, or communicate with providers.

These challenges highlight the urgent need for a centralized, intelligent, and secure hospital management solution.

2.2 Literature Review / Comparative Study of Existing HMS Solutions

The landscape of Hospital Management Systems (HMS) is characterized by a diversity of solutions that vary significantly in their architecture, deployment models, feature sets, and consequently, their development and operational costs. A clear understanding of these fundamental types is essential for contextualizing existing market offerings and informing the design choices for new systems. Generally, HMS development can be categorized based on the platform and deployment strategy, each presenting distinct financial and infrastructural implications (as previously illustrated in Figure 1.3, Chapter 1). It is important to note that the following cost estimates, while based on typical industry ranges, can fluctuate considerably depending on specific project scope, vendor engagement, geographic location, regulatory requirements, and the scale of the healthcare institution.

1. On-Premises Software: Many healthcare institutions have opted for on-premises HMS. This model, often encompassing full-featured Electronic Health Record (EHR) systems or sophisticated diagnostic imaging platforms, typically incurs higher development costs. These costs are driven by system complexity, the need to meet stringent regulatory compliance (such as HIPAA), and frequently, the requirement for custom interfaces and integrations. For small to mid-sized hospitals, the average development expenditure for such systems generally falls within the range of $120,000 to $250,000, contingent upon the number of modules included and the overall depth of system functionality.

2. Cloud-Based Software (SaaS - Software as a Service or PaaS - Platform as a Service): Leveraging the advantages of reduced on-site infrastructure requirements and modern development frameworks that facilitate modular design, cloud-hosted HMS platforms have gained considerable traction. The average development cost for cloud-based solutions ranges from $60,000 to $400,000. This variation is influenced by factors such as the complexity of data handling and migration, the number of distinct user roles and their associated permissions, and the implementation of robust security layers necessary for compliance with standards.

3. Web-Based Software (Custom Development): Web-based HMS, accessible via standard web browsers, offer significant flexibility and cross-device compatibility. These platforms are often considered more cost-effective in their development cycle. Average development costs for custom web-based HMS are estimated to be between $45,000 and $350,000. The final cost is largely determined by the intricacy of user dashboards, the sophistication of reporting and analytics tools, and the extent of integrations required with third-party APIs.

4. Mobile Applications (Apps): Designed to complement broader HMS ecosystems, dedicated mobile applications for smartphones and tablets—catering to either patients (e.g., for appointment check-ins, viewing test results) or healthcare providers (e.g., for mobile charting, secure messaging)—are generally less expensive to develop on a per-platform basis. The average development costs for such hospital mobile apps typically range from $25,000 to $180,000. Key cost drivers include the number of target platforms (iOS, Android, or cross-platform), the need for offline capabilities, real-time data synchronization mechanisms, and the level of polish and sophistication in the UI/UX design.

5. Integrated HMS Solutions (Enterprise-Grade Platforms): Representing the most comprehensive approach, they are typically enterprise-grade platforms that bundle all core hospital modules—such as EHR, billing, pharmacy management, laboratory information systems, radiology information systems, human resources, and advanced scheduling—into a single, cohesive system. For small to mid-sized healthcare facilities, the development costs range from $150,000 to $1,000,000. The final investment is heavily dependent on the degree of integration between modules, the overall feature richness, and the customization required to meet specific institutional workflows.

This understanding of different HMS archetypes and their associated cost structures provides a foundational framework for evaluating specific existing HMS solutions. Such an analysis should consider not only their functional capabilities but also their underlying architectural, deployment, and financial models. In recent years, several Hospital Management Systems (HMS) have been developed to support healthcare institutions. Among the most recognized systems are OpenMRS, Bahmni, and eHospital:

1. OpenMRS is an open-source HMS widely used in clinics and developing countries. It offers flexible customization and basic patient record management. However, it lacks advanced features such as pharmacy management, billing, real-time communication between doctors and patients, and native cloud support.

2. Bahmni builds on OpenMRS by integrating pharmacy, laboratory, and billing modules. While it provides more functionality than OpenMRS, it still lacks support for advanced analytics, AI-powered features, and real-time communication tools. Additionally, it is not fully cloud optimized.

3. eHospital, developed by the National Informatics Centre (NIC) of India, is a government-level HMS used in public hospitals. It supports electronic medical records, billing, and department coordination. However, it is not intended for private institutions, lacks flexibility for customization, and does not include features like AI-based decision support or patient-doctor chat systems.

The "MediCare" project, detailed in this report, is designed to overcome these shortcomings. As described in the Abstract, "MediCare" is conceived as an integrated web (Bootstrap/Django) and mobile (Flutter) solution deployed on PythonAnywhere.com,

aiming to deliver a comprehensive feature set while maintaining cost-effectiveness. "MediCare" addresses the gaps in existing solutions by offering features such as:

1. Flexible and secure appointment system.
2. Full cloud-based deployment and access.
3. A dynamic, role-based dashboard for doctors, patients, and administrators.
4. A system designed with data security and patient privacy as key priorities.
5. Integration of pharmacy, billing, laboratory, and appointment modules.
6. Furthermore, as noted in the project's future scope, "MediCare" is designed to potentially incorporate AI-powered insights and data analysis to further assist in decision-making.

This comparative study highlights the existing gaps in current HMS platforms and justifies the development of a more intelligent, and scalable solution like "MediCare". This approach aims to render "MediCare" a viable option for a spectrum of healthcare providers, from smaller clinics to medium-sized hospitals seeking modern, scalable digital solutions, thereby meeting the evolving needs of modern healthcare environments.

2.3 Requirements Analysis

A successful Hospital Management System (HMS) such as "MediCare" must meet several functional and non-functional requirements to ensure efficiency, scalability, and security. This section outlines these requirements, which have guided the design and development of the system. The requirements are categorized into Functional Requirements, Non-Functional Requirements, and User Requirements. For clarity and traceability, individual requirements within these categories are assigned unique identifiers (IDs).

Requirement ID	Category	Type	Associated Role	Priority
FR-PUB-001	Functional	Public-Facing	Public	High
FR-PAT-001	Functional	Patient Functionality	Patient	High
FR-DOC-003	Functional	Doctor Functionality	Doctor	High
FR-ADM-005	Functional	Admin Functionality	Hospital Administrator	High

FR-LAB-003	Functional	Lab Worker Functionality	Lab Worker	High
FR-PHM-002	Functional	Pharmacist Functionality	Pharmacist	High
FR-SYS-001	Functional	System-Wide	All	Critical
NFR-PER-001	Non-Functional	Performance	All	High
NFR-SEC-003	Non-Functional	Security	All	Critical
NFR-USA-001	Non-Functional	Usability	All	High
NFR-SCA-001	Non-Functional	Scalability	System	High
UR-PAT-001	User	Patient User Req.	Patient	High
UR-DOC-003	User	Doctor User Req.	Doctor	High
UR-ADM-001	User	Admin User Req.	Hospital Administrator	High

Table 2.1: Consolidated Requirements Analysis

2.3.1 Functional Requirements

Functional requirements define the specific behaviors and functions that the "MediCare" system must perform. Each functional requirement is assigned an ID with the prefix "FR-", followed by a three-letter code indicating the primary user or system area (e.g., PUB for Public, PAT for Patient, DOC for Doctor, ADM for Admin, LAB for Lab Worker, PHM for Pharmacist, SYS for System-wide), and a sequential number (e.g., FR-PAT-001)

1. Public-Facing Information Portal:
    - FR-PUB-001: System provides a public-facing homepage showcasing the platform's services.
    - FR-PUB-002: System allows public users to browse listings of affiliated hospitals, their departments, and associated doctors.
    - FR-PUB-003: System provides an "About Us" page with information about the MediCare platform.
    - FR-PUB-004: System provides clear login/registration pathways for different user roles (Patient, Doctor, Admin).

2. Patient Functionalities (Web & Mobile Application):
    - FR-PAT-001: System allows patients to register for a new account.
    - FR-PAT-002: System allows registered patients to securely log in and manage their profiles (view/edit personal information, change password).
    - FR-PAT-003: System allows patients to search and view detailed information of multiple hospitals, including emergency contacts and doctor lists.
    - FR-PAT-004: System allows patients to search for doctors by name, specialization, or hospital, and view their profiles.
    - FR-PAT-005: System allows patients to book appointments with available doctors and receive email confirmations.
    - FR-PAT-006: System notifies patients via email when a doctor accepts or rejects their appointment request.
    - FR-PAT-007: System allows patients to manage their appointments.
    - FR-PAT-008: System allows patients to view their personal prescriptions.
    - FR-PAT-009: System allows patients to view their lab reports.
    - FR-PAT-010: System allows patients to book lab tests associated with their prescriptions.
    - FR-PAT-011: System facilitates online payment for lab tests and other billable services using an integrated payment gateway.
    - FR-PAT-012: System provides an online "Medical Shop" where patients can search for medicines, add them to a cart, and purchase them online.
    - FR-PAT-013: System provides patients with a personalized dashboard summarizing their appointments, prescriptions, and medical records.
    - FR-PAT-014 (Mobile): Android application provides a home screen overview with quick access to departments or featured hospitals/doctors.
    - FR-PAT-015 (Mobile): Android application facilitates secure communication with the "MediCare" backend via REST APIs to fetch and submit data.

3. Doctor Functionalities (Web Application):
    - FR-DOC-001: System allows doctors to register and associate their profile with a specific hospital (subject to admin approval).
    - FR-DOC-002: System provides doctors with a personalized dashboard showing their appointments and relevant patient information.

- FR-DOC-003: System notifies doctors via email when a new appointment is booked with them.
- FR-DOC-004: System allows doctors to manage appointment requests: accept or reject patient appointments.
- FR-DOC-005: System allows doctors to view the profiles and medical history of patients whose appointments are confirmed.
- FR-DOC-006: System enables doctors to create, manage, and view detailed digital prescriptions for patients.
- FR-DOC-007: System allows doctors to search for patients within the system.
- FR-DOC-008: System allows doctors to manage their personal profile settings.

4. Hospital Administrator Functionalities (Web Application):

- FR-ADM-001: System provides hospital administrators with a comprehensive dashboard for an overview of hospital operations.
- FR-ADM-002: System allows administrators to manage doctor registrations: accept or reject doctor applications.
- FR-ADM-003: System notifies doctors via email regarding the status (approved/rejected) of their registration application.
- FR-ADM-004: System allows administrators to manage Lab Worker accounts.
- FR-ADM-005: System allows administrators to manage Pharmacist accounts.
- FR-ADM-006: System allows administrators to perform comprehensive Hospital Information Management: adding, editing, and viewing detailed hospital information.
- FR-ADM-007: System allows administrators to manage bed availability.
- FR-ADM-008: System allows administrators to generate reports relevant to hospital operations (e.g., appointment statistics as indicated by dashboard).

5. Lab Worker Functionalities (Web Application):

- FR-LAB-001: System provides lab workers with a personalized dashboard.
- FR-LAB-002: System allows lab workers to view a list of pending lab reports based on prescriptions where tests have been paid for.
- FR-LAB-003: System enables lab workers to create and submit detailed lab reports, including specimen details, test results, units, and reference values.
- FR-LAB-004: System allows lab workers to manage a list of available tests and their information (e.g., add, delete, update test prices).

6. Pharmacist Functionalities (Web Application):
    - FR-PHM-001: System provides pharmacists with a personalized dashboard.
    - FR-PHM-002: System allows pharmacists to search for and view medicine information.
    - FR-PHM-003: System allows pharmacists to manage the online "Medical Shop" inventory: adding new medicines, editing existing medicine information, and deleting medicines.
    - FR-PHM-004: System allows pharmacists to manage medicine details including name, weight, quantity, category, type, description, price, and image.

7. System-Wide Functional Requirements:
    - FR-SYS-001: System implements Role-Based Access Control (RBAC) to ensure users only access features and data relevant to their roles.
    - FR-SYS-002: System integrates with external services such as Mailtrap (or similar) for email notifications.
    - FR-SYS-003: System integrates with payment gateways like PayPal (or similar) for online transactions.
    - FR-SYS-004: System supports the generation of PDF documents for prescriptions and lab reports.
    - FR-SYS-005: System's backend (Django) shall expose RESTful APIs for seamless communication with mobile application (Flutter).
    - FR-SYS-006: System utilizes SQLite for persistent data storage.

2.3.2 Non-Functional Requirements (Performance, Security, Usability)

Non-functional requirements define the quality attributes of the "MediCare" system. They describe how the system should perform its functions. Each non-functional requirement is assigned an ID with the prefix "NFR-", followed by a three-letter code indicating the quality attribute (e.g., PER for Performance, SEC for Security, USA for Usability, REL for Reliability, SCA for Scalability, MAI for Maintainability, DEP for Deployability, REC for Recovery), and a sequential number.

- NFR-PER-001 (Performance): System should provide responsive performance for all user interactions, with acceptable load times for web pages and mobile application screens under typical concurrent user loads.
- NFR-PER-002 (Performance): API response times between the backend and frontend/mobile clients should be optimized to ensure a smooth user experience.

- NFR-SEC-001 (Security): System must ensure secure user authentication and authorization to protect against unauthorized access.
- NFR-SEC-002 (Security): Role-Based Access Control (RBAC) must be strictly enforced to ensure users can only access data and functionalities appropriate to their roles.
- NFR-SEC-003 (Security): Sensitive data, particularly patient health information and personal identifiable information (PII), must be protected. The system is designed with data security and patient privacy as key priorities. This includes employing appropriate security practices.
- NFR-SEC-004 (Security): The system should be resilient to common web vulnerabilities by secure coding practices and framework-provided protections.
- NFR-USA-001 (Usability): The web and mobile interfaces should be intuitive, user-friendly, and easy to navigate for all defined user roles, from patients to technical administrators.
- NFR-USA-002 (Usability): The system should provide clear feedback to users for their actions (e.g., success messages, error notifications).
- NFR-USA-003 (Usability): The design should be consistent across different modules and platforms (web/mobile) to facilitate ease of learning and use.
- NFR-REL-001 (Reliability): The system should be reliable and maintain high availability with minimal downtime during operational hours.
- NFR-REL-002 (Reliability - Data Integrity): The system must ensure the integrity and consistency of data stored and processed.
- NFR-SCA-001 (Scalability): The system architecture should be scalable to accommodate a growing number of users, increasing data volume, and potential future expansion of features without significant degradation in performance.
- NFR-SCA-002 (Scalability - Database): The SQLite database solution should support the projected data storage and retrieval needs. Considerations for future migration to more scalable cloud-based database solutions.
- NFR-MAI-001 (Maintainability): The system should be developed using a modular architecture and follow good coding practices to facilitate ease of maintenance, updates, and future enhancements.
- NFR-DEP-001 (Deployability): The web application should be deployable to a cloud hosting platform.

2.3.3 User Requirements

User requirements describe what users need to be able to accomplish using the "MediCare" system. These are framed from the perspective of the end-users requirements.

1. Patient User Requirements:
   - Patients need to easily create and manage their MediCare account.
   - Patients need to find information about hospitals and doctors.
   - Patients need to book, view, and manage their medical appointments online through both web and mobile platforms.
   - Patients need to access their medical information, such as prescriptions and lab reports, securely.
   - Patients need to make online payments for services and medications.
   - Patients need to purchase medicines from an online pharmacy integrated within the system.

2. Doctor User Requirements:
   - Doctors need to manage their professional profiles and hospital affiliations.
   - Doctors need to view and manage their appointment schedules.
   - Doctors need to securely access relevant patient information to provide care.
   - Doctors need to create and manage digital prescriptions for their patients.

3. Hospital Administrator User Requirements:
   - Administrators need a centralized dashboard to monitor and manage overall hospital operations.
   - Administrators need tools to manage hospital staff accounts (doctors, lab workers, pharmacists), including registrations and approvals.
   - Administrators need to manage detailed information about the hospital(s), including services, departments, and resources.

4. Lab Worker User Requirements:
   - Lab workers need to view ordered tests that require processing.
   - Lab workers need to accurately submit lab test results into the system.
   - Lab workers need to manage the list and details of tests offered by the lab.

5. Pharmacist User Requirements:
   - Pharmacists need to manage the inventory of the online medical shop, including adding, updating, and removing medicines with their associated details.

2.4 Technologies Stack Selection

The technology stack chosen for the MediCare Hospital Management System was carefully selected to meet the needs of a modern, responsive, and scalable healthcare application. For the web frontend, HTML, CSS, JavaScript, and Bootstrap were used to ensure a responsive and user-friendly interface. The mobile application was built using Flutter and Dart, allowing for cross-platform compatibility with a single codebase. On the backend, Django and Django REST Framework were selected for their rapid development capabilities, built-in security, and robust API support. Communication between the frontend and backend is handled via RESTful APIs, ensuring smooth and consistent data exchange. Authentication is managed using JWT (JSON Web Tokens) & session auth., providing secure, stateless user sessions ideal for both web and mobile environments. SQLite was used as the initial database due to its simplicity during development, with plans to upgrade to PostgreSQL for production scalability. Deployment is handled via PythonAnywhere for the web application. Git and GitHub are used for version control and collaboration, ensuring organized development workflows. Overall, this technology stack was chosen to balance performance, security, developer efficiency, and future scalability.

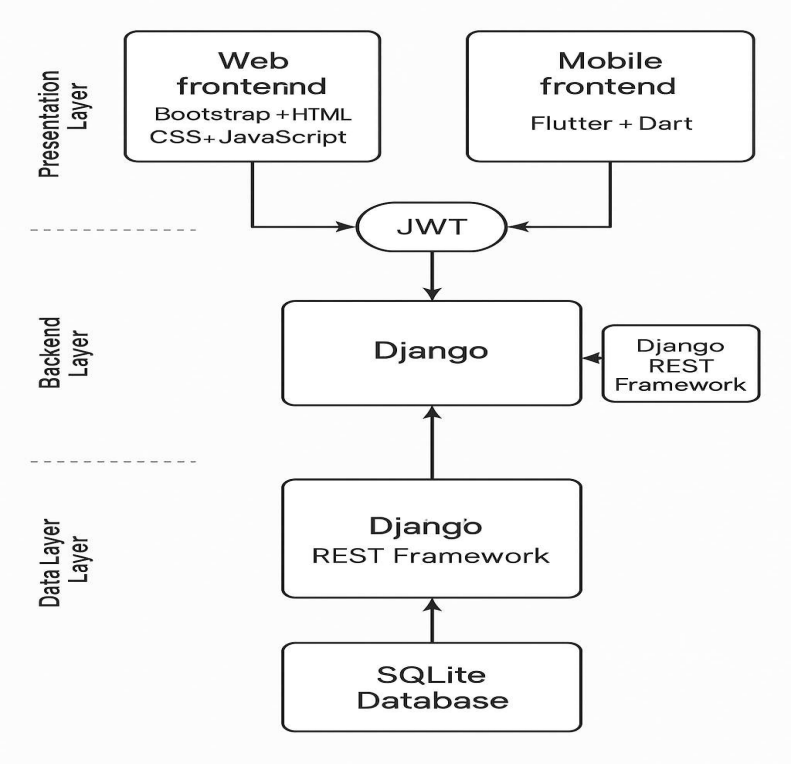

Figure 2.1: Technology Stack Diagram

2.4.1 Frontend Development Technologies (Web Application)

The frontend of the "MediCare" web application is designed to provide an intuitive, responsive, and interactive user experience for all stakeholders, including patients, doctors, and administrators, using a combination of foundational web technologies and a robust UI framework.

1. HTML5 (HyperText Markup Language 5): Serves as the core technology for structuring the content and semantic layout of all web pages within the application.

2. CSS3 (Cascading Style Sheets 3): Used for custom styling of the visual presentation, working in conjunction with the chosen UI framework to ensure a unique and professional appearance.

3. JavaScript (ES6+): Powers the client-side interactivity and dynamic behavior of the web application. This includes handling user events, DOM manipulation, and form validations.

4. Bootstrap (CSS Framework): This widely used front-end component library was the primary UI framework for the "MediCare" web application.
   - Rationale: Bootstrap was chosen for its comprehensive suite of pre-built CSS and JavaScript (jQuery-based) components (such as navigation bars, forms, modals, buttons, and carousels), its powerful responsive grid system, and its utility classes. This significantly accelerated UI development ensured cross-browser compatibility, and facilitated the creation of a mobile-first, responsive design that adapts well to various screen sizes (desktops, tablets, and smartphones). Its well-documented nature and large community support also contributed to this choice, allowing for rapid prototyping and development of a consistent user interface.

This stack was selected to build a functional and user-friendly interface efficiently, leveraging the strengths of Bootstrap for responsive design and componentry, supported by custom HTML, CSS, and JavaScript for specific application needs.

2.4.2 Backend Development Technologies

The backend is the engine of the "MediCare" system, responsible for handling business logic, data processing, user authentication, API provision, and interaction with the database.

1. Python**:** A versatile, high-level programming language known for its readability, extensive libraries, and suitability for web development.

2. Django (Python)**: A** secure and scalable web framework was selected for building the backend.
   - Rationale: Django promotes rapid development and clean, pragmatic design. Its "batteries-included" philosophy provides built-in features for an ORM (Object-Relational Mapper), admin panel, security (protection against common web attacks like XSS, CSRF), and more, which are crucial for a robust HMS.

3. Django REST Framework (DRF): A powerful and flexible toolkit for building Web APIs on top of Django.
   - Rationale: DRF was essential for creating the RESTful APIs (see Section 2.4.5) that facilitates seamless communication between the Django backend and the Flutter mobile application. It simplifies tasks like serialization, authentication, and request/response handling.

2.4.3 Database Management System

The Database is critical for storing and managing all structured data for the "MediCare" system, including patient records, appointments, staff information, and inventory.

- SQLite: To manage structured medical and administrative data efficiently.
  - Rationale: SQLite was chosen for its simplicity, serverless nature, and ease of setup, making it ideal for development and for a project deliverable of this nature. It's a single-file database that is highly portable. SQLite was the primary DBMS for the development and current scope of this project.

2.4.4 Authentication Mechanisms

Secure authentication is a critical component of the "MediCare" system, vital for protecting user accounts and sensitive data. To accommodate the different needs of the traditional web application and the mobile API, a hybrid authentication strategy was implemented, using the most appropriate method for each platform.

- Session-Based Authentication (for the Web Application):
  - For the server-rendered web application, Django's built-in session authentication framework was utilized. This traditional and highly secure mechanism uses server-side sessions and HTTP cookies to manage user login states.

- Rationale: This approach is tightly integrated with the Django framework, providing robust security "out-of-the-box," including built-in protection against Cross-Site Request Forgery (CSRF) for all web forms. It is the standard and most secure method for traditional, server-rendered web applications.

- Token-Based Authentication (JWT for the API):
  - For the RESTful API, which serves the Flutter mobile application, JSON Web Tokens (JWT) were implemented.
  - Rationale: JWTs provide a secure and stateless mechanism for authenticating users and authorizing access to APIs. They are ideally suited for decoupled clients like mobile applications because each token is self-contained with user information and a cryptographic signature. This enables secure session management without requiring the server to maintain session state, which is efficient and scalable for API-driven interactions.

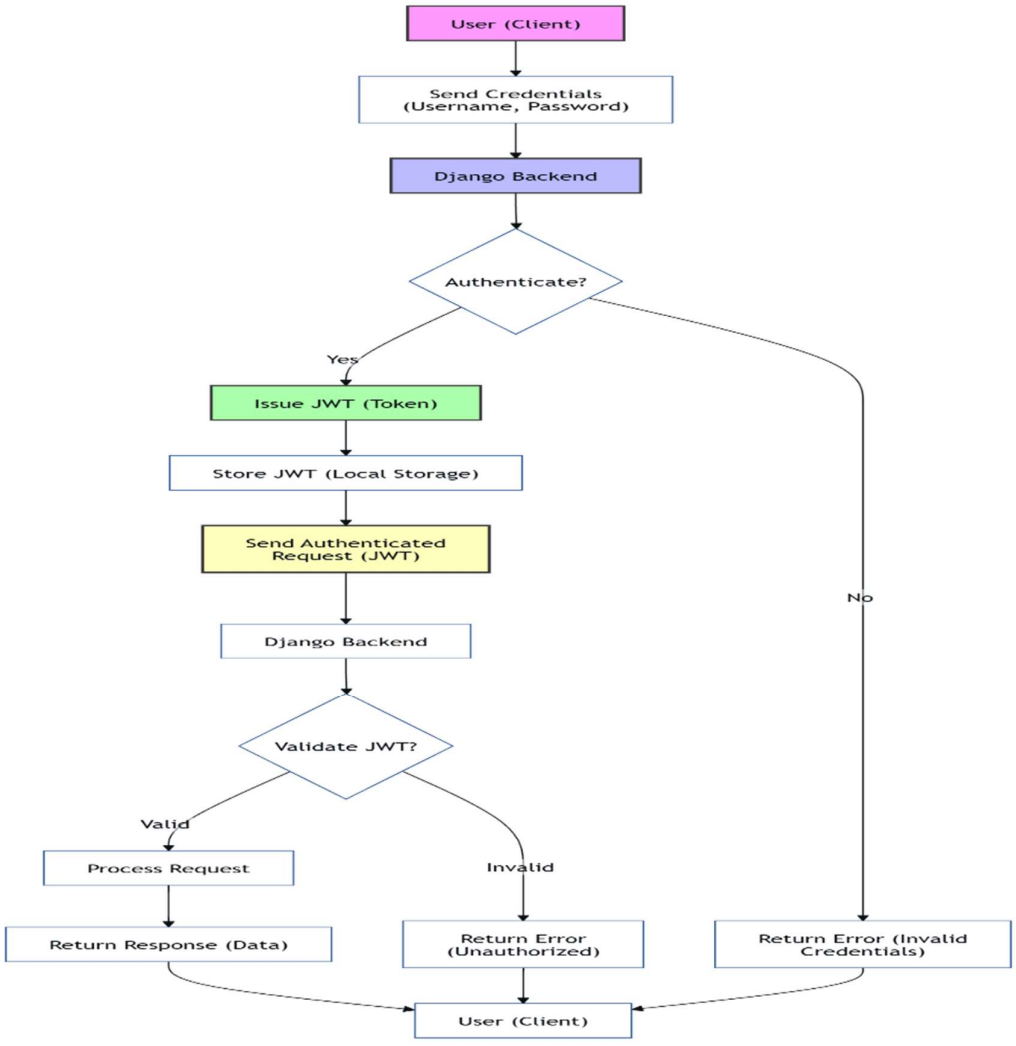

Figure 2.2: JWT authentication flow diagram

2.4.5 API Design

A well-designed API (Application Programming Interface) is crucial for decoupling the frontend and backend, and for enabling the mobile application to interact with the core system logic.

- RESTful APIs: The system employs a RESTful API architecture.
  - Rationale: REST (Representational State Transfer) is an architectural style that uses standard HTTP methods (GET, POST, PUT, DELETE, etc.) and conventions for building web services. This approach leads to APIs that are stateless, scalable, and easy for different clients (web frontend, mobile app) to consume. Django REST Framework was instrumental in implementing these APIs.

2.4.6 Cloud Hosting Platform

The choice of a hosting platform impacts the accessibility, scalability, and reliability of the web application.

- PythonAnywhere.com: The web application component of "MediCare" is deployed on PythonAnywhere, as mentioned in the Abstract (page 2) and outlined in Section 1.7 as part of the deployment discussion in Chapter 4.
  - Rationale**:** PythonAnywhere.com was chosen for its ease of use in deploying web applications built with technologies like Django and Bootstrap-based Django applications. It offers features such as managed databases (potentially for future scaling beyond SQLite), SSL certificates, custom domains, and integration with version control systems (like Git) for continuous deployment, simplifying the operational aspects of hosting the application.
  - Alternatives Considered: Other leading cloud platforms such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP) were considered.

2.4.7 Mobile Application Technologies

A dedicated mobile application enhances patient engagement and accessibility.

- Flutter & Dart: The Android mobile application for "MediCare" was developed using Flutter, Google's UI toolkit, with the Dart programming language.
  - Rationale: Flutter was selected for its ability to build natively compiled applications for mobiles from a single codebase, offering excellent performance and a rich set of customizable widgets for creating a visually appealing and

responsive user interface. Dart, its underlying language, is optimized for UI development. This choice allows for efficient development of a cross-platform (initially Android-focused) mobile experience.

2.5 Security and Privacy Considerations in Healthcare Applications

Security and privacy are critical considerations in the design and development of healthcare applications like "MediCare," given the highly sensitive nature of patient data. The system architecture incorporates multiple layers of protection and was developed with the following key aspects in mind to create a secure environment:

1. Role-Based Access Control (RBAC):

   A fundamental security measure in "MediCare" is the implementation of Role-Based Access Control (RBAC). This ensures that system users, including patients, doctors, hospital administrators, lab workers, and pharmacists, are granted access only to the data and functionalities that are strictly relevant and necessary for their specific roles. This principle of least privilege is enforced throughout the application to prevent unauthorized data access and to maintain data segregation. The details of RBAC were also identified as a key non-functional requirement (NFR-SEC-002). RBAC is enforced using custom Django permissions and DRF permission classes, e.g., IsDoctorUser, IsAdminUser, etc. These are applied per APIView to restrict access to role-specific endpoints.

2. Data Encryption:

   Protecting data confidentiality is crucial. "MediCare" addresses this through:

   - Encryption in Transit: The system is designed to be deployed using HTTPS, which leverages SSL/TLS protocols to encrypt all data exchanged between the client (web browser or mobile application) and the server. This protects sensitive information, such as login credentials, personal details, and medical data, from interception during transmission.
   - Encryption at Rest (Considerations): For data stored in the database (SQLite), password credentials are not stored in plaintext but are securely hashed.

3. User Authentication and Authorization: .

   Robust user authentication and authorization form the first line of defense in "MediCare". The system employs a sophisticated hybrid model, using the most appropriate security mechanism for each platform (web and mobile).

- Hybrid Authentication Model:
  - Web Application (Session-Based): For the server-rendered web application, the system utilizes Django's secure, built-in session-based authentication. This stateful mechanism uses server-side session records and HTTP cookies to manage user logins, providing industry-standard security and integral protection against attacks like Cross-Site Request Forgery (CSRF).
  - API for Mobile App (Token-Based): For the RESTful API that serves the Flutter mobile application, a stateless, token-based authentication system using JSON Web Tokens (JWT) is implemented. Upon successful login, the API provides a JWT to the mobile client, which is then included in the header of all subsequent requests to authenticate the user and blocked when the user logged out to not be used again.
- Authorization and Access Control:
  - Beyond initial authentication, authorization is managed through the Role-Based Access Control (RBAC) model defined in the system. For the web application, Django's user permissions and decorators restrict access to views. For the API, the JWT payload contains user role information, which is validated by the server to authorize or deny requests to specific API endpoints and resources, ensuring a patient cannot access a doctor's API.
- Future Security Enhancements (MFA):
  - While Multi-Factor Authentication (MFA) is recognized as an important security enhancement that significantly strengthens user authentication, its implementation was considered beyond the scope of the current project iteration. However, the system's modular authentication system is designed to be extendable, allowing for MFA to be incorporated in the future.

4. Audit Logs and Monitoring

The capability to log critical activities is important for security analysis, troubleshooting, and potential compliance needs. Django's built-in admin interface provides logging for actions performed within the admin panel. For comprehensive, application-wide audit trails of user activities (e.g., viewing patient records, making system changes), more extensive custom logging mechanisms would need to be developed. This aspect is noted as an area for future enhancement if deeper forensic capabilities or stringent compliance auditing become a requirement.

5. Adherence to Data Privacy

The "MediCare" system was developed with a strong commitment to data privacy and ethical data handling, as highlighted in NFR-SEC-003. Design choices were guided by general principles of data protection, such as data minimization (collecting only necessary data) and controlled access through RBAC. While the foundational security practices implemented aim to create a trustworthy environment for handling sensitive health information.

By focusing on these key areas, "MediCare" endeavors to provide a platform where security and patient privacy are treated with the utmost importance, aiming to protect sensitive healthcare information effectively

CHAPTER 3

System Design and Architecture

System Design and Architecture

This chapter details the comprehensive design and architectural framework of the "MediCare" Hospital Management System. It outlines the project planning methodology, the overall system architecture for both web and mobile platforms, the intricacies of the database design, and the various diagrams used to model system behavior and data flow. Also, it discusses the user interface (UI) and user experience (UX) design principles that guided the development of an intuitive and accessible system for all stakeholders.

3.1 Project Planning and Methodology

The development of the "MediCare" Hospital Management System followed the Agile Software Development Life Cycle (SDLC) methodology. This approach was chosen for its iterative nature and flexibility, allowing for continuous feedback incorporation and adaptive planning throughout the project lifecycle. The Agile methodology enabled the team to manage the project in a series of sprints, focusing on delivering functional increments of the system regularly and responding effectively to evolving requirements or insights gained during development. The key phases within this Agile framework included:

1. Iterative Requirement Gathering & Refinement: While initial requirements were collected (as detailed in Chapter 2), Agile process allowed for ongoing discussions with supervisors to refine understanding to new needs as the project progressed.

2. Sprint-Based Design & Development: The system's design (including wireframes, architecture diagrams, and ER models) and development were broken down into manageable sprints. Each sprint aimed to produce a potentially shippable increment of the software.

    - Development Modules: The development was logically divided into frontend (web templates and client-side scripting), backend (Django business logic and API development), database schema implementation, and mobile application (Flutter) development.

3. Continuous Testing: Testing was an integral part of each sprint, encompassing unit testing of individual components, integration testing to ensure modules worked together, and regular user acceptance testing (UAT) simulations to validate functionality against user needs.

4. Regular Review and Adaptation: Sprint reviews and retrospectives allowed the team to assess progress, identify challenges, and adapt the plan for subsequent sprints, ensuring the project remained aligned with its objectives.

5. Deployment and Maintenance Strategy: While full deployment and long-term maintenance are beyond a typical academic project scope, the system was designed for deployment on a platform like PythonAnywhere the modular design and use of version control (Git) facilitate ongoing maintenance, bug fixes, and future feature additions.

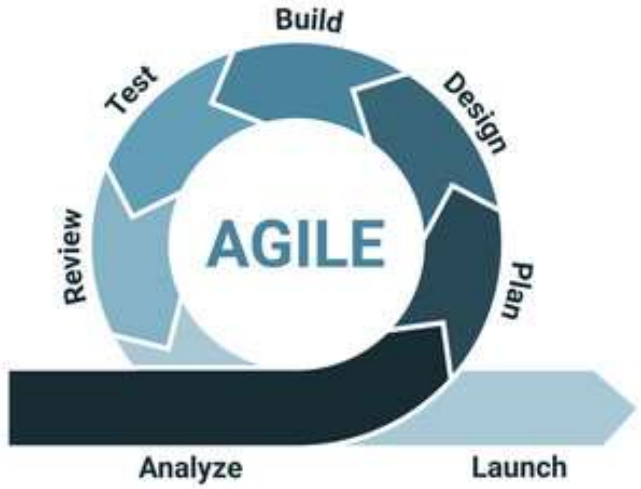

Figure 3.1: Agile Methodology

3.1.1 Timeline and Milestones

While the Agile methodology allows for flexibility, a general project timeline with key milestones was established to guide the development effort. The project was structured over an estimated 42-week period, with the following indicative phases and milestones:

Phase	Task	Duration	Milestone
Week 1 - 3	Requirement Analysis & Initial Research	3 weeks	Initial Scope Definition & Approval
Week 4 - 7	System Design & Architecture, ER Diagram	4 weeks	Design Document & Review
Week 8 - 12	Backend & API Development (Core Modules)	5 weeks	Core Backend Modules Completed

Week 13 - 17	Frontend (Web) & Mobile App Development	5 weeks	UI/UX Integration Complete
Week 18 - 20	Comprehensive Testing & Debugging	3 week	System Functionally Ready
Week 21 - 24	Final Adjustments, Deployment & Presentation Preparation	4 weeks	Project Delivery & Presentation

Table 3.1: Timeline

3.2 System Architecture

The "MediCare" system is designed with a robust and scalable architecture to support its diverse functionalities and user base across web and mobile platforms.

3.2.1 Overall System Architecture

The system employs a Three-Tier Architecture. This architectural pattern separates concerns into distinct layers:

1. Presentation Tier (Frontend):
   - Web Application: Consists of Django-rendered HTML templates styled with Bootstrap, CSS, and enhanced with client-side JavaScript. This tier is responsible for displaying information to users and capturing their input.
   - Mobile Application: A Flutter-based Android application providing a native user experience. It interacts with the backend via RESTful APIs.
2. Application Tier (Backend/Logic):
   - This is the core of the system, built using the Django (Python) web framework. It handles all business logic, processes user requests, manages user authentication and authorization (via JWT and session auth systems), and orchestrates interactions with the database.
   - Django REST Framework (DRF) is utilized within this tier to create and expose RESTful APIs, enabling communication with the Flutter mobile application and potentially other third-party services.
3. Data Tier (Database):
   - It is responsible for persistent data storage. For the current scope of this project, SQLite is used as the primary Database Management System (DBMS)

due to its simplicity, serverless nature, and ease of setup for development and project delivery. The system is designed with Django's ORM, allowing for potential future migration to more scalable relational databases (e.g., PostgreSQL, MySQL) if required.

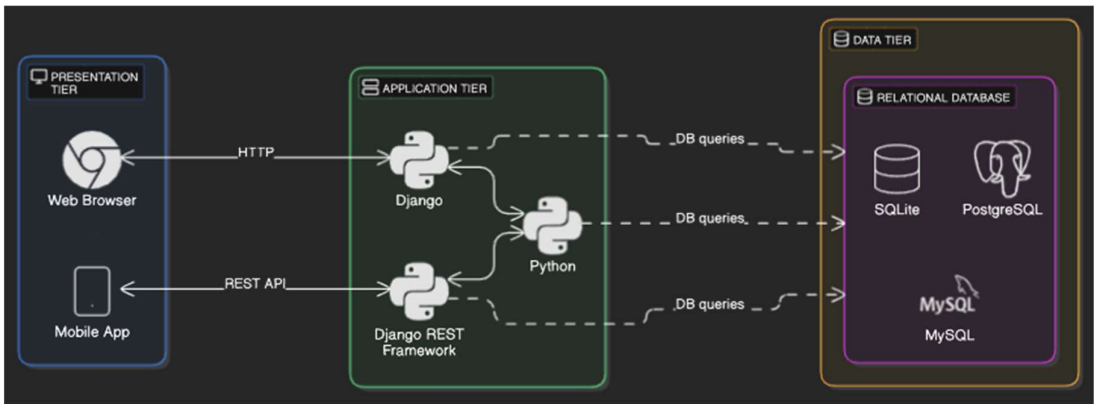

Figure 3.2: Overview of the system architecture

3.2.2 Web Application Architecture

The web application component follows a Model-View-Template (MVT) architecture, which is inherent to Django:

1. Model: Represents the data structure, directly mapping to database tables. The Django ORM handles all database interactions.

2. View: Contains the request-handling logic. Views retrieve data from models, process it, and select an appropriate template to render the response. They handle form submissions, user authentication, and business rule execution.

3. Template: Defines the presentation layer (HTML files within the templates directory). Django's templating engine dynamically generates HTML by populating templates with data passed from the views. Bootstrap is used extensively within these templates for responsive design and UI components.

Client-side interactivity is achieved using standard HTML, CSS, and JavaScript, often leveraging Bootstrap's JavaScript components.

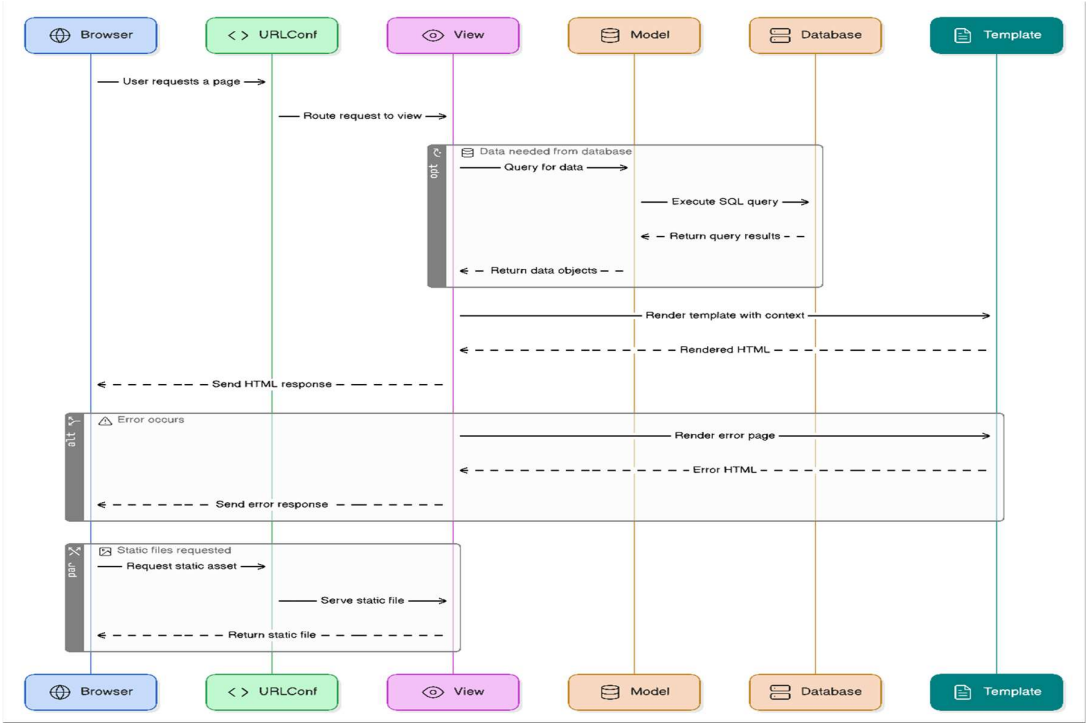

Figure 3.3: Web application Model View Template

3.2.3 Mobile Application Architecture

The "MediCare" Android mobile application is developed using Flutter, a UI toolkit from Google, with the Dart programming language. Flutter's architecture allows for building natively compiled applications from a single codebase, offering excellent performance and a rich set of customizable widgets. The mobile application architecture can be described as:

1. Presentation Layer (UI): Built using Flutter widgets, responsible for rendering the user interface and handling user interactions. This includes screens for patient registration, login, profile management, browsing hospitals/doctors, booking appointments, and viewing medical information.

2. Business Logic Layer: Implemented in Dart, this layer manages the application's state, handles navigation, and processes data obtained from the backend.

3. Data Access/Network Layer: Responsible for communicating with the "MediCare" backend. This layer makes HTTP requests to the RESTful APIs (exposed by the Django backend using DRF) to fetch and submit data (e.g., patient profiles, appointment details, prescriptions). Secure communication is ensured through JWT & session authentications.

4. Local Storage: The app utilizes local storage (e.g., shared_preferences via Flutter plugins) for caching user preferences, session tokens, or offline data access.

The mobile app is designed to be a client to the main backend, ensuring data consistency across platforms.

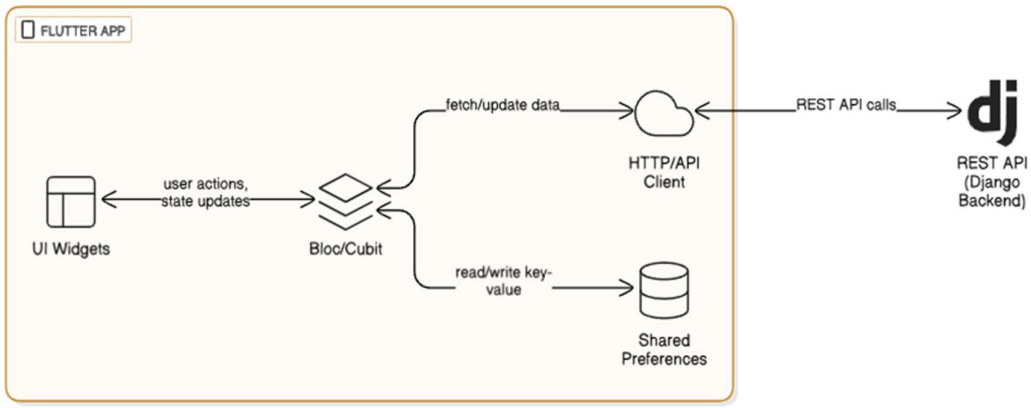

Figure 3.4: Mobile application Model View Template

3.3 Database Design

A well-structured database is critical for the efficient operation and data integrity of the "MediCare" HMS. The design is based on a relational model, implemented using Django's Object-Relational Mapper (ORM) with SQLite as the current database engine.

3.3.1 Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) visually represents the main entities within the "MediCare" system and the relationships between them. Key entities include Users (with distinct roles), Patients, Doctors, Hospitals, Departments, Appointments, Prescriptions, Lab Reports, Pharmacy (Medicines, Orders), and Payments.

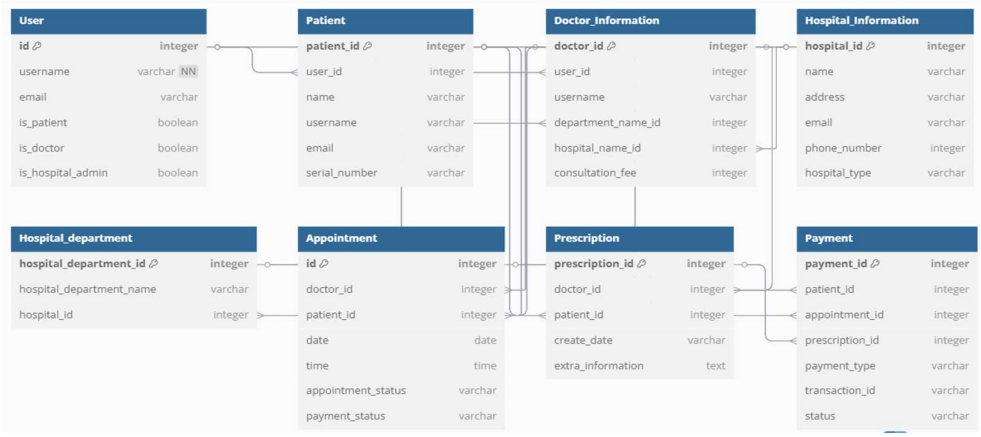

Figure 3.5: Entity Relationship Diagram for the system

The MediCare Hospital Management System (HMS) leverages a well-structured relational database model built upon essential entity relationships to ensure data consistency, scalability, and efficient access patterns. The architecture employs a combination of one-to-one, one-to-many, and many-to-many relationships across its modules. Understanding these relationships is critical for both database design and application logic.

1. One-to-One (1: 1) Relationships

   This type of relationship enforces a strict link between two entities where each row in one table is related to exactly one row in another.

   - Example: User → Patient / Doctor_Information
     - Each user account can have one and only one associated patient profile or doctor profile. This ensures personalized and secure access.
     - Enforced using a unique foreign key from the profile table (e.g., Patient.user_id) referencing the User.id.

2. One-to-Many (1: N) Relationships

   This structure allows one record in a parent table to relate to multiple records in a child table, while each child links back to one parent only.

   - Example 3: Patient → Appointment
     - A patient can book many appointments over time, but each appointment belongs to a single patient.

3. Many-to-Many (N:M) Relationships

   This relationship occurs when multiple records in one table relate to multiple records in another, typically managed via a junction/intermediate table.

   - Example: Doctor ↔ Patient (via Appointment)
     - A doctor may see many patients, and a patient may consult multiple doctors over time.

3.3.2 Database Schema

The database schema serves as the backbone of the *MediCare* Hospital Management System, defining the logical structure and organization of all stored data. It outlines how data is grouped into tables, how each table is structured using columns and data types, and how relationships are enforced using primary keys, foreign keys, and other integrity constraints. It ensures the efficient data storage and retrieval.

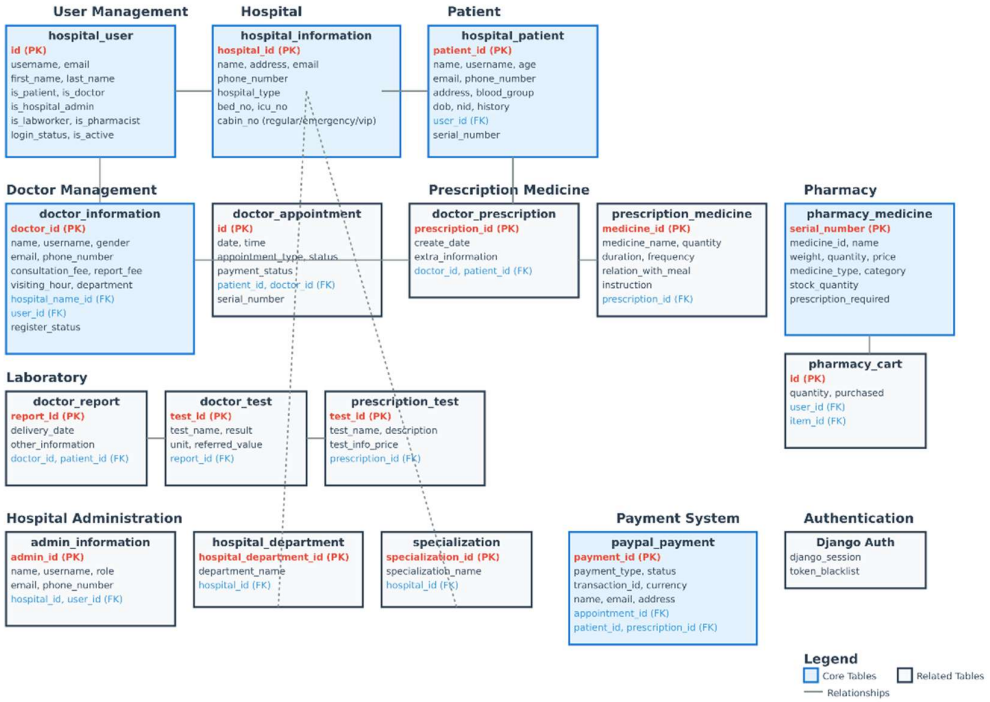

Figure 3.6: Database Schema

3.4 Data Flow Diagrams (DFDs)

Data Flow Diagrams (DFDs) are used for visualizing the flow of information within the "MediCare" platform. They display the communication between different system components, processes, data stores, and external entities. DFDs are generally used during problem inquiry or system analysis to understand how data is processed.

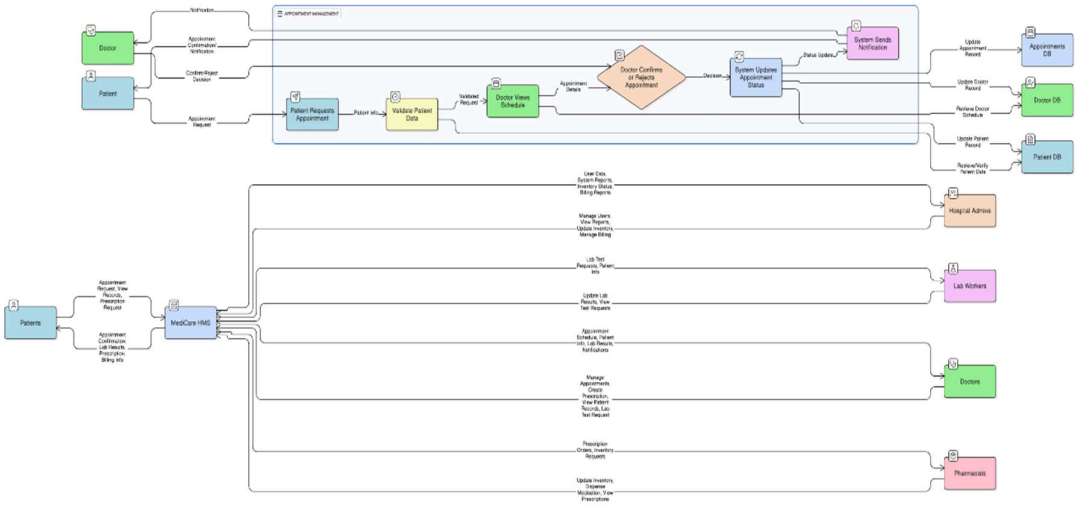

Figure 3.7: Context of Level 1 DFD

3.5 Use Case Diagrams

Use Case diagrams illustrate the different ways users (actors) interact with the "MediCare" system and the functionalities (use cases) they can perform. They are valuable for understanding system requirements from a user's perspective.

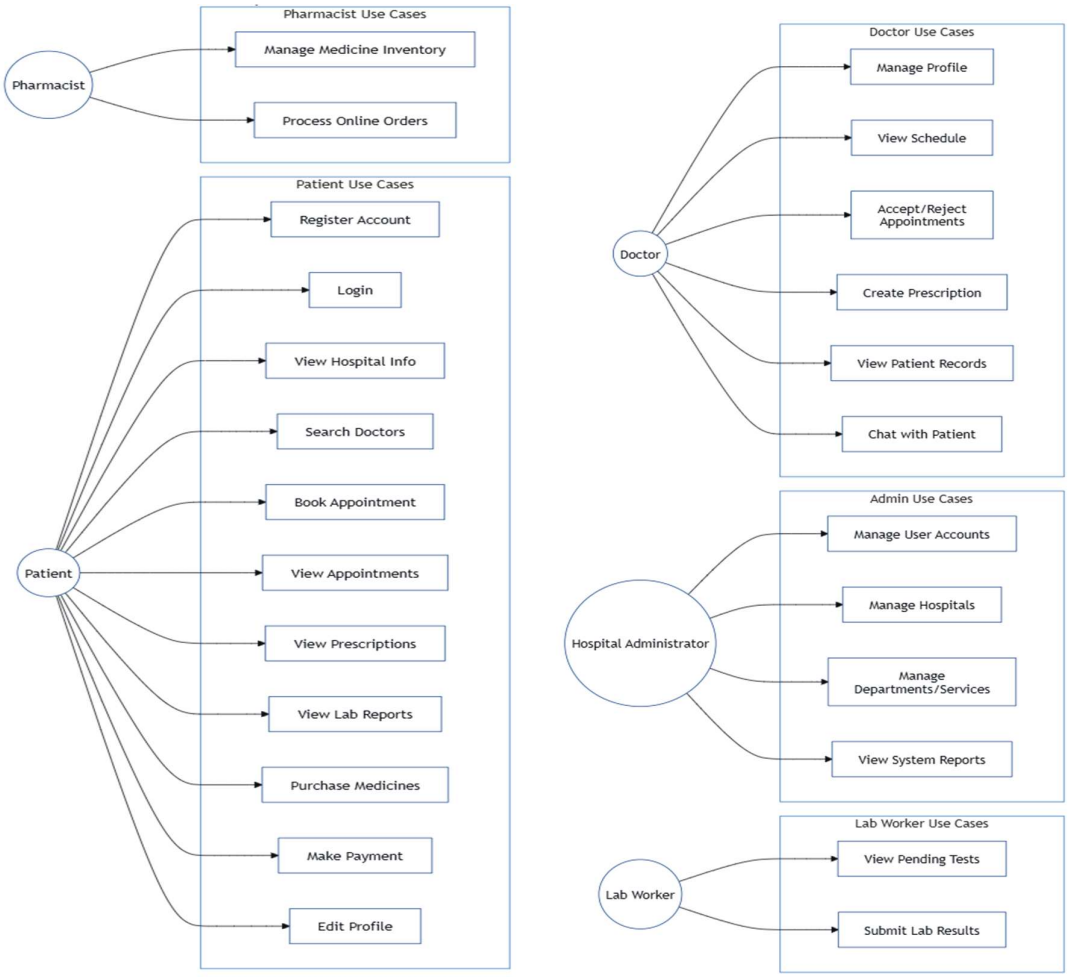

Figure 3.8: Main Use Case Diagram for MediCare System.

3.6 User Interface (UI) and User Experience (UX) Design Principles

The design of the "MediCare" user interfaces for both the web and mobile platforms adheres to a set of core principles aimed at ensuring usability, accessibility, and a smooth experience for all user roles—patients, doctors, lab workers, admins, and pharmacy staff.

1. Clarity and Simplicity: All interfaces were designed to be intuitive and easy to navigate. Clutter was minimized to ensure users can quickly find and complete their tasks.

2. Consistency: A consistent layout, terminology, and navigation structure were maintained across the web and mobile platforms. Shared color schemes, button styles, and icons reinforce this consistency.

3. Responsiveness (Web & Mobile): The web application uses Bootstrap's responsive grid system to ensure layouts adapt gracefully to different screen sizes—from desktop monitors to tablets and smartphones, while the mobile application uses the Flutter Screen Util package.

4. Native Feel (Mobile): Android application, built using Flutter, adopts Material Design principles to deliver a user experience that feels native to Android devices.

5. Efficiency: Workflows, such as booking appointments or viewing prescriptions, are streamlined to require minimal steps, improving speed and user satisfaction.

6. Feedback: The system provides real-time visual feedback for user actions, including success alerts, error messages, and loading indicators. This is handled on the web via Django's messages framework and message.html template.

7. Accessibility Considerations: While full WCAG compliance is outside the current scope, we incorporated readable font sizes, accessible color contrasts, and logical element ordering to improve usability for all users.

8. Role-Based Design: Interfaces are customized based on the user's role. Each role accesses specific dashboards and functionality relevant to their daily tasks, improving focus and efficiency.

3.6.1 Web Application UI/UX Design

The web application of the MediCare Hospital Management System was meticulously designed to cater to a diverse set of user roles - including patients, doctors, administrators, lab technicians, and pharmacists. Its UI/UX approach combines consistency, functionality, and responsiveness to ensure that each user can navigate the system with ease and clarity.

1. Role-Based Layouts: Each authenticated user is directed to a dedicated interface based on their role. The layout employs a sidebar-driven navigation system (doctor-sidebar.html, hospital_admin-sidebar.html, etc.), allowing quick access to the most relevant modules for that user type:
   - Doctors can quickly access their appointments, patient files, and prescriptions.

- Administrators have full control over hospital, department, and user management modules.
- Patients are shown a simplified interface with options to browse doctors, schedule appointments, and view health records.

2. Responsive Design via Bootstrap: To ensure compatibility across different screen sizes and devices, the UI was built using Bootstrap 5, a popular frontend framework. Core UI elements such as:
   - Navigation bars
   - Accordion menus
   - Input forms
   - Tables and Models

3. Dynamic and Informative Dashboards: The dashboard for each user role is populated with live data relevant to that role. Examples include:
   - Doctors: Today's appointments, and quick links to patient records.
   - Patients: Upcoming appointments, accessible lab results, and prescriptions.
   - Admins: System activity metrics, new registration requests, and shortcut buttons for hospital resource management.

4. Interactive Forms and Tabular Data

   Most backend data entry operations are handled via dynamic forms, including validation and helpful placeholder texts. CRUD operations for hospitals, users, departments, and medicines are facilitated via modal forms to enhance user interaction without full page reloads. Data listings (doctors, reports, medicines, etc.) are presented using sortable and paginated tables, powered by Bootstrap components and Django template tags for efficient navigation and readability.

5. Search and Filter Functionality

   For improved usability in data-heavy views, search bars and dropdown filters are implemented. These features were built with Django's query sets and GET parameters to enable server-side filtering, ensuring both speed and security.

   Users can:
   - Search for a doctor by name, or hospital.
   - Filter appointments by status or date.
   - Locate patients by their names.

6. Accessibility and User Feedback

The UI adheres to basic accessibility principles:

- Form labels and error messages for screen reader compatibility.
- Icon-based buttons with tooltips for enhanced understanding.
- Confirmation modals and success alerts for user actions like saving or deleting data.

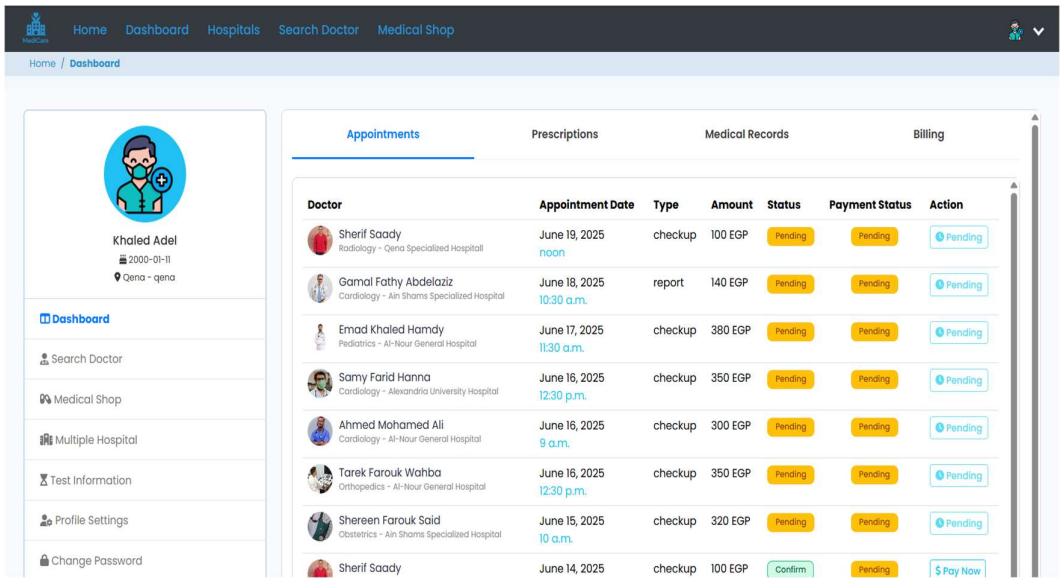

Figure 3.9: Patient Dashboard Web Page

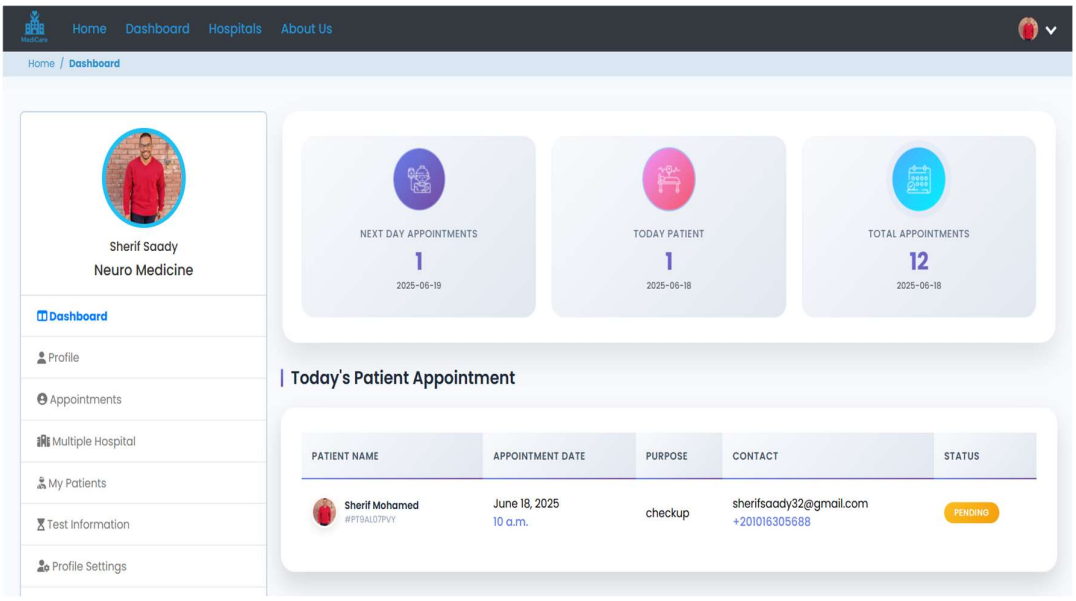

Figure 3.10: Doctor Dashboard Web Page

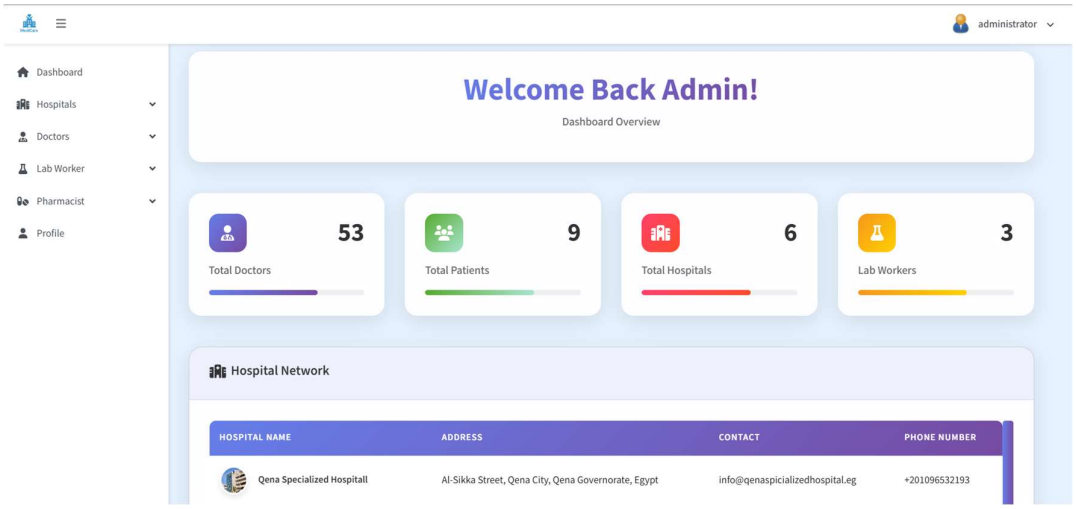

Figure 3.11: Admin Dashboard Web Page

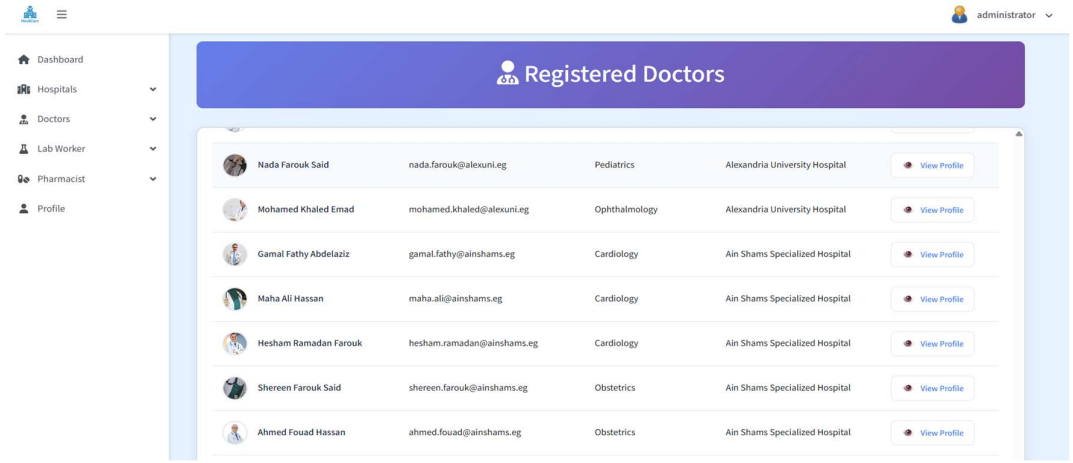

Figure 3.12: List of Registered Doctors Web Page

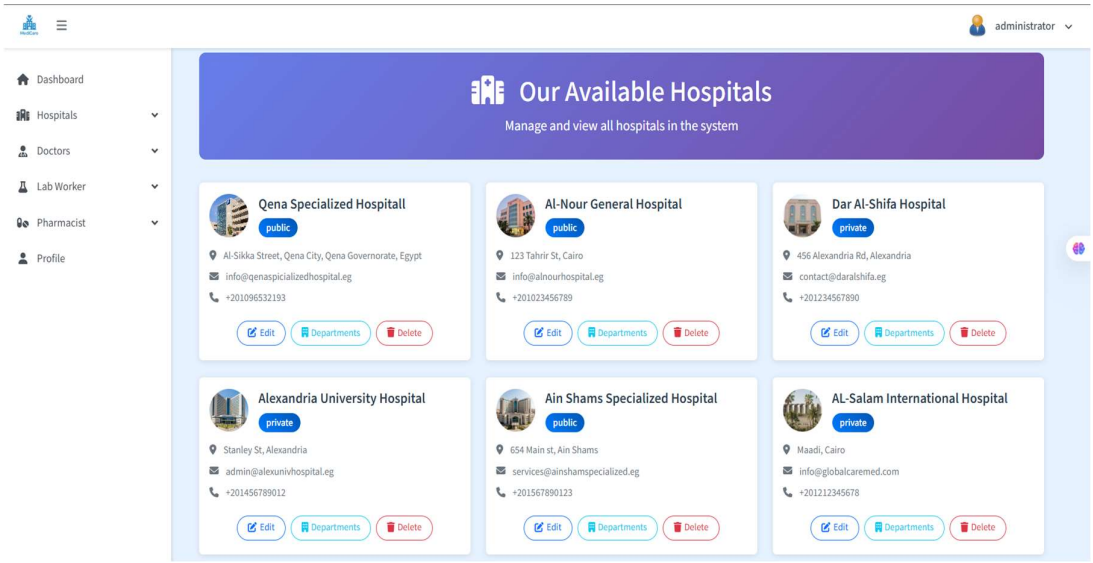

Figure 3.13: All Registered Hospitals Web Page

3.6.2 Mobile Application UI/UX Design

The "MediCare" Android mobile application, developed using Flutter and Dart, was specifically designed to offer a fast, user-friendly, and feature-rich experience for patients. The design emphasizes clarity, accessibility, and responsiveness, ensuring that users can complete core health-related tasks efficiently and intuitively.

1. Design Philosophy: The mobile app follows a clean, minimalistic design approach with an emphasis on mobile-native UI/UX patterns.

   Core principles include:

   - Simplicity: Reduced cognitive load by limiting screen elements.
   - Consistency: Uniform colors, fonts, and button styles across all screens.
   - Efficiency: Fast access to appointments, health records, and profile actions.
   - Platform-Native Feel: Leveraging Flutter's Material Design components for a polished Android-native experience.

2. Core Screens and Features: Each screen in the mobile application is designed to serve a critical patient function:

   - Login & Registration:
     - Supports secure sign-up and authentication using JWT tokens.
     - Backend validation ensures account integrity.
     - Role detection (patients only, currently) is handled post-authentication.
   - Home / Dashboard:
     - Provides patients with a summarized view of hospitals' departments, related doctors from these departments.
     - Blue container that takes you to the doctors list screen.
     - A drawer that contains first the profile photo, name, and email address, then the buttons of appointments, prescriptions, hospitals list, …etc.
   - Search & Browse Doctors/Hospitals:
     - Real-time API-driven lists display hospitals, departments, and doctors.
     - Includes filtering by specialty, availability, and hospital name.
     - Uses search input with powered by Flutter's TextEditingController.
   - Book Appointments:
     - Patients can select a doctor, pick a date and time then book the appointment.
     - A summary screen shows the users their selections after the submission.

- View Prescriptions & Lab Reports:
  - Health records are displayed in scrollable cards showing doctor's info.
  - Navigation is managed using ListView.builder and dynamic data fetched via Dio from REST APIs.
- Profile Management:
  - Patients can view and update profile details such as age, address, blood group, and contact info.
  - Form validation is handled client-side using Form and TextFormField widgets with validators.

3. Flutter Implementation Details:

- The application is built using Flutter's Material Components, ensuring seamless animations, transitions, and consistent styling.
- State management is handled primarily via setState(), with migration to BLoC.
- Network interactions are managed using the Dio package for efficient API communication, error handling, and token-based authentication.

4. Responsive Layout and Device Compatibility:

- The flutter_screenutil package is used to scale fonts, icons, and spacing dynamically based on device screen size and resolution.
- The app supports both portrait and landscape orientations, ensuring optimal viewing across various Android phones and tablets.
- The UI is made adaptive using Flutter layout widgets such as:
  - Expanded, Flex, and Flexible for proportional sizing.
  - ListView and SingleChildScrollView for scrollable content.

5. Usability Enhancements:

- Icons from flutter_vector_icons and intuitive labels improve accessibility.
- Feedback mechanisms such as toasts (FlutterToast) and loading indicators (CircularProgressIndicator) provide real-time user guidance.
- Future iterations will add dark mode support and transition animations using PageRouteBuilder and Hero widgets for better UX.

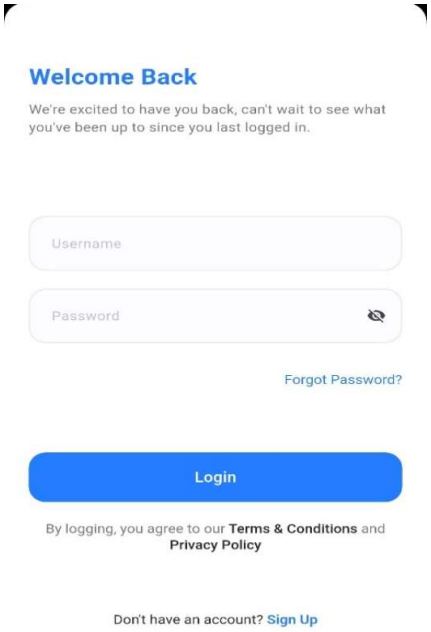

Figure 3.14: Mobile Login Screen

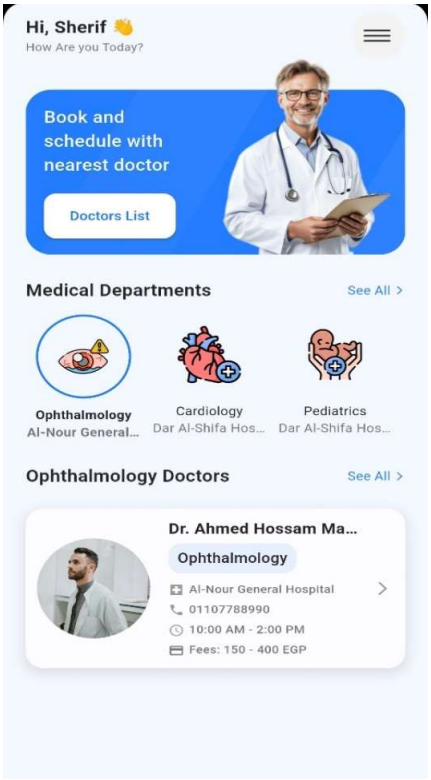

Figure 3.15: Mobile Home Screen

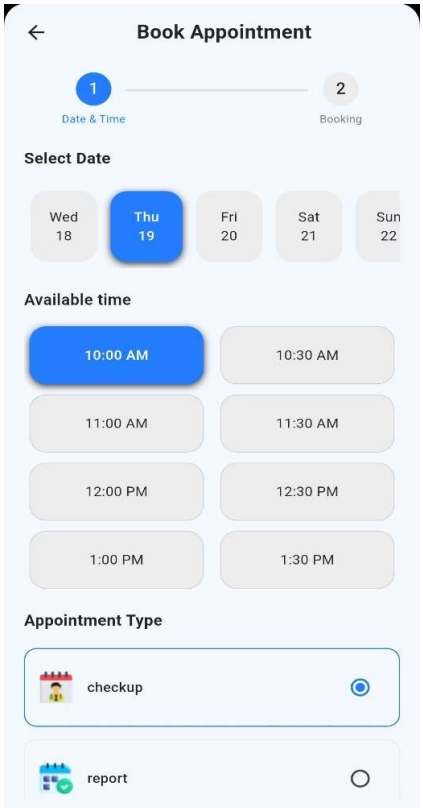

Figure 3.16: Booking Appointment Screen1

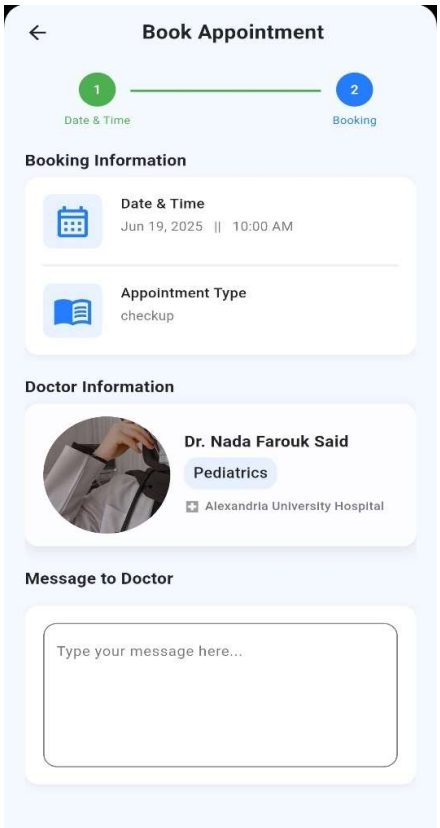

Figure 3.17: Booking Appointment Screen 2

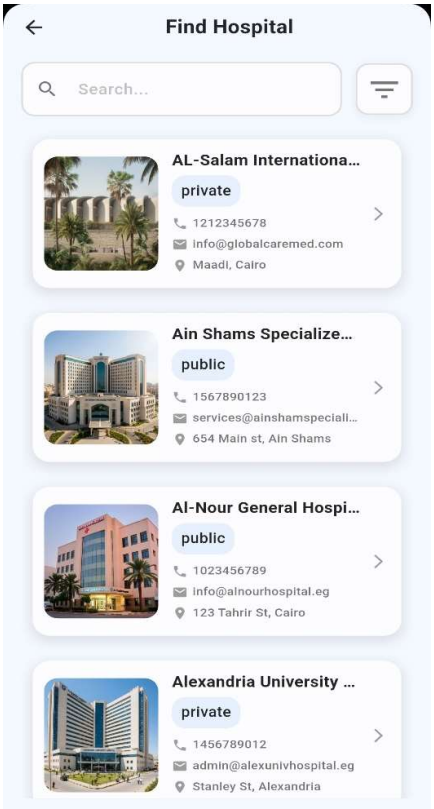

Figure 3.18: Hospitals List Screen

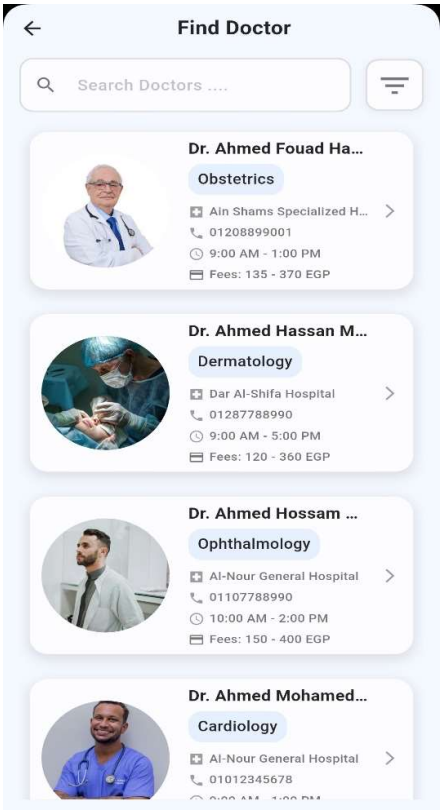

Figure 3.19: Doctors list Screen

CHAPTER 4
System Implementation

System Implementation

This chapter details the technical implementation of the "MediCare" Hospital Management System. It covers the setup of the development environment, the construction of the backend logic and APIs, the development of the frontend web application and the patient-focused mobile application, and the implementation of key security and deployment strategies.

4.1 Development Environment Setup

The successful implementation of the "MediCare" system required a carefully configured development environment to support both backend and frontend development for the web and mobile platforms. The primary tools and technologies used were:

1. Version Control: Git was used for version control, with a repository hosted on a platform like GitHub. This facilitated collaboration, change tracking, and code management throughout the project lifecycle.

2. Code Editor/IDE: Visual Studio Code (VS Code) was the primary code editor, chosen for its lightweight nature, extensive library of extensions (for Python, Django, Flutter, and more), integrated terminal, and Git support.

3. Backend Environment (Python/Django):
   - Python: The project was developed using Python (version 3.13).
   - Virtual Environment: A Python virtual environment (using venv) was created to isolate project dependencies, ensuring that packages required for "MediCare" (like Django, Django REST Framework, Pillow for image handling, etc.) did not conflict with other system-wide Python installations. Key dependencies were managed and can be documented in a requirements.txt file.

4. Mobile Environment (Flutter/Dart):
   - Flutter SDK: The Flutter Software Development Kit was installed to build the cross-platform mobile application (version 3.29.2).
   - Dart SDK: Included with the Flutter installation (version 3.7.2).
   - Android Studio: Used for its powerful Android emulator, which allows testing the Flutter application on various virtual Android devices, screen sizes, and OS versions. Android Studio also provides essential Android platform tools required by Flutter.

5. API Testing Tool:

- Postman: Used extensively for testing the RESTful APIs developed with Django REST Framework. It allows sending various types of HTTP requests (GET, POST, PUT, DELETE) to API endpoints, inspecting responses, and verifying authentication (JWT) and data formats (JSON).

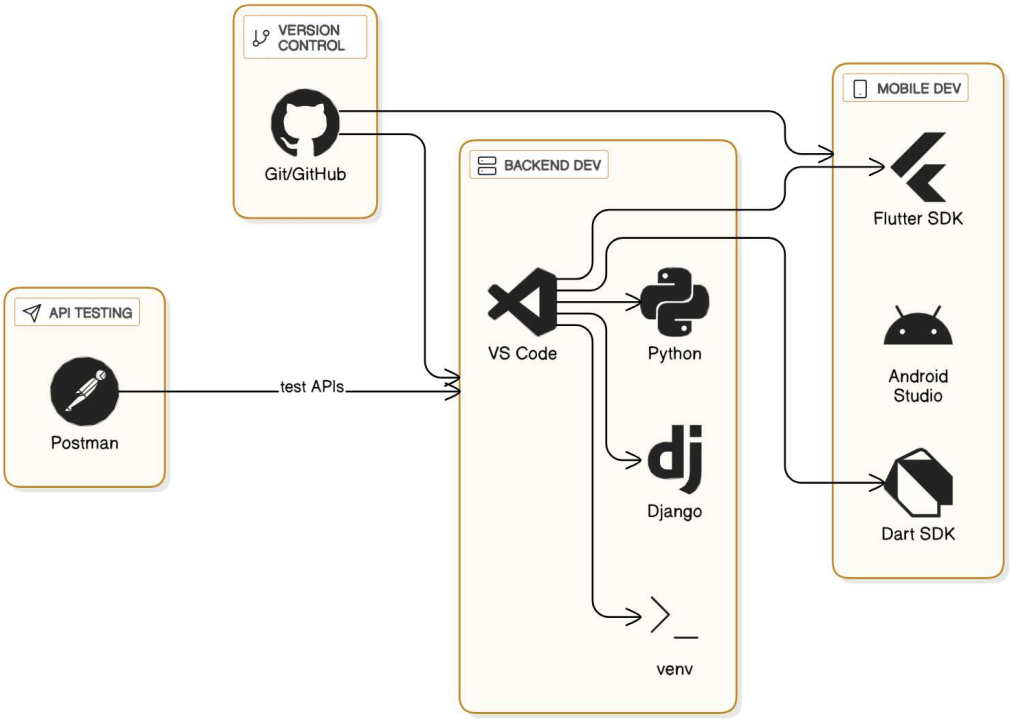

Figure 4.1: Development Environment Setup

4.2 Backend Implementation

The backend is the core engine of the "MediCare" system, built with Django and the Django REST Framework (DRF), as it's responsible for the secure transaction between the web or the mobile and the database. It handles all business logic, data persistence, and provides a secure API for client applications.

4.2.1 Database Integration

As detailed in Chapter 3, the system uses SQLite via Django's Object-Relational Mapper (ORM). The ORM translates Python model classes (models.py) into database tables and provides a high-level API for all database operations (Create, Read, Update, Delete - CRUD), eliminating the need to write raw SQL queries. This approach ensures data integrity through defined model relationships (OneToOneField, ForeignKey, ManyToManyField) and facilitates potential future migration to other database systems.

4.2.2 Business Logic Implementation

The business logic is implemented within Django views (views.py files in each app). These views process incoming requests from both the web and mobile clients, interact with the database via models, and return appropriate responses. Key implemented logic includes:

1. User Role Management: Logic in views checks user roles (is_doctor, is patient, etc.) to grant access to specific functionalities.

2. Appointment Management: The logic handles booking requests, checks for doctor availability, updates appointment statuses (Pending, Confirmed).

3. Prescription and Report Generation: Views allow doctors to create detailed prescriptions and lab workers to submit reports, which are then linked to the respective patient profiles. PDF generation logic (using libraries like ReportLab).

4. Payment Processing: Views integrate with the PayPal SDK to handle payment requests, process transactions, and update order statuses in the database upon successful payment.

5. Pharmacy Logic: Manages the online "Medical Shop," including cart management (add/remove items), order creation, and inventory checks.

6. Automated Email Notifications: The business logic for key events such as appointment booking, status changes, and doctor registration approval/rejection و includes calls to an email utility function. This function dynamically constructs and sends context-specific emails to the relevant user (patient or doctor) to keep them informed.

4.2.3 API Development (RESTful Endpoints)

A comprehensive RESTful API was developed using DRF to enable the Flutter mobile application to interact securely with the backend. The API design follows standard REST principles, using appropriate HTTP methods for different actions (e.g., GET for retrieval, POST for creation). Key API endpoints include:

1. User Authentication Endpoints: For patient registration, login (issuing JWT), and profile management.

2. Hospital and Doctor Endpoints: For fetching lists of hospitals, departments, and doctors.

3. Appointment Endpoint: For booking, viewing, and managing patient appointments.

4. Medical Record Endpoint: For retrieving patient tests and specimens.

5. Prescription Endpoint: For retrieving patient medicine and tests.

6. Departments Endpoint: For retrieving hospital departments.

7. Change Password Endpoint: For changing the password.

4.2.4 Media Handling and File Management:

The backend supports file uploads and downloads, particularly for medical reports and prescription documents. Files such as PDFs, images, or scanned reports are stored in a dedicated media directory configured via Django's MEDIA_ROOT and MEDIA_URL settings. When a lab worker uploads a report, it is stored with a unique filename and associated with a specific patient and test record. On the other hand, doctors or patients can request those files via secure endpoints. PDF generation is also integrated in the backend using ReportLab. Doctors can generate patient prescriptions or medical summaries, which are formatted into PDFs and returned either for download or for direct viewing in the frontend.

```
152     class HospitalSerializer(serializers.ModelSerializer):
153         class Meta:
154             model = Hospital_Information
155             fields = '__all__'
156 
157     class DoctorSerializer(serializers.ModelSerializer):
158         class Meta:
159             model = Doctor_Information
160             fields = '__all__'
161 
162     class PatientProfileSerializer(serializers.ModelSerializer):
163         class Meta:
164             model = Patient
165             fields = '__all__'
166 
167     class PatientAppointmentSerializer(serializers.ModelSerializer):
168         appointment_status = serializers.CharField(default="pending", read_only=True)
169         payment_status  = serializers.CharField(default="pending", read_only=True)
170         patient = serializers.CharField(default=serializers.CurrentUserDefault(),read_only=True)
171         serial_number = serializers.CharField(read_only=True)
172         transaction_id = serializers.CharField(read_only=True)
173         class Meta:
174             model = Appointment
175             fields = '__all__'
176 
177     class PrescriptionSerializer(serializers.ModelSerializer):
178         class Meta:
179             model = Prescription
180             fields = '__all__'
```

Figure 4.2: Sample DRF Serializer for Appointment Model

```python
from django.urls import path
from . import views

app_name = 'api'
urlpatterns = [
    path('', views.getRoutes),
    path('patient_register/', views.PatientRegister.as_view()),
    path('doctor_register/', views.DoctorRegister.as_view()),
    # path('admin_register/', views.AdminRegister.as_view()),

    path('login/', views.LoginView.as_view()),
    path('logout/', views.LogoutView.as_view(), name='logout'),

    path('password_reset/', views.PasswordResetView.as_view(), name='password-reset'),

    path('change_password/', views.ChangePasswordView.as_view(), name='change_password'),

    path('hospital/', views.getHospitals.as_view()),
    path('hospital/<int:pk>/', views.GetOneHospital.as_view()),

    path('doctor/', views.GetDoctors.as_view()),
    path('doctor/<int:pk>/', views.GetOneDoctor.as_view()),

    path('patient_profile/', views.PatientProfile.as_view()),

    path('appointment/', views.PatientAppointment.as_view()),

    path('prescription/', views.PatientPrescription.as_view()),
    path('prescription_medicine/', views.PatientPrescriptionMedicine.as_view()),
    path('prescription_test/', views.PatientPrescriptionTest.as_view()),
    path('report/', views.PatientReport.as_view()),
    path('payment/', views.PatientPayment.as_view()),

    path('all_prescription_data/', views.CombinedDataView.as_view()),

    path('hospital_department/', views.HospitalDepartment.as_view()),

]
```

Figure 4.3: Sample API URL Configuration

4.3 Frontend Implementation (Web Application)

The web application provides a comprehensive and role-based interface tailored to administrators, doctors, patients, pharmacists, and laboratory staff. Built using Django's powerful templating engine, the platform dynamically renders content based on user authentication and permissions, ensuring personalized and secure access for each user type. Bootstrap was utilized extensively to achieve a responsive design that adapts seamlessly across various screen sizes and devices, offering a consistent and intuitive user experience. Integration with JavaScript and jQuery enhanced real-time interactivity, such as dynamic form validation, filtering, and UI feedback.

4.3.1 Patient Registration and Management Module

When users visit the platform, they are greeted with a responsive and dynamic Home Page. The system guides them through various services such as booking appointments, logging in, or registering a new account. The patient module provides a clean and simple interface where patients can:

1. Register and log in securely.
2. View and edit their profiles, including changing passwords.
3. Search for doctors by specialization or hospital.
4. View detailed information about hospitals, including departments and services.

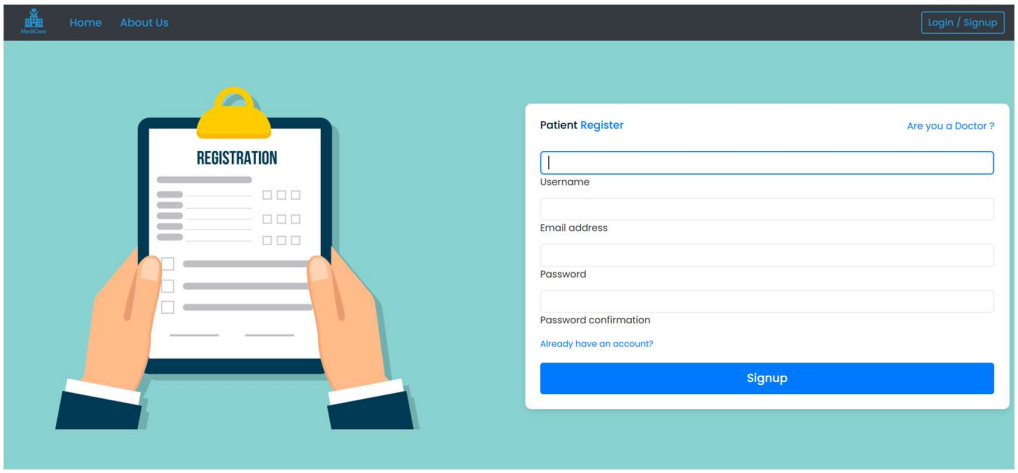

Figure 4.4: Web Application Patient Registration

4.3.2 Appointment Scheduling Module

The appointment booking module is a core feature of the system, designed to facilitate efficient and user-friendly scheduling for both patients and doctors. Once logged in, patients can browse through the list of registered hospitals and view available doctors based on department, specialization, and location. The interface provides filtering and search functionalities to help patients find the most suitable healthcare provider. After selecting a doctor, patients can view available time slots and book an appointment seamlessly through the web interface.

On the doctor's side, a real-time, interactive calendar view allows them to visualize and manage their daily and weekly schedules. Appointment requests appear in a pending state and can be easily accepted or rejected by the doctor based on their availability.
Key implemented features include:

1. Real-time calendar views for doctors to manage their schedule.
2. Doctor-side interface to accept or reject pending appointment requests.
3. Clear status tracking for appointments (Pending, Confirmed, Completed).

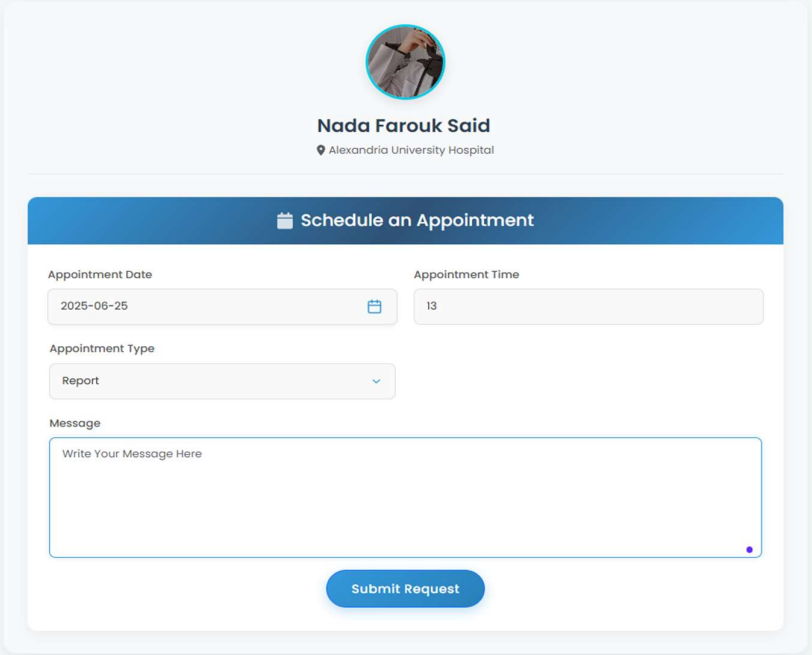

Figure 4.5: Booking Appointment Web Page

4.3.3 Billing and Invoicing Module

A dedicated billing section is integrated to enhance financial transactions. Patients can securely pay for services like lab tests online. The implementation includes:

1. Integration with PayPal for secure online payments, offering options to pay via a PayPal account or directly with a debit/credit card.
2. A clear payment interface displaying the service details, doctor's name, total amount, and payment options.
3. Real-time payment confirmation and tracking of payment history.

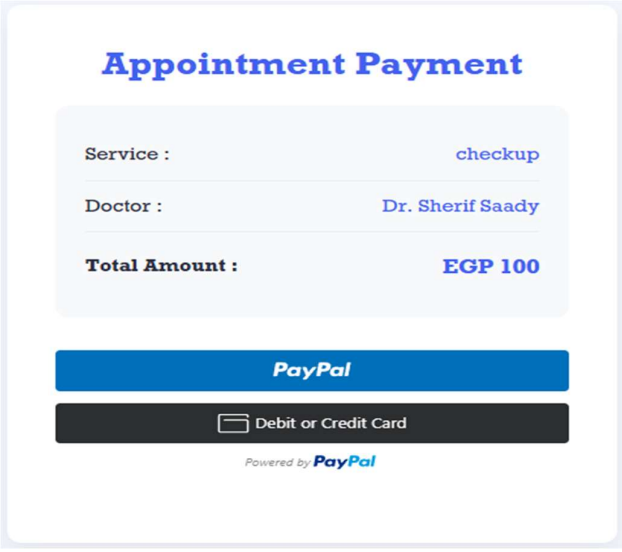

Figure 4.6: Online Appointment Payment Interface with PayPal Integration

4.3.4 Staff and Admin Panel

The Admin Module provides complete control and oversight of the platform. Admins are responsible for managing users, system records, and maintaining operational structure. The key functionalities implemented are:

1. Comprehensive Dashboard: An overview of key system metrics like total patients, doctors, hospitals, and lab workers.
2. User Management: Admins can add, view, and manage accounts for lab workers, and pharmacists.
3. Hospital Management: Admins can add and edit detailed hospital information, including departments, specializations, and services offered.
4. Doctor Management: Admins can accept and deny doctors registration.
5. Monitoring and Reporting: The admin panel allows for viewing appointment histories, user session logs, and contact us queries.

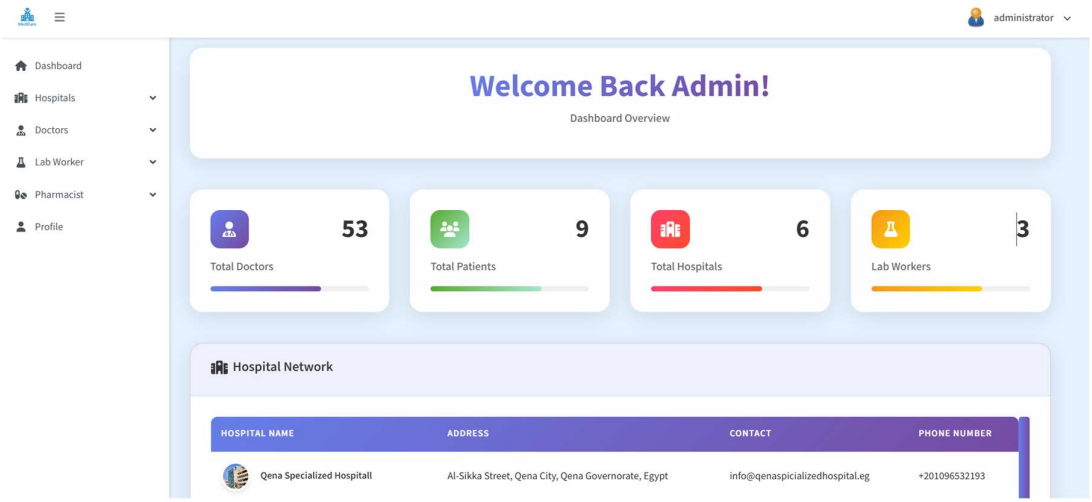

Figure 4.7: Administrator's Dashboard Web Page

4.3.5 Inventory Management Module

The Inventory Management Module is a critical component of the MediCare system, ensuring the effective handling of pharmaceutical stock and its seamless integration with the patient-facing "Medical Shop." This module is primarily operated by pharmacists through the web, granting them full control over medicine-related data and stock levels. Key functionalities include:

1. Full CRUD (Create, Read, Update, Delete) operations for managing the stock of medicines available in the online shop.

2. Pharmacists can add new medicines with details like name, image, category, price, and stock quantity.
3. This inventory is directly linked to the patient-facing "Medical Shop" for real-time availability.
4. Stock Control and Updates: Pharmacists can monitor inventory levels and restock items, helping to prevent shortages and manage high-demand products.

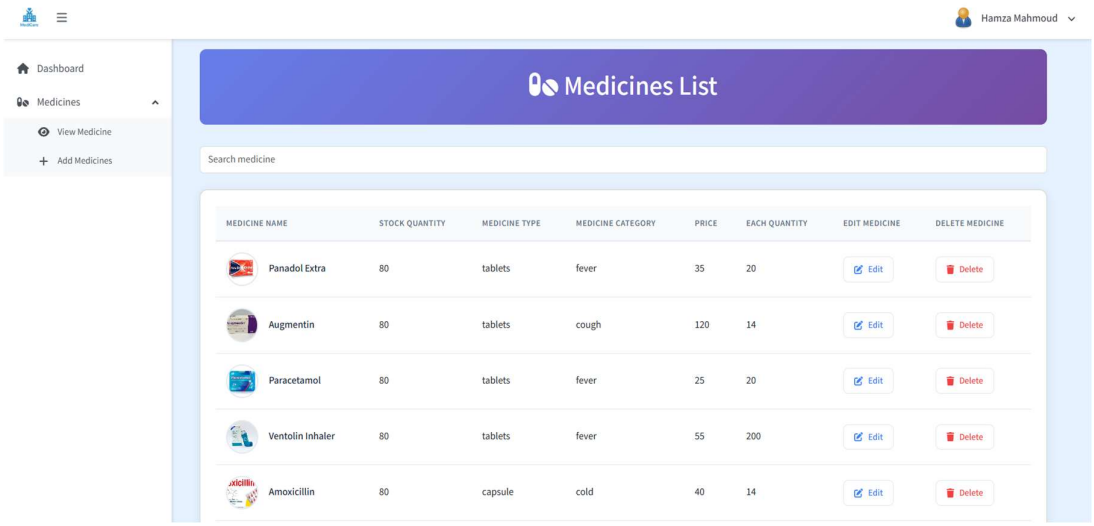

Figure 4.8: Pharmacist Inventory Management Web Page

4.3.6 Report Generation Module

The Report Generation Module plays a vital role in supporting the clinical documentation workflow for doctors, lab workers, and patients within the MediCare system. It ensures the seamless creation, storage, and sharing of key medical records, which improves transparency, accessibility, and the continuity of patient care.

Key functionalities include:

1. Prescription Generation: Doctors can generate detailed digital prescriptions for patients from their dashboard. These prescriptions are saved to the patient's record and become immediately viewable by the patient.
2. Lab Report Submission: Lab workers can view paid-for test requests, enter results, and submit the final lab report, which is then linked to the patient's account.
3. PDF Generation: Both prescriptions and lab reports can be generated as PDF documents for easy downloading and printing.

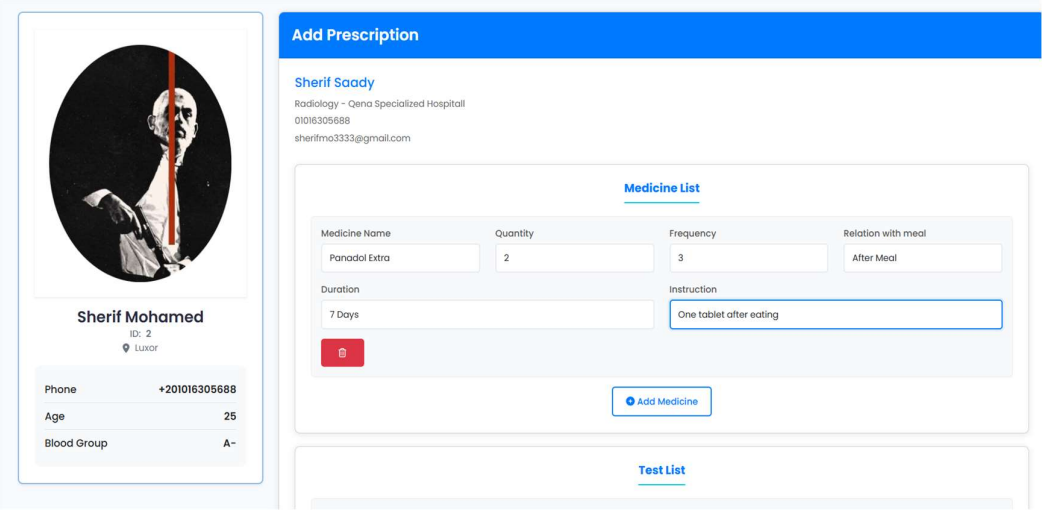

Figure 4.9: Doctor Adding Prescription to Patient Web Page

4.4 Mobile Application Implementation (Android - Patient Focused)

The "MediCare" Android application was developed using Flutter and Dart, enabling cross-platform performance with a native-like user experience. The mobile app was designed specifically with the patient role in mind, offering streamlined access to core features such as doctor browsing, appointment booking, and medical record viewing. The architecture follows clean code principles and adopts separation of concerns, making the application scalable and maintainable.

4.4.1 User Authentication and Profile Management

The app implements a secure authentication flow using JWT (JSON Web Token), which is issued by the backend Django REST API upon successful login. Patients can register, log in, and maintain session persistence across app restarts. Once authenticated, users have access to a profile management module where they can view and edit personal details such as their name, email, and contact information. Validation feedback and loading indicators are implemented for a responsive user experience.

4.4.2 Hospital, Department, and Doctor Browsing

Using data fetched from RESTful APIs, the app presents patients with searchable and filterable lists of:

1. All registered doctors
2. All registered hospitals
3. Their associated departments
4. The doctors affiliated with each department

Each doctor's profile includes key information such as specialty, available time slots, and affiliated hospital, allowing patients to make informed choices. Efficient pagination and caching mechanisms ensure smooth data retrieval even on slower networks.

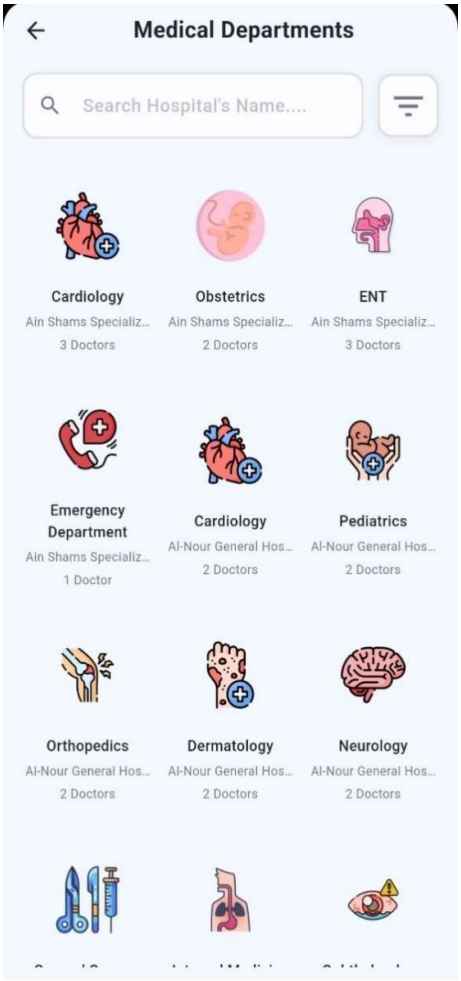

Figure 4.10: Departments List Screen

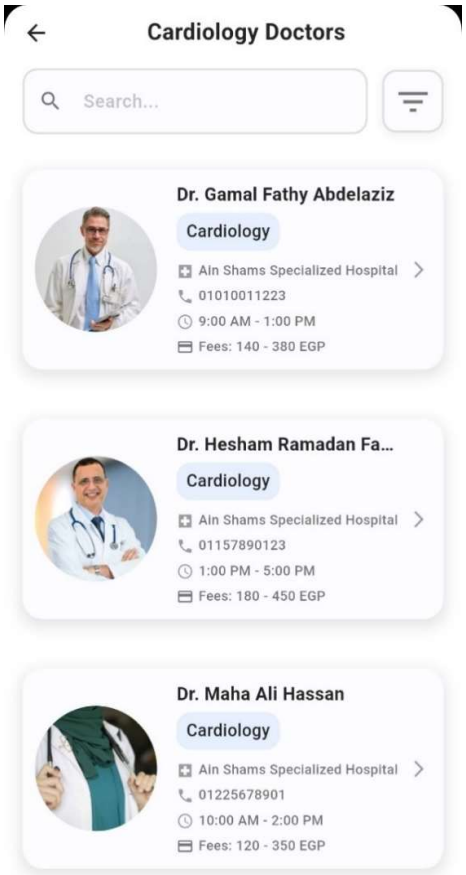

Figure 4.11: Department's Doctors Screen

4.4.3 Appointment Booking and Viewing

Patients can browse available doctors and schedule appointments directly through the mobile interface. After selecting a doctor, the app queries and displays their available visiting hours, allowing patients to select a convenient time slot. Each appointment entry clearly displays its **status** (e.g., Pending, Confirmed, Cancelled), and patients can view detailed appointment information. Booked appointments are categorized as:

- Upcoming Appointments
- Confirmed Appointments
- Cancelled Appointments

4.4.4 API Integration with Backend

The app communicates with the Django backend through RESTful APIs to fetch and submit data such as profile details, hospital information, and appointment records. Data received from the backend is deserialized into Dart models, ensuring consistency between the frontend and backend data structures. The UI is dynamically updated based on the latest responses from the API, promoting a real-time feel for patients using the application. The app's networking layer is powered by the **Dio** HTTP client library, which supports:

- Structured HTTP request/response handling
- Custom interceptors for logging and authentication token injection
- Error management for graceful fallback in case of failed requests

4.5 Implementation of Role-Based Access Control (RBAC)

RBAC was implemented primarily on the backend using Django's decorator and middleware functionalities.

1. View-Level Protection: Decorators (like login_required and custom decorators) are used to protect views, ensuring that only authenticated users with the correct role (e.g., if user.is_doctor:) can access specific pages or perform certain actions.

2. API-Level Protection: DRF permissions classes and the custom JWT middleware were used to secure API endpoints, checking the user's role from their token before allowing access to data. This prevents a patient, for example, from accessing an API endpoint intended for doctors.

4.6 Security Implementation

Building on the principles in Chapter 2 - Section 2.5, several key security features were implemented:

1. Authentication: JWTs are used for secure, stateless session management, especially for the API and sessions auth used for the web.
2. Authorization: RBAC, as described above, forms the authorization strategy.
3. CSRF Protection: Django's built-in Cross-Site Request Forgery protection is enabled for all state-changing forms submitted through the web application.
4. Password Security: Django's default password management system is used, which automatically hashes and salts passwords, ensuring they aren't stored in plaintext.
5. Payment Security: By integrating external payment gateways like PayPal, the system avoids storing sensitive credit card information to the secure provider.

4.7 Deployment

The web application component of the "MediCare" system was successfully deployed to PythonAnywhere.com, a Platform-as-a-Service (PaaS) specializing in hosting Python applications. The deployment process involved:

1. Code Upload: Uploading the Django project code to the PythonAnywhere file system, typically via Git integration.
2. Web App Configuration: Setting up a new web app in the PythonAnywhere dashboard, pointing to the project's WSGI file.
3. Database Setup: Configuring the application to use a database on the platform (while SQLite works, production environments on PythonAnywhere often use MySQL or PostgreSQL, which the platform manages).
4. Static File Configuration: Configuring the server to correctly serve static files (CSS, JavaScript, images).
5. Domain and SSL: Setting up a domain and enabling the provided SSL certificate for secure HTTPS connections.

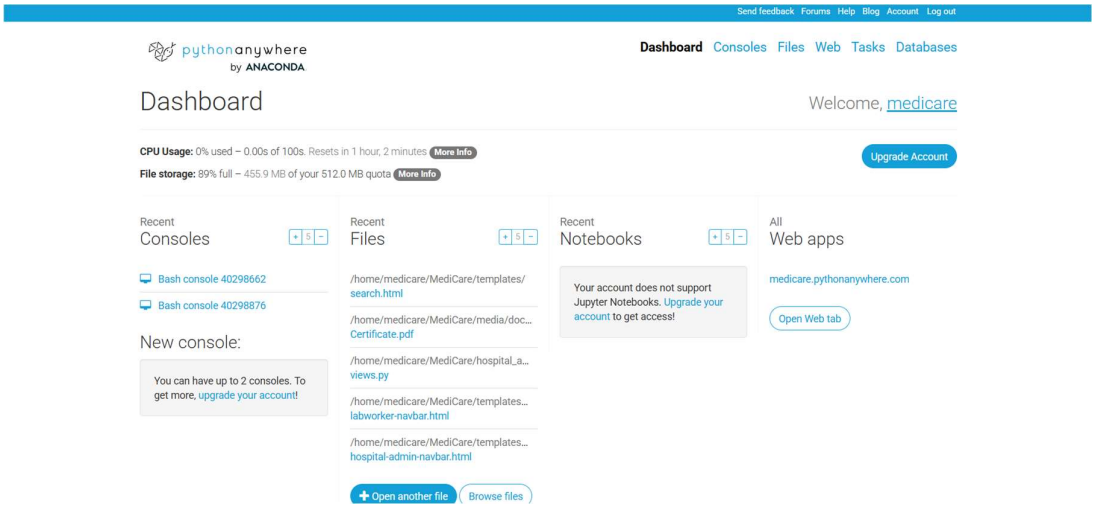

Figure 4.12: PythonAnywhere web app dashboard

CHAPTER 5
Testing and Evaluation

Testing and Evaluation

This chapter provides a comprehensive evaluation of the "MediCare" Hospital Management System, detailing the strategies, methodologies, and results of the testing phase. This phase is crucial for identifying and rectifying defects, verifying that individual modules function correctly, ensuring integrated components cooperate seamlessly, and evaluating the overall system's performance and user experience. The chapter presents the various levels of testing performed, from unit to system-level, and includes sample test cases to demonstrate the system's reliability and readiness for its intended use.

5.1 Testing Strategy and Approach

A multi-layered testing strategy was adopted to ensure the reliability, correctness, and usability of the "MediCare" system across all its modules for both web and mobile platforms. The approach combined manual testing for exploratory and usability aspects with automated testing for repetitive and API-level checks. The testing pyramid concept was followed, with a broad base of unit tests, followed by integration tests, and culminating in system and user acceptance tests. The primary focus areas of our testing strategy were:

1. Functional Testing: To verify that every feature, from patient registration to admin report generation, performs according to its specified functional requirements (as defined in Chapter 2 - Section 2.3.1).

2. Usability Testing: To ensure that the user interfaces for both web and mobile applications are intuitive, user-friendly, and efficient for all target roles (patients, doctors, administrators, etc.).

3. Performance Testing: To evaluate the system's responsiveness, stability, and resource consumption under anticipated load conditions.

4. Security Testing: To identify potential vulnerabilities and ensure that authentication, authorization, and data protection mechanisms are implemented correctly.

5. Compatibility Testing: To ensure the web application renders and functions correctly across major web browsers and that the mobile application performs reliably on different Android versions.

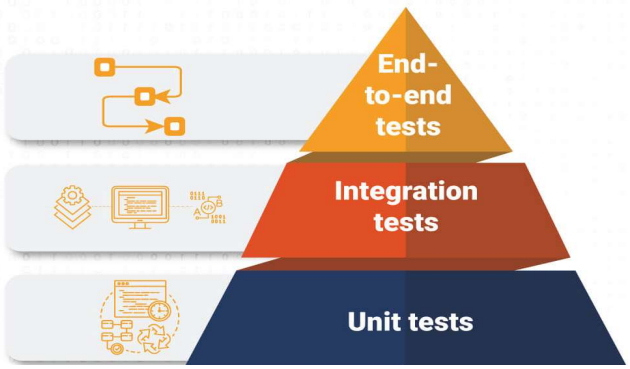

Figure 5.1: Software Testing Pyramid

5.2 Unit Testing (Frontend and Backend)

Unit testing focused on validating the smallest individual components of the application in isolation to ensure they function correctly.

1. Backend Unit Testing (Django):
   - Methodology: Django's built-in testing framework, based on Python's unit test module, was used to create and run tests.
   - Models: Tests were written to verify model field constraints, default values, and custom methods. For example, the Appointment model's validation logic was unit tested to ensure it rejected appointments with dates in the past.
   - Views & Forms: Tests were created to check view logic, ensure correct template rendering, proper handling of GET and POST requests, and valid form processing. For instance, the user login and registration forms were tested for successful validation with correct data, rejection of invalid data, and the presence of CSRF token protection.
   - APIs: While API endpoints were more heavily tested during integration testing with Postman, basic unit tests can verify the behavior of individual serializer fields in isolation.

2. Mobile Unit Testing (Flutter):
   - Methodology: Flutter's built-in testing framework was used to test individual widgets and business logic functions in the mobile app.
   - Widget Tests: These tests verified that UI widgets rendered correctly and responded as expected to user interaction and state changes. For example, a test was created to ensure the "Login" button was giving a warning text until both email and password fields were filled.
   - Logic Tests: Plain Dart functions, such as those for formatting dates or validating user input before sending it to the API, were tested in isolation.

```
Windsurf: Refactor | Explain
class AppointmentModelTest(TestCase):
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def setUp(self):
        # Create patient user and associated Patient record
        self.patient_user = User.objects.create_user(
            username='patient1', password='strongpass'
        )
        self.patient = Patient.objects.create(
            user=self.patient_user,
            username='patient1',
            email='p1@example.com',
            serial_number='SN123456'
        )
        # Create doctor user and associated Doctor_Information record
        self.doctor_user = User.objects.create_user(
            username='doctor1', password='strongpass'
        )
        self.doctor = Doctor_Information.objects.create(
            user=self.doctor_user,
            username='doctor1',
            email='d1@example.com'
        )

    Windsurf: Refactor | Explain | Generate Docstring | ×
    def test_default_appointment_status(self):
        appointment = Appointment.objects.create(
            doctor=self.doctor,
            patient=self.patient,
            date='2025-06-18',
            time='10:00'
        )
        self.assertEqual(
            appointment.appointment_status,
            'pending',
            "Default appointment_status should be 'pending'"
        )
```

Figure 5.2: Sample Django Unit Test for a Model

5.3 Integration Testing

After verifying individual units, integration testing was conducted to ensure that modules and components of the "MediCare" system worked together as a cohesive whole.

1. Web Application Integration Testing:
   - Methodology: Manual end-to-end workflow testing was performed.
   - Example Flows Tested:
     - Patient Appointment Flow: Patient registration → Login → Searching for a doctor → Booking an appointment, Doctor login → Viewing & confirming the appointment.
     - Admin Management Flow: Admin login → Approving a pending doctor registration, Verifying the doctor can now log in.
     - Pharmacy Flow: Patient login → Browsing the "Medical Shop" → Adding medicine to cart → Proceeding to checkout, Pharmacist login → Viewing the order.

2. API Integration Testing:
   - Methodology: Postman was used extensively to test the integration between the mobile app (client) and the Django backend (server).
   - Tests Conducted: Collections of requests were created in Postman to simulate full user workflows, such as registering, logging in (saving the JWT), and then using that token in the header of the requests to access protected endpoints like

fetching appointments or prescriptions. This verified the entire API stack, including routing, serialization, authentication, and database interaction.

5.4 System Testing

System testing was performed on the fully integrated "MediCare" system to evaluate its compliance with all specified requirements. This black-box testing approach treated the entire system as a single entity, focusing on the end-to-end functionality from a user's perspective without knowledge of the internal code structure.

- Methodology: The system was deployed on a staging environment (similar to the final PythonAnywhere environment) and tested against the functional and non-functional requirements documented in Chapter 2.
- Areas Covered:
  - Cross-Browser Compatibility: The web application was manually tested on the latest versions of major browsers, including Google Chrome, Mozilla Firefox, and Microsoft Edge, to ensure consistent rendering and functionality.
  - Mobile Responsiveness: The web application's layout and usability were tested on various screen sizes, from large desktop monitors to small mobile phone viewports, to verify the effectiveness of the Bootstrap responsive design.
  - Security Testing: Basic checks for common vulnerabilities like Cross-Site Scripting (XSS) (mitigated by Django's template auto-escaping). The effectiveness of the RBAC was also tested by attempting to access admin URLs while logged in as a patient, and vice versa.

5.5 User Acceptance Testing (UAT)

While formal UAT with external end-users was outside the project's scope, a simulated UAT was conducted by team members who were not involved in the development of a specific module.

- Methodology: Testers were provided with user-centric scenarios and asked to perform tasks without technical guidance. For example, a "patient" tester was asked to "Find a cardiologist and book an appointment for next Tuesday," while an "admin" tester was asked to "Add a new department to Qena Hospital."
- Objective: The goal was to evaluate the system's usability, intuitiveness, and overall user experience. Feedback was collected on ease of navigation, clarity of instructions, and whether the system behaved as a non-technical user would expect. This feedback was crucial for making final UI/UX refinements.

5.6 Performance Testing

Performance testing was conducted to measure the system's responsiveness, stability, and speed underload. The primary tools used were Postman for API response time measurement and browser developer tools for web page load analysis.

- Page Load Time: Key pages of the web application, such as the homepage, patient dashboard, and doctor listings, were tested for load times. The target was to keep load times under 3 seconds for a positive user experience.
- API Response Time: The response times for critical API endpoints used by the mobile app (e.g., login, fetch appointments, get prescriptions) were measured using Postman. The average response time was found to be about 500ms, ensuring a smooth mobile experience.
- Database Query Optimization: Django's ORM helps prevent many inefficient queries, but key lookups were analyzed. Fields frequently used in filtering, such as doctor_id, patient_id, and appointment_date, are indexed by default as foreign keys or primary keys, which significantly improves query performance.

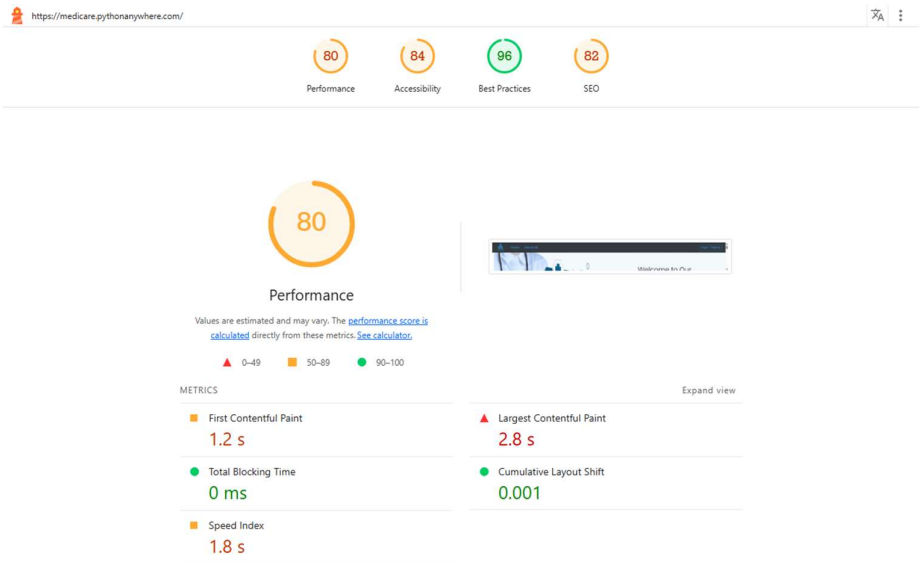

Figure 5.3: Performance Testing

5.7 Test Cases and Results

To ensure the **functional reliability**, **correctness**, and **usability** of the *MediCare* Hospital Management System, a structured set of test cases was developed and executed for both the web and mobile platforms. These test cases validate that the implemented features align with the system's functional requirements, as described in the software design and requirement specification phases.

5.7.1 Web Application Test Cases

Test Case ID	Feature / Module	Test Description	Steps to Reproduce	Status
TC-WEB-01	Patient Registration	Verifying a new patient can register with valid data.	1. Navigate to Sign Up 2. Enter valid details. 3. Click "Register."	✓
TC-WEB-02	Invalid Login	Verify system rejects login with incorrect credentials.	1. Navigate to Login. 2. Enter valid email, wrong password. 3. Click "Login."	✓
TC-WEB-03	Appointment Booking	Verify a logged-in patient can book an appointment.	1. Login as patient. 2. Search for a doctor. 3. Click "Book Appointment." 4. Select date/time.	✓
TC-WEB-04	Doctor Appointment Mgmt.	Verify a doctor can accept a pending appointment.	1. Login as doctor. 2. Go to dashboard. 3. Find pending appointment. 4. Click "Confirm."	✓
TC-WEB-05	Admin Hospital Mgmt.	Verifying an admin can add a new hospital.	1. Login as admin. 2. Go to Hospitals 3. Add Hospital. 4. Fill form. 5. Submit.	✓
TC-WEB-06	Pharmacy Inventory	Verify a pharmacist can add a new medicine to the inventory.	1. Login as pharmacist 2. Go to Medicines Mgmt. 3. Add Medicine. 4. Fill form. 5. Submit.	✓
TC-WEB-07	Prescription	Verify a doctor can generate prescription for a patient.	1. Login as doctor. 2. Select a patient. 3. Create prescription.	✓

Table 5.1: Web Application Test Cases

5.7.2 Mobile Application Test Cases

Test Case ID	Feature / Module	Test Description	Steps to Reproduce	Status
TC-MOB-01	Mobile Login	Verify a patient can log in via the mobile app.	1. Open app. 2. Enter valid patient credentials. 3. Tap "Login."	✓
TC-MOB-02	API Data Fetch	Verify the mobile app can fetch and display a list of hospitals.	1. Login as patient. 2. Navigate to the "Hospitals" screen.	✓
TC-MOB-03	Mobile Appointment View	Verify a patient can view their booked appointments on the mobile app.	1. Login as patient. 2. Navigate to "My Appointments."	✓
TC-MOB-04	Mobile Profile Edit	Verify a patient can update their profile information.	1. Login. 2. Go to Profile Edit. 3. Change a field (e.g., phone number). 4. Save.	✓

Table 5.2: Mobile Application Test Cases

5.8 Bug Reporting and Resolution

During the testing phases, any identified defects or unexpected behaviors were systematically tracked and managed.

1. Tracking Method: A shared document or a simple issue tracker (like GitHub Issues) was used to log bugs.
2. Bug Report Details: Each bug report included a unique ID, a descriptive title, steps to reproduce, the expected result, the actual result, and its severity (e.g., Critical, Major, Minor).
3. Resolution Process: Developers were assigned bugs based on the module affected. Once a fix was implemented, the change was committed to the Git repository. The tester who reported the bug would then re-test the functionality in a clean environment to verify the fix before closing the issue. This iterative process ensured that defects were resolved efficiently and did not introduce new regressions.

5.9 Evaluation of Results against Objectives

The final phase of testing involves evaluating the outcomes against the primary project objectives outlined in Chapter 1 – Section 1.4. The successful execution of the test cases demonstrates that the "MediCare" system has met its core goals.

Project Objective	Evaluation Method & Result	Status
1. Design a comprehensive, integrated HMS (Web & Mobile)	System testing (5.4) and integration testing (5.3) confirmed that the web platform (with all its modules) and the mobile application work together as a single, integrated system via the REST API.	Met
2. Empower patients (via Web & Mobile)	UAT scenarios (5.5) and functional test cases (TC-WEB-01, TC-WEB-03, TC-MOB-01 to 04) confirmed that patients can register, book appointments, view records, and manage their profiles on both platforms.	Met
3. Equip doctors and clinical staff with efficient tools	Functional test cases (TC-WEB-04, TC-WEB-07) and workflow testing confirmed that doctors can manage appointments, view patient data, and create prescriptions efficiently through their dedicated web dashboard.	Met
4. Provide hospital administrators with robust tools for control	Functional test cases (TC-WEB-05) and manual system testing confirmed that admins can manage users, hospitals, and view system data from their centralized panel.	Met
5. Ensure a secure, reliable, and user-friendly interface	Security testing (5.4), NFRs for usability (USA-001 to 003), and UAT feedback (5.5) confirmed that the system is intuitive, and that security measures like RBAC and JWT & session authentications are functioning correctly.	Met
6. Build a scalable and maintainable system architecture	Partially Achieved & Demonstrated. The use of a modular Django architecture and a decoupled Flutter app demonstrates a maintainable and scalable design. While SQLite limits current data scalability, the use of Django's ORM ensures the system is ready for migration to a more robust database, meeting the architectural goal.	Met
7. Integrate essential third-party services (Payment, Email)	Integration testing confirmed that email notifications (via Mailtrap for dev) and payment processing (via PayPal sandbox) are working as intended within their respective workflows.	Met

Table 5.3: Evaluation of Objectives

CHAPTER 6

Conclusion and Future Work

Conclusion and Future Work

The primary objective of this project was to design, implement, and deploy a robust, secure, and user-friendly Hospital Management System (HMS) capable of digitizing and streamlining day-to-day operations across hospitals. This objective was successfully realized with the development of "MediCare", an integrated dual-platform solution offering distinct functionalities through both web and mobile interfaces.

6.1 Summary of Achievements

The primary objective of this project was to design, develop, and deploy a robust, user-friendly, and secure Hospital Management System to streamline and digitize hospital operations. This goal was successfully met through the creation of the "MediCare" platform, an integrated web and mobile solution that facilitates efficient coordination among patients, doctors, hospital administrators, lab workers, and pharmacists.

The key accomplishments of this project include:

1. Development of an Integrated Dual-Platform System: A comprehensive web application was built using Django and Bootstrap, complemented by a patient-centric mobile application developed with Flutter, providing a cohesive user experience across devices.

2. Comprehensive Multi-Role Access System: A secure system with distinct, role-based dashboards and privileges was implemented for all key stakeholders: Patients, Doctors, Hospital Administrators, Lab Workers, and Pharmacists.

3. Full-Featured Patient and Doctor Modules: The system provides extensive functionalities, including online appointment scheduling, secure medical record access (prescriptions and lab reports).

4. Complete Administrative and Operational Control: The Admin dashboard provides tools to manage hospital information, user registrations (doctor approvals), staff accounts, and system-wide activities.

5. Integrated Pharmacy and Billing Modules: A functional online "Medical Shop" with inventory management for pharmacists and an integrated payment system (utilizing PayPal) for services have been successfully implemented

6. Secure and Scalable Architecture: Data security was ensured through robust Role-Based Access Control (RBAC), JWT authentication for APIs, session authentication for the web, and HTTPS encryption. The system's modular architecture, built on Django and deployed on PythonAnywhere, is designed for future scalability.

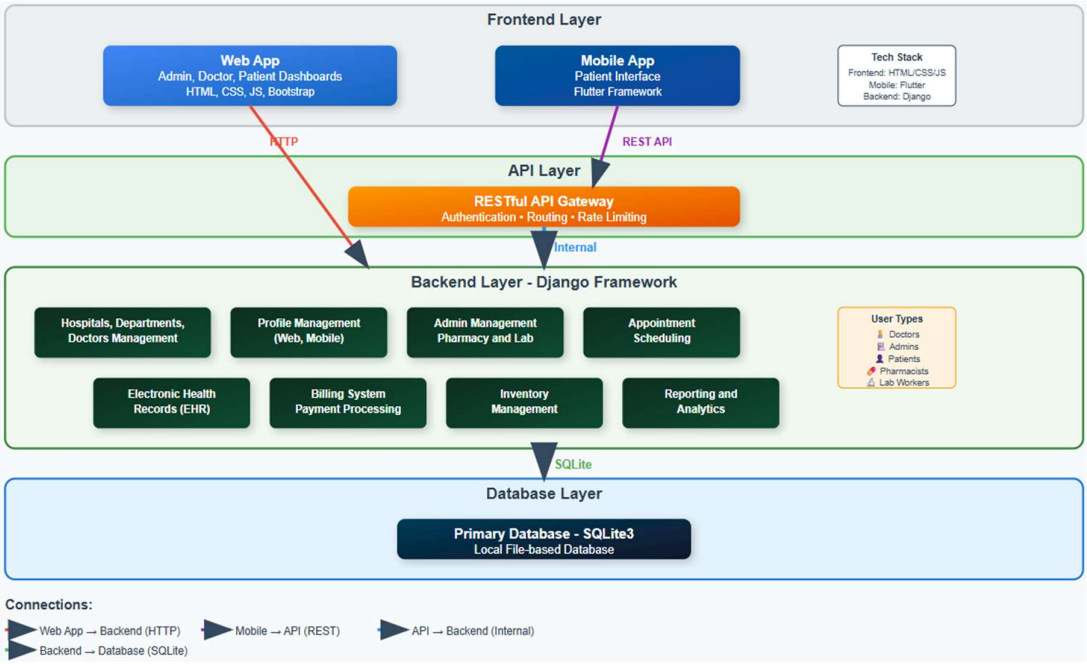

Figure 6.1: System Architecture and Functional Modules of MediCare

6.2 Challenges Encountered and Lessons Learned

The development of a complex system like "MediCare" presented several challenges, each providing valuable learning opportunities for the project team.

1. Managing Multi-Role Complexity: Implementing and securing distinct workflows for five different user roles (Patient, Doctor, Admin, Lab Worker, Pharmacist) was a significant architectural challenge.
   - Lesson Learned: The importance of a robust Role-Based Access Control (RBAC) system, defined early in the design phase, was paramount. Using Django's built-in user model with custom Boolean flags proved to be an effective and scalable approach for managing permissions at both the view and API level.

2. Cross-Platform Data Consistency: Ensuring that data entered or modified on the web platform was instantly and accurately reflected in the Flutter mobile app (and vice-versa) required a meticulously designed API.

- Lesson Learned: A well-structured RESTful API using Django REST Framework is the critical bridge for such systems. Rigorous testing of API endpoints with tools like Postman was essential to guarantee data integrity and a seamless user experience across platforms.

3. Third-Party Service Integration: Integrating external services like a payment gateway (PayPal) introduced external dependencies and required careful handling of SDKs, API keys, and asynchronous responses.
   - Lesson Learned: Thoroughly reading the documentation for third-party services and using sandboxed environments for testing are crucial steps. This mitigates risks and ensures that integrations are secure and reliable before deployment.

4. Challenge 4: Scope Management within a Timeline: The project had an ambitious scope. Balancing the development of numerous features with the project timeline requires effective planning and prioritization.
   - Lesson Learned: Adopting an Agile methodology was highly beneficial. It allowed the team to prioritize core functionalities first and then iteratively add features, ensuring that a viable product was always in a deliverable state throughout the development cycle.

6.3 Limitations of the Current System

While the "MediCare" system successfully covers the core hospital workflows as defined in the project scope, it is important to acknowledge certain limitations in its current iteration. These limitations exist at both the system-wide level and within specific platforms (web vs. mobile).

6.3.1 System-Wide Limitations:

These limitations pertain to the overall architecture and feature set of the "MediCare" system, affecting both the web and mobile platforms.

1. No Offline Functionality: The system is entirely dependent on an active internet connection. Neither the web nor the mobile application currently supports offline data access or synchronization, which could be a significant limitation in areas with poor or intermittent connectivity.

2. Limited AI and Advanced Analytics: The project lays the groundwork for data collection, but it does not yet implement Artificial Intelligence (AI) for tasks like

diagnostic suggestions or predictive analytics. The reporting features are currently limited to basic statistics rather than advanced data visualizations or business intelligence dashboards.

3. Scalability Constraints with Current Database: The use of SQLite, while perfect for development and the current scale of deployment, is not suitable for a large-scale, high-concurrency production environment that would be expected in a national or large hospital network deployment.

4. Single Language Support: The user interface currently supports only English, which may present a barrier to usability in non-English speaking regions or for users who prefer their native language.

5. Lack of HL7/FHIR Compliance: The system does not currently adhere to healthcare interoperability standards like HL7 or FHIR, which would be necessary for seamless data exchange with other certified healthcare systems.

6. Lack of Real-Time Synchronous Communication:
   - No Instant Chat: A real-time, synchronous chat feature between doctors and patients has not been implemented on either the web or mobile platform.
   - No Push Notifications: While email notifications are implemented, the system lacks real-time mobile push notifications for immediate alerts.

6.3.2Mobile Application Specific Limitations:

While the Android application provides core patient functionalities, there are several features available on the web platform that have not yet been implemented on the mobile app:

1. Single User Role (Patient Only): The current mobile application is designed exclusively for the "Patient" role. It does not provide interfaces for Doctors, Administrators, Lab Workers, or Pharmacists, who must use the web application to perform their duties.

2. No Document Downloads: Patients can view their prescriptions and lab reports on the mobile app, but the functionality to download these documents as PDF files is only available on the web platform.

3. Absence of Lab Test Booking: Although patients can see which tests have been prescribed for them, the mobile app does not currently have the functionality to book or pay for these lab tests. This action must be completed through the web.

4. No Online Pharmacy ('Medical Shop'): The mobile application does not include the "Medical Shop" module. Therefore, patients cannot browse the pharmacy inventory, add medicines to a cart, or make a purchase through the app; this entire workflow is exclusive to the web application.

5. Absence of a Centralized Billing Interface: Flowing from the points above, the mobile app lacks any billing or payment gateway integration. There is no section to pay for services or view a history of financial transactions, functionalities which are present on the web platform.

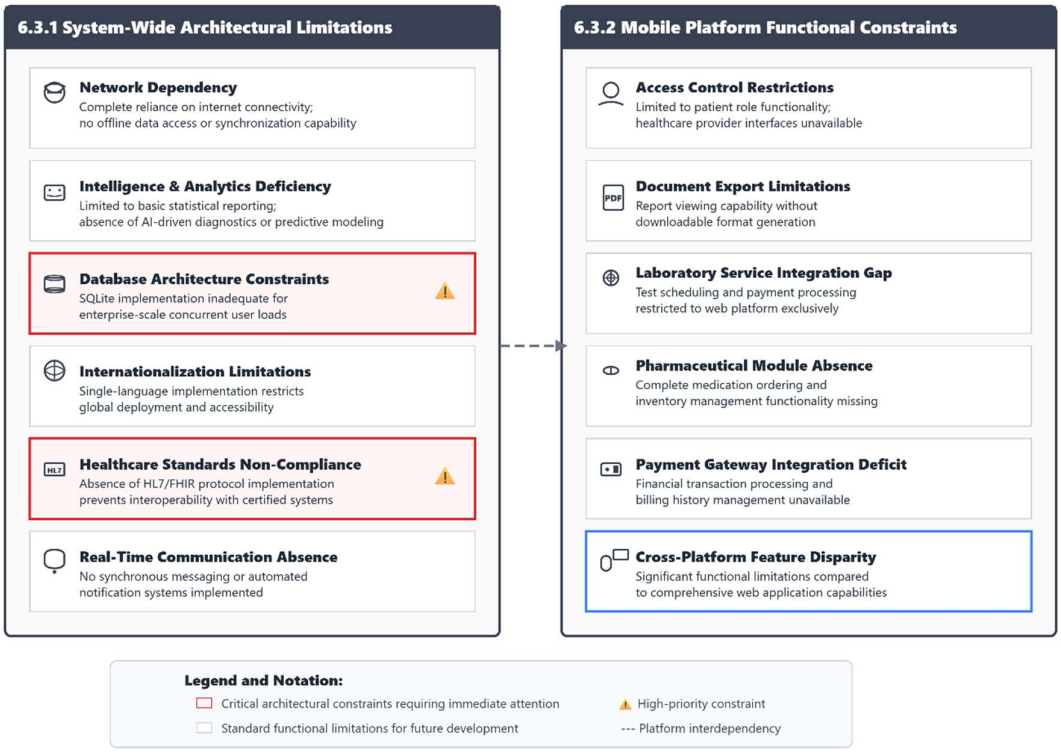

Figure 6.2: Identified System Limitations and Constrains

6.4 Future Enhancements and Scope

Building on the stable platform established by the "MediCare" project, and in direct response to the limitations identified in the previous section, a clear roadmap of future enhancements can be defined. These improvements aim to address current gaps, expand functionality, and transform the system into a next-generation smart health platform.

6.4.1 Web Application Future Work

Enhancements of the web platform will focus on adding intelligence, improving scalability, and increasing user engagement.

1. AI-Powered Clinical Decision Support & Advanced Analytics: To address the current lack of advanced analytics, machine learning models could be integrated to provide doctors with diagnostic suggestions based on patient symptoms and history. The admin panel could be enhanced with a business intelligence dashboard (using libraries like Chart.js or D3.js) to offer rich data visualizations on patient demographics, financial trends, and operational efficiency.

2. Real-Time Communication (Telemedicine & Chat): To overcome the absence of real-time communication, a secure video consultation module could be built directly into the web platform for telemedicine appointments. Concurrently, a real-time text chat feature using WebSockets could be implemented to allow for instant communication between patients and doctors.

3. Internationalization (Multilingual Support): To break the single-language barrier, the web application could be refactored to support multiple languages (e.g., Arabic). This would involve abstracting all user-facing text into language files and allowing users to switch their preferred language.

4. Backend and Database Scalability: To resolve the scalability constraints of SQLite, the immediate next step for a production-grade system would be to migrate the database to a more robust solution like PostgreSQL or MySQL. This is facilitated by Django's ORM and would be essential for handling a large number of concurrent users and a growing volume of data.

6.4.2 Mobile Application Future Work

The primary goal for the mobile application is to achieve full feature parity with the web platform while also introducing mobile-specific enhancements.

1. Full Feature Implementation for Patient App: To address the current feature disparity, the following modules from the web application would be developed within the patient mobile app:
   - Online Pharmacy ("Medical Shop"): Implementing the full pharmacy module, allowing patients to browse, add to cart, and purchase medicines.

- Billing and Test Booking: Integrating the payment system to allow patients to book and pay for lab tests and other services directly through the app.
- Document Downloads: Adding functionality for patients to securely download their prescriptions and lab reports as PDF files to their devices.

2. Dedicated Doctor-Side Application: To address the "patient-only" limitation, a separate Flutter application would be developed specifically for doctors. This would allow them to manage their schedules, view patient records, accept appointments, and potentially conduct video consultations on the go.

3. Real-Time Push Notifications: To complement the existing email notification system, Firebase Cloud Messaging (FCM) would be integrated to send real-time push notifications directly to the mobile application. This would provide immediate alerts for appointment confirmations, reminders, and future chat messages, significantly improving user engagement.

4. Offline Functionality: To support users in areas with poor connectivity, offline capabilities would be implemented. This could involve caching essential data (like upcoming appointments and prescriptions) locally on the device and creating a queueing system to sync any actions (like appointment requests) once an internet connection is restored.

5. Wearable Device Integration & iOS Deployment: Further enhancements would include integrating with health tracking data from wearable devices (via Google Fit/Apple HealthKit APIs) and compiling the existing Flutter codebase to release an iOS version of the patient application, doubling the potential user base.

6.4.3 Integration with Other Systems

To ensure "MediCare" can operate effectively within a broader healthcare ecosystem, the following integrations are critical future steps.

1. HL7/FHIR Compliance for Interoperability: To address the current lack of interoperability, a key priority would be to refactor data models and create API endpoints that adhere to HL7/FHIR standards. This would enable seamless and secure data exchange with other certified hospitals, external laboratories, and public health systems.

2. Insurance and Billing Automation: The billing module could be enhanced by integrating with insurance company APIs. This would automate the process of verifying patient insurance coverage and submitting electronic claims, drastically reducing administrative workload.

3. Blockchain for Enhanced Record Security: For unparalleled data integrity and patient control, a future iteration could explore the use of blockchain technology to create a tamper-proof, decentralized, and patient-controlled ledger for records.

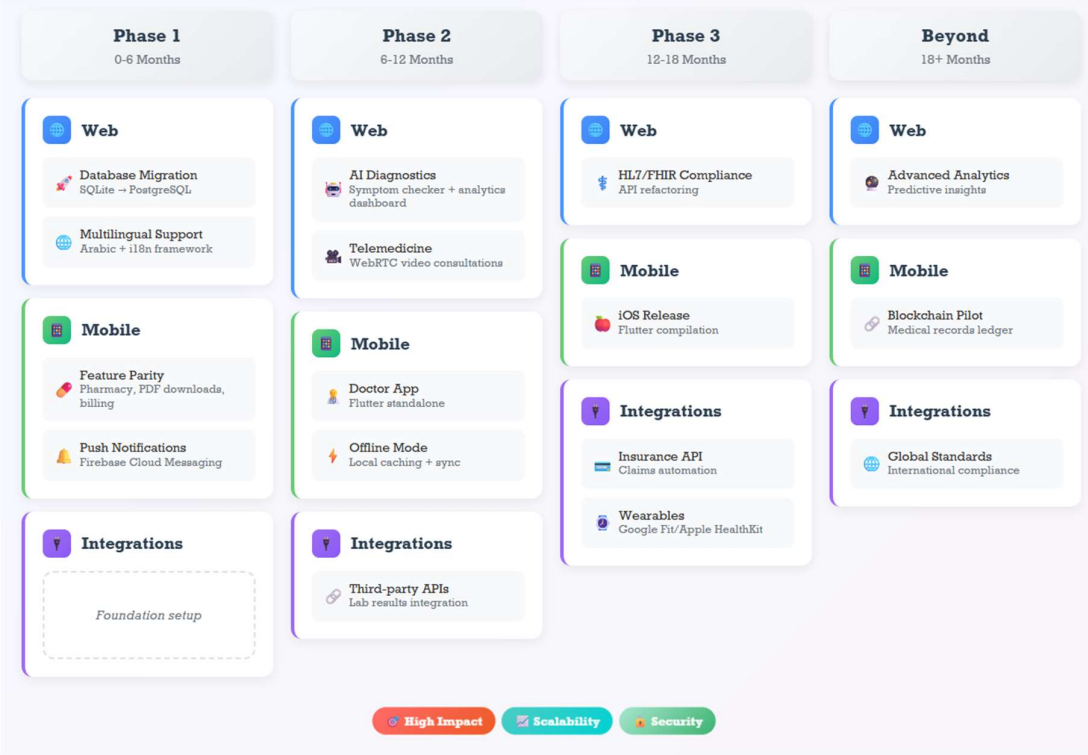

Figure 6.3: Proposed Roadmap for HMS Expansion and Innovation

6.5 Final Reflections and Conclusion

This project successfully achieved its goal of developing a comprehensive, integrated, and secure Hospital Management System. The "MediCare" platform represents a significant step toward digital healthcare transformation, offering a cost-effective and scalable solution that addresses the core needs of hospital administration, clinical staff, and patients. By automating essential functions, the system is poised to improve resource management, enhance the patient experience, ensure data accuracy, and reduce administrative overhead.

The journey of creating "MediCare" from concept to a deployed application has been an immense learning experience, bridging theoretical engineering knowledge with the practical challenges of real-world software development. While the current system is fully functional within its defined scope, the true potential of "MediCare" lies in its future evolution. The planned integration of AI, telemedicine, advanced mobile capabilities, and interoperability standards will further amplify its value in modern healthcare delivery.

In conclusion, "MediCare" provides a robust foundation for an intelligent, adaptive, and inclusive healthcare ecosystem. It demonstrates the power of modern web and mobile technologies to create solutions that can evolve with technological advances and meet the ever-changing demands of the healthcare industry, ultimately contributing to better patient outcomes and more efficient healthcare delivery for all.

References and Appendices

References

[1] S. Wickramasinghe, "Choosing the Best Web Application Architecture," *Crowdbotics Blog*, Oct. 6, 2021. [Online]. Available: https://crowdbotics.com/posts/blog/how-to-choose-the-best-architecture-for-your-web-application

[2] C. Singh, "Fundamentals Of Web Application Architecture," *Appventurez Blog*, Sep. 18, 2024. [Online]. Available: https://www.appventurez.com/blog/web-application-architecture

[3] O. I. Malik, "What is Sensitive Data Exposure Vulnerability & How to Avoid It?," *Securiti Blog*, Aug. 12, 2023. [Online]. Available: https://securiti.ai/blog/sensitive-data-exposure/

[4] E. Makieiev, "How to Create a Modern Web Application Architecture?," *Integrio Systems Blog*, Jan. 7, 2021. [Online]. Available: https://integrio.net/blog/modern-web-application-architecture

[5] A. Singleton, "Word Processing in HTML," *artandlogic.com Blog*, Oct. 8, 2024. [Online]. Available: https://artandlogic.com/word-processing-in-html/

[6] D. Gaizauskas, "What Is CSS and How Does It Work?," *Hostinger Tutorials*, April. 8, 2024. [Online]. Available: https://www.hostinger.com/tutorials/what-is-css

[7] R. Miles and K. Hamilton, "Learning UML 2.0: A Pragmatic Introduction to UML," 1st ed. *O'Reilly Media*, 2006. Available: https://shorturl.at/pHRxA

[8] Django Software Foundation, "Django Documentation (v5.0)," 2024. [Online]. Available: https://docs.djangoproject.com/

[9] Google, "Flutter Documentation," 2024. [Online]. Available: https://docs.flutter.dev/

[10] Google, "Dart Language Documentation," 2024. [Online]. Available: https://dart.dev/guides

[11] The Flutter Team, "dio: A powerful HTTP networking package for Dart and Flutter," *pub.dev*. [Online]. Available: https://pub.dev/packages/dio

[12] The Flutter Team, "shared_preferences: platform-specific persistent storage for simple data," *pub.dev*. [Online]. Available: https://pub.dev/packages/shared_preferences

[13] American Institute of Health Care Professionals, "Ethical Issues in Healthcare: Patient Confidentiality Challenges," *AIHCP Health Care Blog*, Oct. 3, 2024. [Online]. Available: https://shorturl.at/3hFdq

[14] Health Level Seven International, "FHIR – Fast Healthcare Interoperability Resources," *HL7.org*. [Online]. Available: https://www.hl7.org/fhir/

[15] A. Dennis, B. H. Wixom and R. M. Roth, "System Analysis and Design," 5th ed. *University of Information Technology and Communications,* [Online].
Available: https://shorturl.at/qXg2B

[16] O. O. Lawal, B. O. Afeni, and J. O. Mebawondu, "Development of Hospital Information Management Systems," *Joseph Ayo Babalola University*, *The Federal University of Technology*, Oct. 2016. [Online]. Available: https://shorturl.at/EZSvF

[17] O. Michael, "Design and implementation of a hospital management system," *SlideShare,* 2020. [Online]. Available: https://rb.gy/wu9qr9

[18] Avanttec Medical Systems, "Hospital Management System (HMS)," *avanttec.net*. [Online]. Available: https://www.avanttec.net/hospital-management-system/

[19] OpenMRS Team, "OpenMRS Architecture Guide Documentation," *OpenMRS Wiki*. [Online]. Available: https://wiki.openmrs.org/display/docs

[20] Stanford Medicine, "Clinical Trials and Patient-Centered Digital Health Systems," *Stanford Medicine*, 2024. [Online].
Available: https://med.stanford.edu/clinicaltrials

[21] E. J. Topol, "High-performance medicine: the convergence of human and artificial intelligence," *Nature Medicine*, vol. 25, no. 1, pp. 44–56, Jan. 2019. [Online]. Available: https://doi.org/10.1038/s41591-018-0300-7

[22] USAID, "Health Management Information System (HMIS) Facilitator's Guide for Training of Trainers," *MEASURE Evaluation*, Rep. MS-13-74, Version 1.1, Feb. 2010.

[Online]. Available: https://www.measureevaluation.org/resources/publications/ms-13-74/at_download/document

[23] World Health Organization, *Digital Health Guidelines: Recommendations on Digital Interventions for Health System Strengthening*, Geneva, Switzerland: WHO, 2019. [Online]. Available: https://www.who.int/publications/i/item/9789241550505

[24] M. Malewicz, "The User Interface (UI) Design," *Interaction Design Foundation*, 2020. [Online]. Available: https://www.interaction-design.org/literature/topics/ui-design.

[25] F. Guimarães and Aela Team, "Nielsen's Heuristics: 10 Usability Principles to Improve UI Design," *Aela School Blog*, Apr. 15, 2021. [Online]. Available: https://www.aela.io/en/blog/all/10-usability-heuristics-ui-design

[26] Google, "About PageSpeed Insights v5," *Google Developers*, Oct. 2024. [Online]. Available: https://developers.google.com/speed/docs/insights/v5/about

[27] PostgreSQL Global Development Group, "PostgreSQL Documentation," *PostgreSQL*, 2024. [Online]. Available: https://www.postgresql.org/docs/

[28] Amazon Web Services, "Deploying Web Applications on AWS Cloud Infrastructure," *AWS*, 2024. [Online]. Available: https://aws.amazon.com/getting-started/

[29] Talentelgia Technologies, "How Much Does EHR Implementation Cost?" *Talentelgia.com*, 2024. [Online]. Available: https://www.talentelgia.com/blog/how-much-does-ehr-implementation-cost

[30] Wikipedia, "Electronic Health Record – Implementation Costs," *Wikipedia*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Electronic_health_record

[31] ScienceSoft, "How Much Does It Cost to Build a Hospital App?" *ScienceSoft*, 2024. [Online]. Available: https://www.scnsoft.com/healthcare/mobile-app-development

[32] Octal IT Solution, "How Much Does It Cost to Develop a Medical App?" *Octalsoft*, 2023. [Online]. Available: https://www.octalsoftware.com/blog/healthcare-app-development-cost

[33] ScienceSoft, "Custom Healthcare Software Development: Cost and Features," *ScienceSoft*, 2024. [Online]. Available: https://www.scnsoft.com/healthcare

Appendices

Appendix A: Sample Code Snippets:

```python
class PatientProfileSerializer(serializers.ModelSerializer):
    class Meta:
        model = Patient
        fields = '__all__'

class PatientAppointmentSerializer(serializers.ModelSerializer):
    appointment_status = serializers.CharField(default="pending", read_only=True)
    payment_status  = serializers.CharField(default="pending", read_only=True)
    patient = serializers.CharField(default=serializers.CurrentUserDefault(),read_only=True)
    serial_number = serializers.CharField(read_only=True)
    transaction_id = serializers.CharField(read_only=True)
    class Meta:
        model = Appointment
        fields = '__all__'

class PrescriptionSerializer(serializers.ModelSerializer):
    class Meta:
        model = Prescription
        fields = '__all__'

class PrescriptionMedicineSerializer(serializers.ModelSerializer):
    class Meta:
        model = Prescription_medicine
        fields = '__all__'

class PrescriptionTestSerializer(serializers.ModelSerializer):
    class Meta:
        model = Prescription_test
        fields = '__all__'

class ReportSerializer(serializers.ModelSerializer):
    class Meta:
        model = Report
        fields = '__all__'
```

Figure A.1 Backend: Django REST Framework API Serializer

```python
    @csrf_exempt
    @login_required(login_url="doctor:doctor-login")
    def accept_appointment(request, pk):
        appointment = Appointment.objects.get(id=pk)
        appointment.appointment_status = 'confirmed'
        appointment.save()

        # Mailtrap

        patient_email = appointment.patient.email
        patient_name = appointment.patient.name
        patient_username = appointment.patient.username
        patient_serial_number = appointment.patient.serial_number
        doctor_name = appointment.doctor.name

        appointment_serial_number = appointment.serial_number
        appointment_date = appointment.date
        appointment_time = appointment.time
        appointment_status = appointment.appointment_status

        subject = "Appointment Acceptance Email"

        values = {
                "email":patient_email,
                "name":patient_name,
                "username":patient_username,
                "serial_number":patient_serial_number,
                "doctor_name":doctor_name,
                "appointment_serial_num":appointment_serial_number,
                "appointment_date":appointment_date,
                "appointment_time":appointment_time,
                "appointment_status":appointment_status,
        }

        html_message = render_to_string('appointment_accept_mail.html', {'values': values})
        plain_message = strip_tags(html_message)
```

Figure A.2 Backend: Django View with Business Logic

```python
class Patient(models.Model):
    patient_id = models.AutoField(primary_key=True)
    user = models.OneToOneField(User, on_delete=models.CASCADE, null=True, blank=True, related_name='patient')
    name = models.CharField(max_length=200, null=True, blank=True)
    username = models.CharField(max_length=200, null=True, blank=True)
    age = models.IntegerField(null=True, blank=True)
    email = models.EmailField(max_length=200, null=True, blank=True)
    phone_number = models.IntegerField(null=True, blank=True)
    address = models.CharField(max_length=200, null=True, blank=True)
    featured_image = models.ImageField(upload_to='patients/', default='patients/user-default.png', null=True, blank=True)
    blood_group = models.CharField(max_length=200, null=True, blank=True)
    history = models.CharField(max_length=200, null=True, blank=True)
    dob = models.CharField(max_length=200, null=True, blank=True)
    nid = models.CharField(max_length=200, null=True, blank=True)
    serial_number = models.CharField(max_length=20,unique=True,default=str(uuid.uuid4())[:8])

    # Chat
    login_status = models.CharField(max_length=200, null=False, blank=False, default="offline")

    def __str__(self):
        return str(self.user.username) # type: ignore
```

Figure A.3 Backend: Custom User Model

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: ColorsManager.lightBlue,
      body: SafeArea(
        child: Padding(
          padding: EdgeInsets.symmetric(vertical: 10.h, horizontal: 14.w),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const CustomTopBar(title: 'Find Doctor'),
              verticalSpace(15),
              SearchAndFilterBar(
                searchController: searchController,
                onFilterPressed: _onFilterPressed,
                hintText: "Search Doctors ....",
              ),
              verticalSpace(10),
              Expanded(
                child: DoctorsListBlocBuilder(
                  searchQuery: searchQuery,
                  isSorted: isSorted,
                ),
              ),
            ],
          )
        )
      ),
    );
  }
}
```

Figure A.4 Mobile: Flutter Widget for UI (Doctors List Screen)

```dart
@RestApi(baseUrl: ApiConstants.apiBaseUrl)
abstract class ApiService {
  factory ApiService(Dio dio, {String baseUrl}) = _ApiService;

  @POST(ApiConstants.login)
  Future<LoginResponse> login(
    @Body() LoginRequestBody loginRequestBody,
  );

  @POST(ApiConstants.signup)
  Future<SignupResponse> signup(
    @Body() SignupRequestBody signupRequestBody,
  );

  @POST(ApiConstants.resetPassword)
  Future<ForgetPasswordResponse> requestPasswordReset(
    @Body() ForgetPasswordRequestBody forgetPasswordRequestBody,
  );

  @POST(ApiConstants.appointments)
  Future<BookAppointmentResponseModel> bookAppointment(
    @Body() BookAppointmentRequestModel bookAppointmentRequestBody,
  );
```

Figure A.5 Mobile: Flutter API Service/Client

Appendix B: Screenshots of the Web and Mobile Application Interface

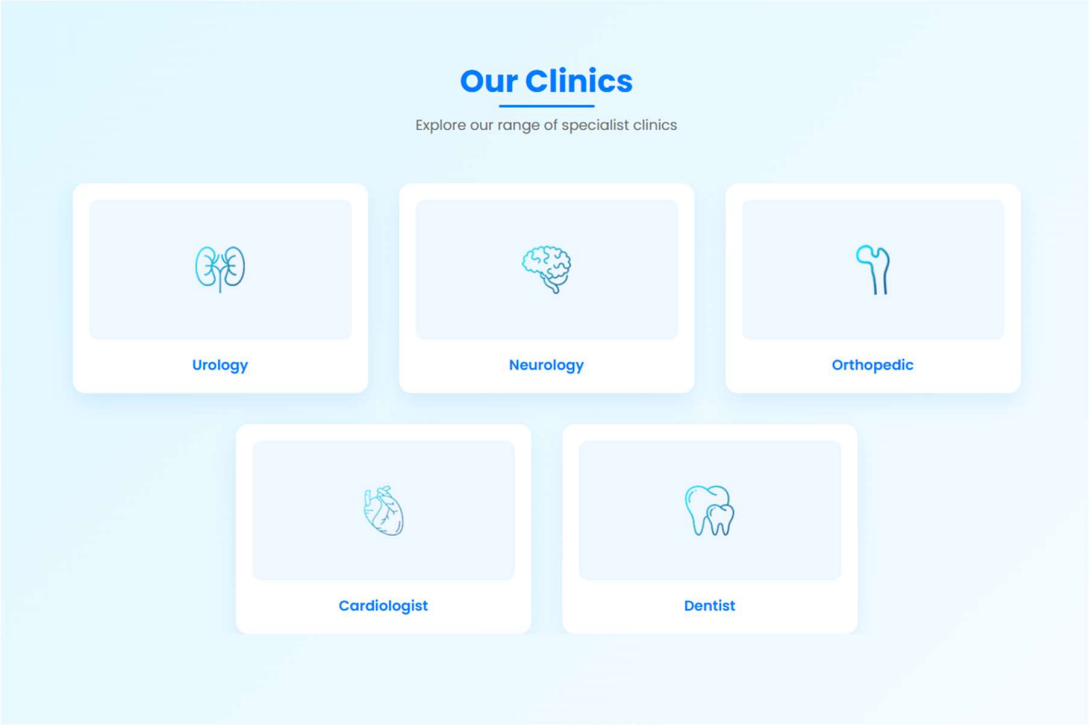

Figure B.1: Public Facing Web Page (Clinics Section on the home screen)

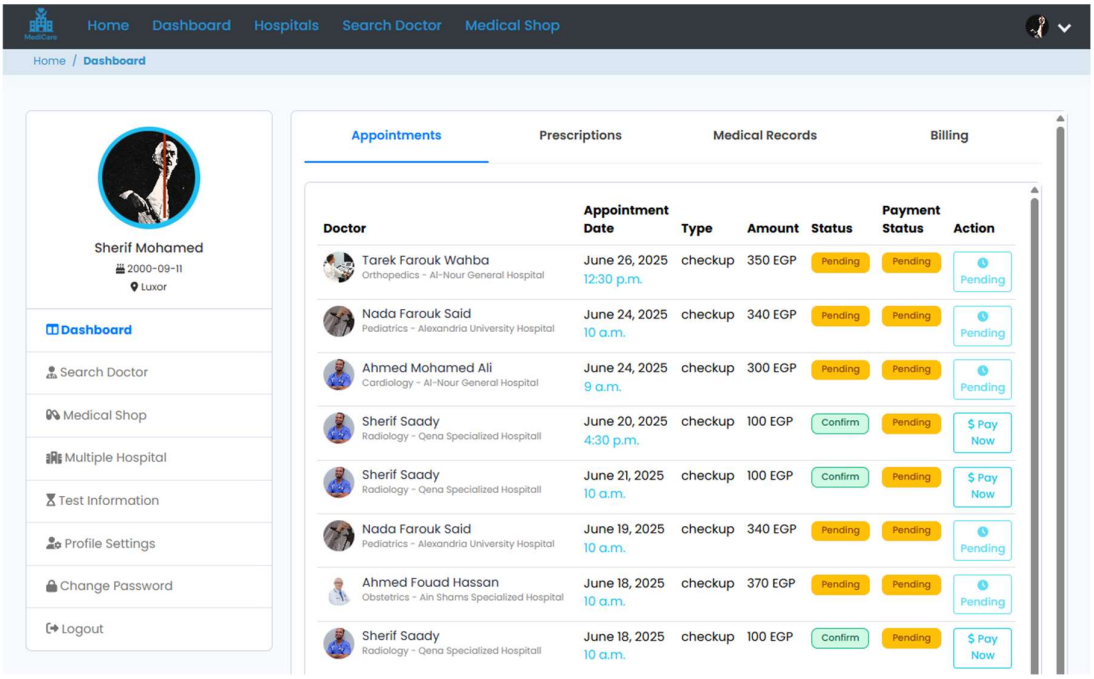

Figure B.2: Patient Role Web Page (Patient Dashboard)

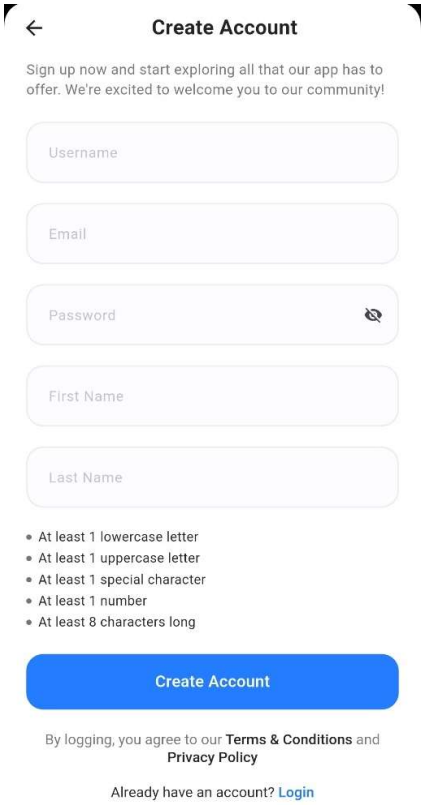

Figure B.3: Patient Sign Up Screen

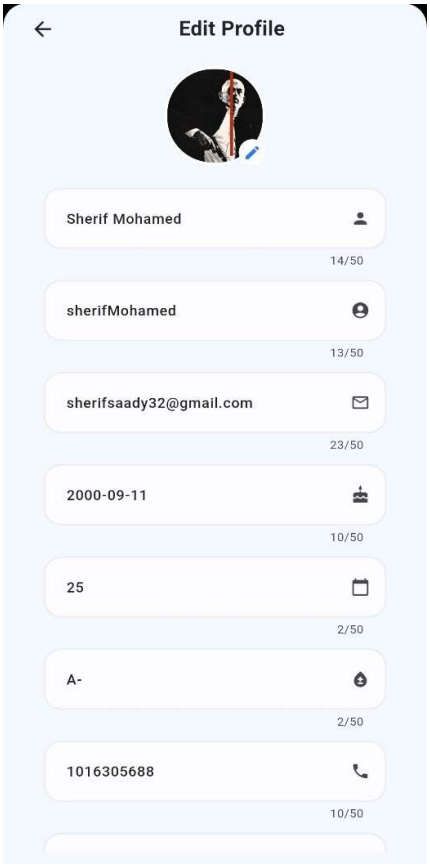

Figure B.4: Patient Profile Data Screen

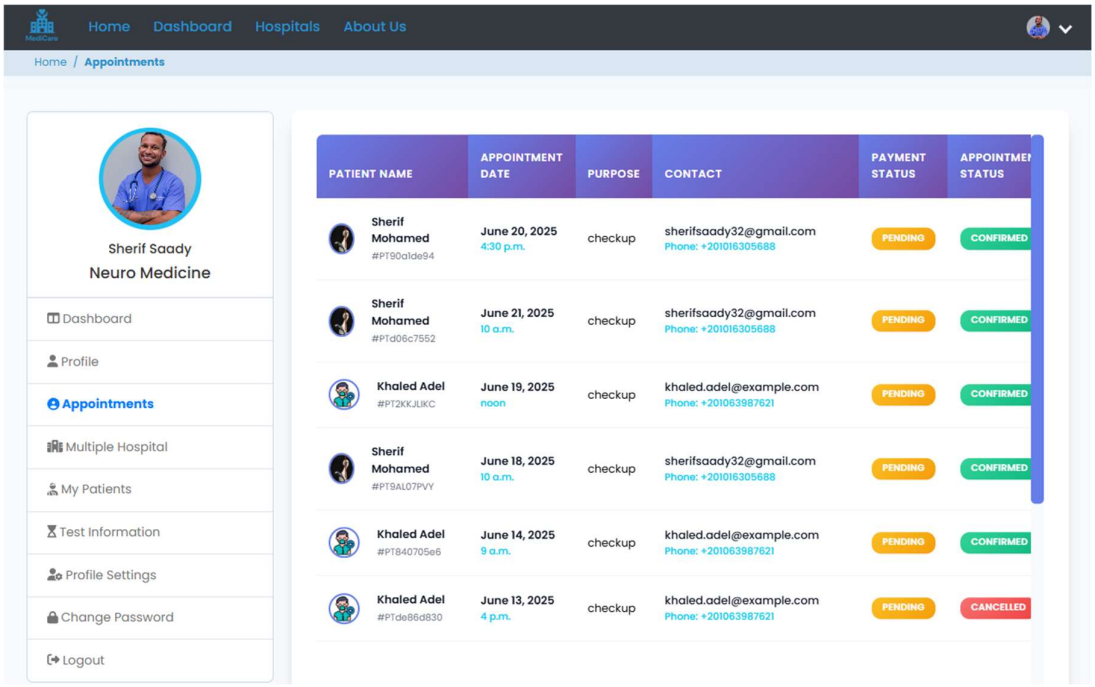

Figure B.5: Doctors Appointments Web Page

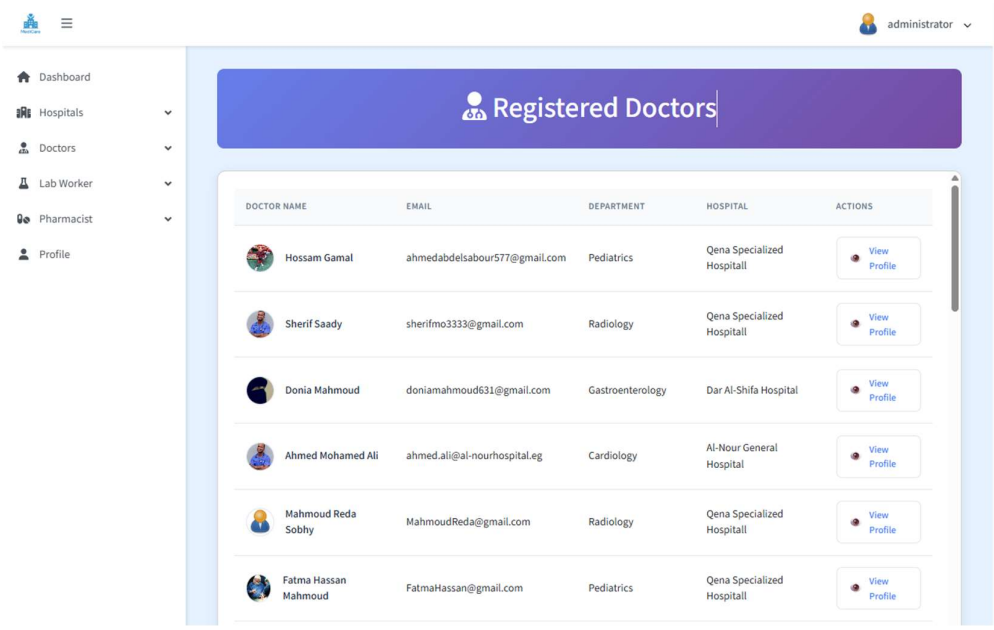

Figure B.6: Admin Registered Doctors List Web Page

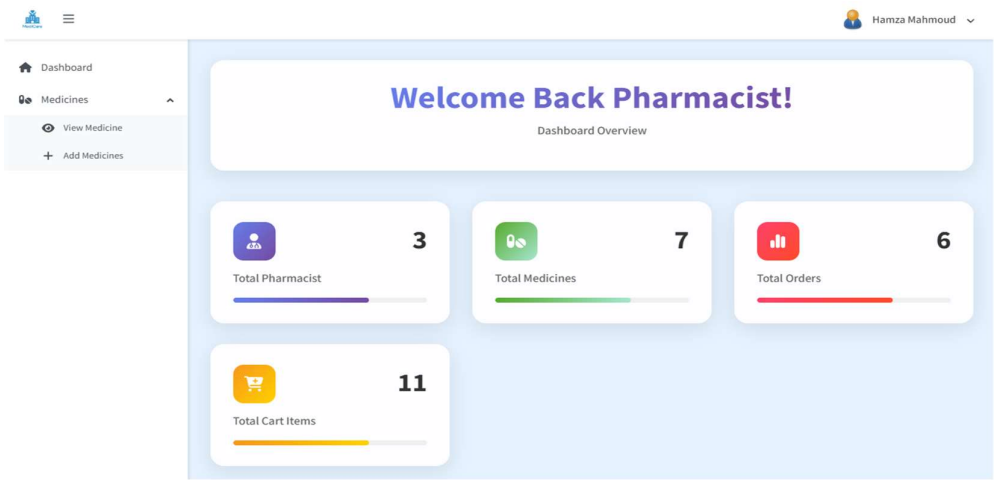

Figure B.7: Pharmacist Dashboard Web Page

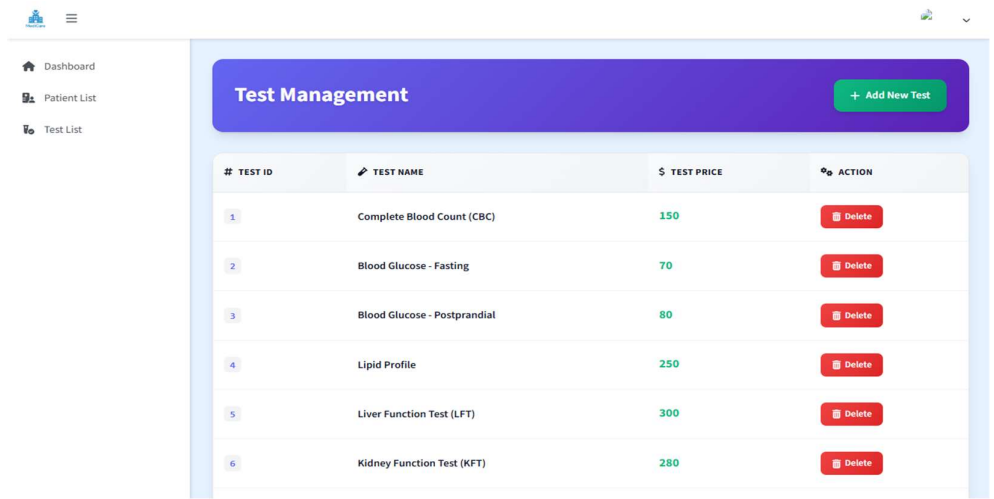

Figure B.8: Lab Worker Test Management Web Page

Appendix C: User Manual

C.1 Patient User Manual (Login as Patient)

1. **How to Register and Login:**
   - Navigate to: https://medicare.pythonanywhere.com
   - To register, click Register as Patient and fill in your basic info (username, email, password).
   - Click Signup
   - To login, go back to the Login as Patient
   - Enter your credentials.
   - Click Login.

2. **How to Search for a Doctor**
   - After logging in, go to Search Doctor tab or on the side bar.
   - Use the search bar to search the doctor's name.
   - Or go to the Hospitals tab then choose the wanted hospital
   - Click on the Department & Doctor List then choose the Department
   - You will see the Doctors in this department.

3. **How to Book an Appointment**
   - Choose the doctor you want to book an appointment with him
   - Click on Book Appointment button.
   - Select the Date, Time, Appointment Type, and write a Message
   - Click Submit to send the request.
   - You can view status updates under the Dashboard tab.

4. **How to View Prescriptions and Lab Reports**
   - Go to Prescriptions tab to view your doctor's orders.
   - Navigate to Medical Records to see test results.

5. **How to Purchase from the Medical Shop**
   - Click on Medical Shop tab.
   - Browse available medicines.
   - Click Add to Cart, then Checkout.
   - Complete payment with PayPal.

C.2 Doctor User Manual (Login as Doctor)

1. How to Manage Your Schedule

- Dashboard shows upcoming appointments.
- Use Appointments tab on sidebar to view your schedule for any date.

2. How to Accept/Reject an Appointment

- On Appointments tab
- Each request shows Accept and Reject buttons.
- Click on any one to Accept or Reject the appointment
- Status will update accordingly.

3. How to Create a Prescription

- Go to Appointments, select a confirmed appointment.
- Click Add Prescription.
- Enter medicines, tests, and additional notes.
- Save to link it to the patient's record.

C.3 Administrator User Manual (Login as Admin)

1. How to Approve a Doctor's Registration

- Navigate to Doctors on sidebar, then Pending.
- Review profiles and click Approve or Reject.

2. How to Add a New Hospital

- Go to Hospital on sidebar, then Add Hospital.
- Scroll down to see Add New Hospital
- Fill out name, address, contact info, and upload image.
- Submit to add it to the active hospital list.

C.4 Django Admin Panel Access

- Go to: https://medicare.pythonanywhere.com/admin
- Login with the highest privilege account
- From here, you can:
  - Manage all database models directly
  - Add/edit/delete users, hospitals, medicines, departments, etc.

Appendix D: Project Timeline / Gantt Chart

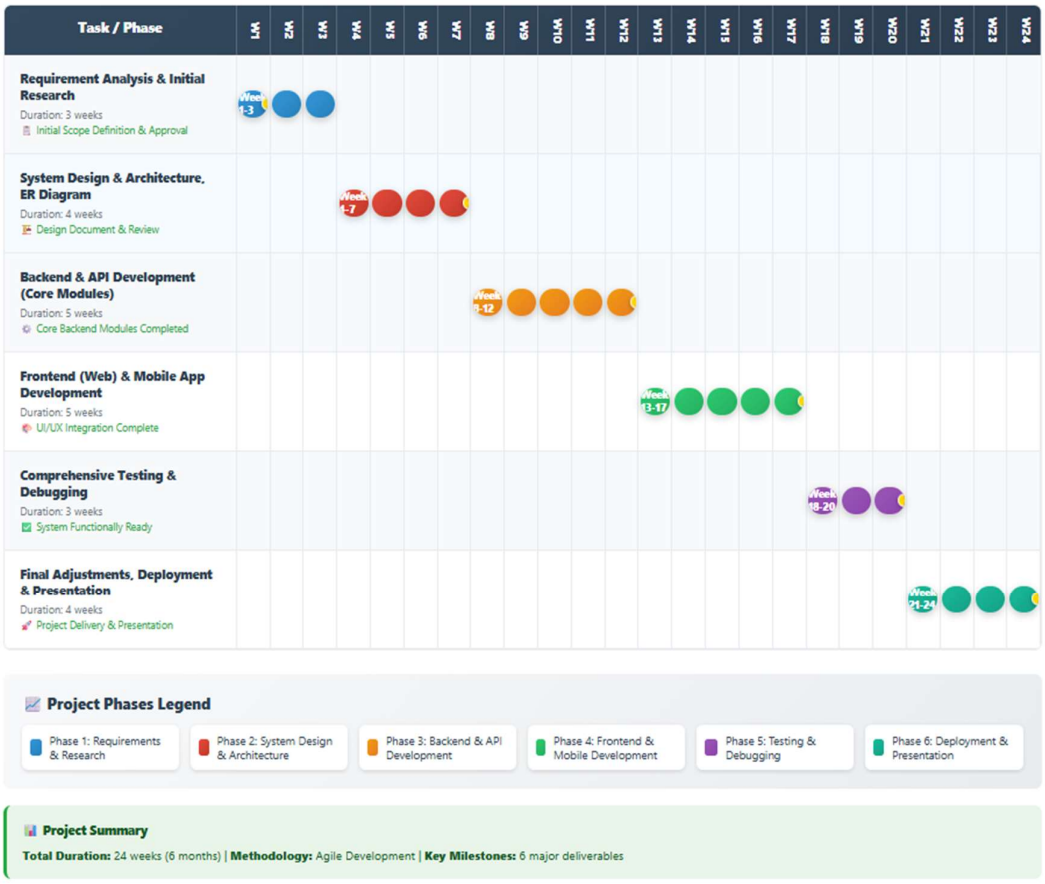

Figure D.9: Gantt Chart for the Project

Appendix F: Full ER Diagram

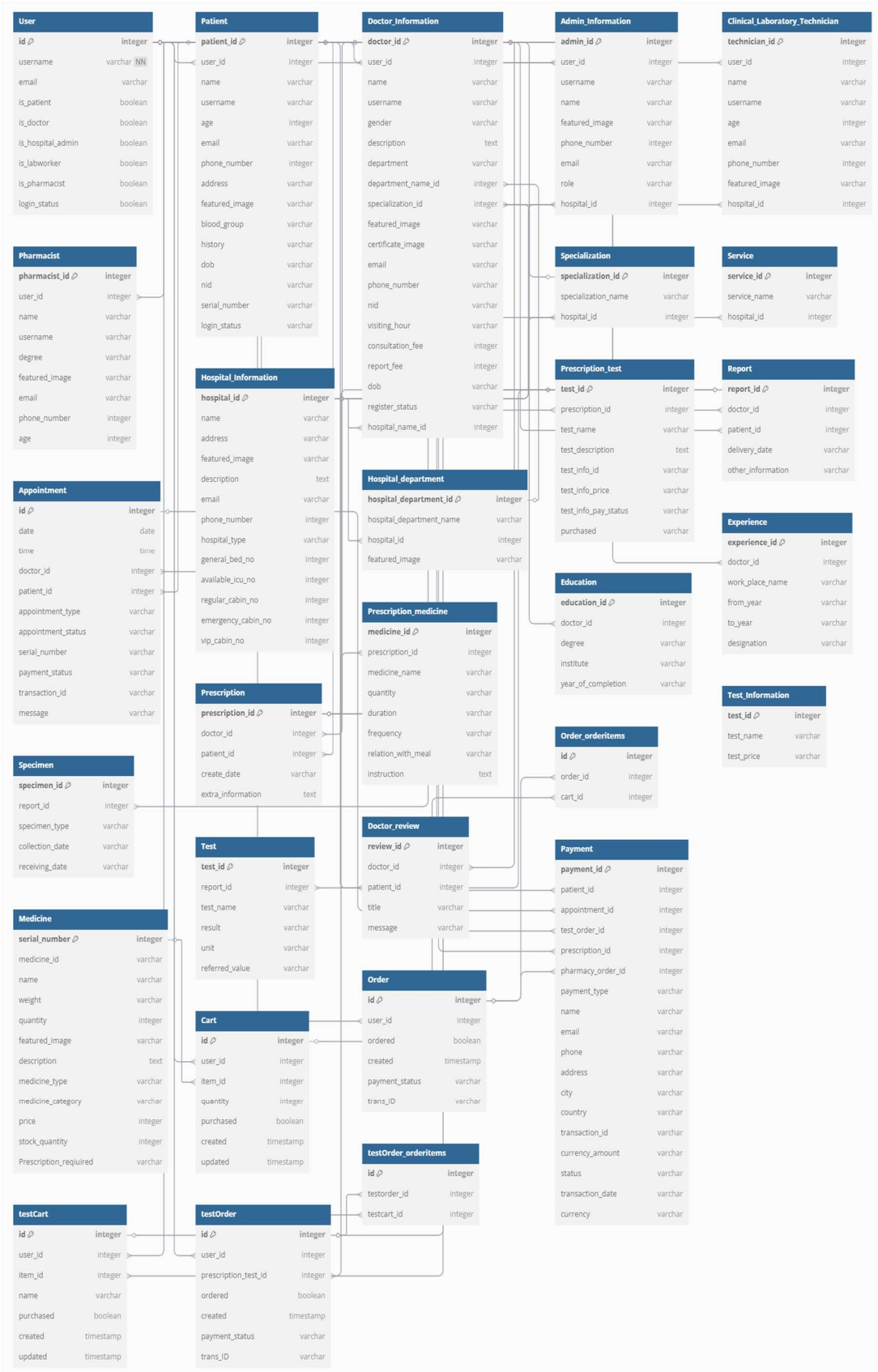

Figure F.10: Full ER Diagram